

High-Level Guiding Principles (Architectural Constitution)

These should remain fairly stable throughout the rest of the project's life.

1. Correctness before convenience.

The game must produce consistent, trustworthy outcomes. Convenience should never come at the expense of correctness.

2. The architecture should be understandable.

Every major concept should have one obvious home, one clear responsibility, and one canonical owner.

3. Modularity is a feature.

Rulesets, AI providers, subsystems, content packs, and future capabilities should extend the chassis rather than require rewriting it.

4. Optimize for evolution.

Adding meaningful features should require minimal architectural coordination and minimal unintended side effects.

5. Minimize computational waste.

Respect player hardware, latency, token costs, context limits, storage, and runtime overhead.

6. Separate concerns aggressively.

Game truth, AI narration, player interaction, replay/evidence, developer tooling, and content should remain distinct wherever practical.

7. Preserve player trust.

The player should be able to understand, question, and recover from AI behavior without hidden state or unexplained outcomes.

8. Design for multiple AI backends.

Hosted models and local models should both be viable targets without fundamentally changing the architecture.

9. Keep the repository intentional.

Development tooling, historical artifacts, audits, tests, and shipping runtime should each have a clearly defined place.

10. Engineering exists to improve the game.

Architectural elegance is valuable only insofar as it improves the tabletop experience.

Architecture Reconciliation Objectives (Cycle Generation)

These are the themes from which we'll generate AR1, AR2, AR3... They are intentionally investigative before they become implementation-oriented.

1. Identify the architecture that actually emerged.

Compare the original design intent with the architecture created through stabilization.

2. Distinguish permanent architecture from temporary scaffolding.

Separate enduring design from historical maintenance mechanisms and transitional compatibility layers.

3. Reevaluate every major architectural concept.

Determine which concepts are essential, which overlap, and which can be unified or removed.

4. Reduce architectural coordination cost.

Find opportunities where a single conceptual change currently requires edits across too many locations.

5. Simplify architectural vocabulary.

Collapse redundant terminology, ownership models, replay concepts, routing language, fallback semantics, and related abstractions where appropriate.

6. Reassess canonical owners.

Confirm that each canonical owner is justified, appropriately scoped, and not accumulating unrelated responsibilities.

7. Clarify architectural boundaries.

Strengthen the interfaces between realization, replay, validation, repair, diagnostics, persistence, rules, AI interaction, and presentation.

8. Evaluate governance itself.

Determine whether governance mechanisms remain proportional to the value they provide or have become unnecessary maintenance overhead.

9. Design for long-term extensibility.

Ensure future additions—rulesets, AI providers, maps, voice, portraits, subsystems, UI improvements—fit naturally into the architecture.

10. Design for efficient AI operation.

Evaluate context management, prompt construction, state retrieval, replay usage, caching, and memory architecture for both hosted and local models.

11. Separate runtime from development infrastructure.

Identify what belongs in the shipping product versus developer tools, audits, diagnostics, historical artifacts, and testing infrastructure.

12. Develop the long-term chassis strategy.

Produce recommendations for the architectural shape that should guide the next several years of feature development.

Architecture Reconciliation — Campaign Roadmap

Naming Convention

Use AR-AA, AR-AB, AR-AC... through AR-AZ, then AR-BA, AR-BB, etc. AR identifies the era. The suffix identifies the individual cycle.

Campaign 1 — Architectural Mapping

Estimated Length: 2–5 cycles

Starting Cycle: AR-AA

Goal

Determine what architecture actually emerged from Foundation Stabilization.

Success

A clear map of major subsystems, ownership surfaces, runtime flows, replay/evidence surfaces, and architectural tensions.

Primary Metric

Architectural Understanding

Campaign 2 — Final Vision Compatibility

Estimated Length: 2–4 cycles

Goal

Compare the emerged architecture against the desired long-term vision: modular rulesets, multiple AI backends, clean handoff, low-context operation, extensibility, UI readiness, and player-trust controls.

Success

A clear statement of which final-vision goals are already supported, which are partially supported, and which require architectural change.

Primary Metric

Vision Alignment

Campaign 3 — Concept Reconciliation

Estimated Length: 3–6 cycles

Goal

Determine which architectural concepts are essential, accidental, overlapping, transitional, or removable.

Success

A justified concept inventory covering replay, realization, ownership, routing, provenance, fallback, validators, repair, sanitizers, diagnostics, and evidence.

Primary Metric

Conceptual Economy

Campaign 4 — Boundary Reconciliation

Estimated Length: 3–6 cycles

Goal

Clarify the long-term boundaries between runtime, rulesets, AI interaction, replay, evidence, diagnostics, persistence, tooling, UI, and content.

Success

A boundary model showing what belongs in the core chassis versus adapters, tools, modules, plugins, or archives.

Primary Metric

Boundary Clarity

Campaign 5 — Coordination Cost Reduction

Estimated Length: 3–7 cycles

Goal

Identify why meaningful architectural changes still require coordinated edits across many surfaces.

Success

A ranked set of simplification opportunities that reduce edit fan-out, canonical-owner pressure, governance overhead, duplicated vocabulary, and compatibility residue.

Primary Metric

Coordination Cost

Campaign 6 — Chassis Strategy

Estimated Length: 4–8 cycles

Goal

Define the intended long-term architectural chassis for Ashen Thrones going into future feature expansion.

Success

A practical architecture recommendation covering package shape, extension points, ruleset interfaces, AI-provider interfaces, replay role, product/runtime separation, and future feature attachment points.

Primary Metric

Chassis Readiness

Campaign Closeout Rule

Each campaign should end when its central question has been answered well enough to make the next campaign meaningful. Cycle ranges are planning estimates, not quotas. If a campaign answers its question early, close it. If it reveals unexpected architectural complexity, extend it.

Campaign Closeout Questions

1. What did we learn?
2. What architectural decisions resulted?
3. What question naturally comes next?
4. Should the next planned campaign still proceed, or should the roadmap adapt?

Era Mindset

Do not assume something is broken. Assume the stabilized architecture contains valuable design. Prove which parts are essential, which are accidental, which are transitional, and which can be elegantly unified.

Campaign 2 — Final Vision Compatibility

Estimated Length: 2–4 cycles

Goal

Evaluate how well the reconciled architecture supports the project's long-term vision. Determine which vision goals are already enabled by the current architecture, which are only partially supported, and which would require meaningful architectural evolution before implementation.

The evaluation should treat the architecture as the product of Foundation Stabilization and Architecture Reconciliation—not as something presumed to be inadequate. The objective is to identify where the architecture already enables the desired future and where genuine architectural gaps remain.

The long-term vision includes:

- Modular ruleset support
- Multiple AI backend support
- Clean repository handoff
- Low-context runtime operation
- Straightforward feature extensibility
- UI and tooling readiness
- Player trust, provenance, and explainability

Success

A clear architectural assessment showing, for each long-term vision objective:

- Whether it is already architecturally supported
- Whether it is partially supported but would benefit from refinement
- Whether it requires additional architectural evolution
- Why the current architecture succeeds or falls short
- Whether future work is primarily documentation, architectural refinement, or implementation

The campaign should conclude with a prioritized view of the remaining architectural work needed before pursuing large scale feature expansion.

Primary Metric

Vision Alignment

Campaign Closeout Rule

End the campaign when the relationship between the current architecture and the desired longterm vision is sufficiently understood to guide future implementation priorities. The goal is not to prove every vision objective is complete, but to determine whether the architecture is already capable of supporting it, or what architectural evolution would still be required.

Campaign Closeout Questions

1. Which parts of the long-term vision are already enabled by the current architecture?
2. Which vision goals require only architectural refinement rather than redesign?
3. Which vision goals would require genuine architectural evolution?
4. What implementation priorities naturally emerge from this assessment?
5. Should the roadmap continue as planned, or should implementation priorities be reordered?

Era Mindset

Do not evaluate the architecture as though it were unfinished. Evaluate it as an intentional system whose capabilities and limitations have now been made explicit. Assume the architecture exists for good reasons until evidence suggests otherwise. The objective is to understand how effectively that architecture supports the project's ultimate vision and to identify where deliberate evolution—not unnecessary redesign—will produce the greatest long-term benefit.

Campaign 5 — Coordination Cost Reduction

Estimated Length: 3–7 cycles

Goal

Evaluate how efficiently the recovered architecture supports future implementation without sacrificing the architectural doctrine established during Campaigns 1–4.

Determine which remaining coordination costs are inherent to preserving correct ownership, evidence, replay integrity, provenance, and governance, and which are simply implementation friction that can be reduced without changing the architecture itself.

The evaluation should treat the architecture as the completed product of Foundation Stabilization, Architecture Reconciliation, Final Vision Compatibility, Concept Reconciliation, and Boundary Reconciliation—not as a system presumed to require additional architectural simplification.

The objective is to understand where implementation can naturally become easier while preserving the recovered architectural boundaries and ownership model.

Areas of investigation include:

- Edit fan-out during feature implementation
- Canonical-owner pressure
- Governance overhead
- Compatibility residue
- Cross-module vocabulary duplication
- Contract publication
- Adapter boundaries
- Test maintenance
- Documentation burden
- Feature implementation workflow

Success

A repository-grounded assessment showing, for each remaining source of coordination cost:

- Whether it is already minimized by the current architecture
- Whether it represents intentional architectural separation
- Whether it is temporary compatibility or transitional residue
- Whether it results primarily from implementation rather than architecture
- Whether meaningful simplification is possible without weakening ownership, replay correctness, provenance, extensibility, or governance
- Why the current architecture either succeeds or imposes unavoidable coordination

The campaign should conclude with a prioritized set of implementation-focused opportunities that reduce development effort while preserving the architectural doctrine established during the previous campaigns.

Primary Metric

Coordination Cost

Campaign Closeout Rule

End the campaign when the remaining coordination costs are sufficiently understood to distinguish between:

- intentional architectural tradeoffs,
- implementation inefficiencies,
- compatibility residue,
- governance overhead,
- and genuine simplification opportunities.

The goal is not to minimize coordination universally, but to determine where coordination is architecturally necessary and where future implementation can safely reduce it.

Campaign Closeout Questions

1. Which coordination costs are an intentional consequence of the recovered architecture?
2. Which coordination costs arise primarily from implementation rather than architectural design?
3. Which coordination costs are expected to disappear as compatibility layers mature or are retired?
4. Which coordination costs should remain because they preserve ownership, evidence quality, replay integrity, or long-term maintainability?
5. What implementation priorities would provide the greatest reduction in coordination effort while preserving the recovered doctrine?
6. Should the roadmap continue as planned, or should implementation priorities be reordered to reduce future coordination costs?

Era Mindset

Do not assume that coordination is evidence of poor architecture.

Evaluate the recovered architecture as an intentionally decomposed system whose ownership boundaries have already been validated. Assume that coordination exists for good reasons unless repository evidence demonstrates otherwise.

The objective is to understand how efficiently future implementation can proceed within the established doctrine, identifying where implementation practices—not architectural redesign—can produce meaningful reductions in development effort while preserving correctness, extensibility, replay integrity, provenance, and governance.

Campaign 3 — Concept Reconciliation

Estimated Length: 3–6 cycles

Goal

Evaluate the conceptual architecture that emerged from Foundation Stabilization, Architecture Reconciliation, and Final Vision Compatibility. Determine which architectural concepts are fundamental to the system, which exist primarily as transitional implementation aids, which intentionally represent separate responsibilities despite appearing similar, and which have become redundant through architectural maturation.

The evaluation should treat the conceptual model as the product of an architecture that has already been shown to support the project's long-term vision—not as an overly complex system presumed to require simplification. The objective is to understand why each concept exists, what architectural responsibility it protects, and whether that responsibility remains necessary.

The primary concepts include, but are not limited to:

- Replay
- Realization
- Ownership
- Authority
- Routing
- Provenance
- Feedback
- Validators
- Repair
- Sanitizers
- Diagnostics
- Evidence

Additional concepts discovered during the campaign should be included where appropriate.

Success

A justified conceptual inventory showing, for each architectural concept:

- Whether it is **Foundational, Essential, Transitional, Overlapping, Accidental, or Removable**
- What architectural responsibility the concept protects
- Which architectural invariants depend upon it
- Whether similar concepts are intentionally distinct or unnecessarily duplicated
- Whether future work is primarily documentation, conceptual clarification, architectural refinement, implementation cleanup, or removal

The campaign should conclude with a concept map that clearly distinguishes the project's permanent conceptual vocabulary from transitional mechanisms and historical artifacts, providing a stable conceptual foundation for future implementation.

Primary Metric

Conceptual Economy

Campaign Closeout Rule

End the campaign when the purpose, ownership, and long-term status of the major architectural concepts are sufficiently understood that future implementation can rely on a stable conceptual vocabulary without repeatedly revisiting terminology, responsibility boundaries, or conceptual intent.

The goal is not to minimize the number of concepts, but to ensure that every retained concept has a clear architectural purpose and every removable concept is supported by evidence rather than preference.

Campaign Closeout Questions

1. Which architectural concepts are truly foundational to the reconciled architecture?
2. Which concepts remain essential but primarily represent implementation or governance concerns?
3. Which concepts are transitional and should eventually disappear as implementation matures?
4. Which concepts appear similar but intentionally protect different architectural responsibilities?
5. Which concepts are genuinely redundant or obsolete?
6. What conceptual vocabulary should future implementation treat as permanent?
7. Should any implementation priorities change based on the conceptual simplification achieved during this campaign?

Era Mindset

Do not evaluate conceptual complexity as though simplicity were inherently preferable.

Campaign 2 established that the reconciled architecture already supports the project's long-term vision without requiring broad architectural evolution. Therefore, assume architectural concepts exist for legitimate reasons unless concrete evidence demonstrates otherwise.

Treat conceptual overlap as a hypothesis rather than a defect. Similar concepts may intentionally represent different authority boundaries, lifecycle stages, governance responsibilities, runtime behaviors, or evidence models.

Favor conceptual clarity over conceptual minimalism. The objective is to eliminate accidental complexity while preserving the intentional separation of responsibilities that gives the architecture its robustness.

Campaign 6 — Chassis Strategy

Estimated Length: 4–8 cycles

Goal

Define the long-term architectural evolution strategy for Ashen Thrones now that the architecture has been recovered, reconciled, and operationalized through Campaigns 1–5.

The objective is not to redesign the architecture, but to determine which structural decisions should remain stable, which extension points are intentionally permanent, and where future architectural evolution should occur before large-scale feature expansion.

The evaluation should treat the current architecture as an intentional, production-ready foundation. The purpose is to establish the long-term chassis that future gameplay systems, AI capabilities, tooling, and content can safely build upon.

The architectural chassis should evaluate topics including:

- Package and repository organization
- Stable architectural boundaries
- Backend/provider extension strategy
- Ruleset extension strategy
- Replay and provenance as architectural infrastructure
- Product/runtime separation
- Executable versus documentation governance
- Public extension points
- Internal implementation boundaries
- Long-term feature attachment strategy
- Modularity expectations
- Evolution versus permanence of existing systems

Success

A practical architectural strategy describing:

- Which architectural boundaries should now be considered stable
- Which systems are intended extension points
- Which systems should remain internal implementation details
- Where future architectural evolution is expected
- Which future work is primarily implementation rather than architectural change
- How future features should attach to the recovered architecture without increasing architectural complexity

The campaign should conclude with a long-term chassis strategy that guides future feature expansion while preserving the architectural consistency established during the first five campaigns.

Primary Metric

Chassis Readiness

Campaign Closeout Rule

End the campaign when the long-term architectural shape of the repository is sufficiently understood that future implementation can proceed without repeatedly reconsidering foundational structural decisions.

The objective is not to eliminate future evolution, but to distinguish between:

- stable architectural doctrine,
- expected extension mechanisms,
- future implementation work,
- and areas where deliberate architectural evolution is still anticipated.

Campaign Closeout Questions

1. Which architectural boundaries should now be considered effectively permanent?
2. Which systems are the intended extension points for future capabilities?
3. Which architectural responsibilities should remain internal implementation details?
4. What future work represents implementation rather than architecture?
5. Which architectural areas are intentionally expected to evolve over time?
6. Does the current chassis support long-term expansion without increasing coordination cost?
7. Should campaign focus primarily on implementation, or is another major architectural challenge still warranted?

Campaign 4 — Boundary Reconciliation

Estimated Length: 3–6 cycles

Goal

Evaluate how well the reconciled architectural model defines the long-term boundaries between the project's major responsibilities. Determine which boundaries are already stable, which intentionally overlap, which remain transitional, and which may still require architectural evolution before large-scale feature expansion.

The evaluation should treat the boundary model as the product of Foundation Stabilization, Architecture Reconciliation, Final Vision Compatibility, and Concept Reconciliation—not as something presumed to be incomplete. The objective is to validate whether the architecture now expresses clear long-term responsibility boundaries and to identify the remaining areas where boundary evolution, rather than conceptual discovery, may still be warranted.

The major architectural boundary families include:

- Runtime orchestration
- Domain simulation
- Ruleset architecture
- AI interaction
- Final emission
- Replay and projection
- Evidence and provenance
- Diagnostics and telemetry
- Persistence
- Governance
- Tooling
- UI integration
- Plugins and adapters
- Content

Success

A comprehensive architectural assessment showing, for each major boundary:

- Whether the boundary is already architecturally stable
- Whether it intentionally overlaps adjacent responsibilities
- Whether remaining overlap is transitional rather than conceptual
- Whether implementation responsibilities naturally align with the boundary
- Whether future architectural evolution is likely to change the boundary itself or merely its implementation

The campaign should conclude with a coherent long-term boundary model describing what permanently belongs within the architectural chassis, what belongs in specialized subsystems, and what exists primarily as compatibility or implementation support.

Primary Metric

Boundary Clarity

Campaign Closeout Rule

End the campaign when the long-term architectural boundaries are sufficiently understood that future implementation can proceed without repeatedly reconsidering ownership, placement, or architectural responsibility. The objective is not to freeze implementation forever, but to establish durable architectural boundaries that future work can confidently build upon.

Campaign Closeout Questions

1. Which architectural boundaries are now effectively permanent?
2. Which boundaries intentionally overlap, and why?
3. Which remaining boundary ambiguity is transitional rather than conceptual?
4. Which future implementation work is unlikely to alter the architecture itself?
5. Which areas may legitimately require future architectural evolution?
6. What responsibilities permanently belong in the core runtime chassis?
7. What responsibilities belong in specialized runtime subsystems?
8. What responsibilities belong in tooling, adapters, plugins, UI, or external systems?
9. Does the current architecture now possess a sufficiently stable boundary model to support long-term feature expansion?

Era Mindset

Do not search for architectural boundaries as though they remain undiscovered. Evaluate the boundary model as an intentionally evolved architectural system whose major concepts, responsibilities, and vocabulary have already been reconciled through the previous campaigns. Assume boundaries are correct unless evidence demonstrates a genuine architectural inconsistency. The objective is to determine whether those boundaries are sufficiently stable to guide years of future implementation, while distinguishing permanent architectural structure from transitional implementation detail.

Era Mindset

Do not evaluate the architecture as though a better chassis necessarily exists.

Campaigns 1–5 established that the architecture is coherent, internally consistent, and maintainable.

Assume the current structural decisions are intentional unless repository evidence demonstrates a

compelling reason to evolve them.

The objective is to identify which parts of the architecture should now become long-term doctrine,

which should remain explicit extension mechanisms, and where future evolution should occur

deliberately rather than incrementally.

Future architectural work should preserve the simplicity, ownership clarity, and coordination

improvements achieved during Campaign 5 while enabling years of feature expansion without repeated

structural redesign.

C1: Architectural Mapping

Wednesday, July 1, 2026 8:15 PM

AR-AA — Architectural Mapping Discovery

Goal:
Gather enough architectural evidence to generate the first Architectural Mapping cycle block for Campaign 1.

Context:
We are entering the Architecture Reconciliation era after Foundation Stabilization. Do not assume the code is broken. The goal is to determine what architecture actually emerged: major subsystems, ownership surfaces, runtime flows, replay/evidence surfaces, and architectural tensions.

Please inspect the repository and produce a discovery report that answers the following.

1. Major Subsystems
Identify the major architectural subsystems currently present. For each, include:

- Purpose
- Key files/modules/classes/functions
- Inputs and outputs
- Whether it appears stable, transitional, duplicated, or unclear

2. Ownership Surface
Identify where authority/ownership is currently assigned or enforced, especially around:

- State authority
- Runtime authority
- Realization authority
- Replay authority
- API/emission authority
- Falback authority
- Test/provenance/evidence authority

3. Runtime Flows
Trace the main runtime flows, especially:

- Campaign/session start
- Opening scene generation/realization
- Falback handling
- Upstream response handling
- Final emission
- Replay/projection
- Provenance/evidence capture

For each flow, list the files involved and summarize the sequence.

4. Replay and Evidence Surfaces
Identify all files/texts/docs that define or validate:

- Determinism
- Replay boundaries
- Final emission projection
- Provenance
- Evidence capture
- Runtime vs replay separation

5. Architectural Tensions
Look for tensions, overlaps, or unresolved boundaries, examples:

- Multiple modules appearing to own the same decision
- Runtime code doing replay work
- Falbacks bypassing provenance
- API surfaces leaking internal authority
- Tests implying architecture not yet documented
- Documentation disagreeing with implementation

Do not fix anything yet unless there is a trivial typo in the report itself. This is discovery only.

6. Candidate File List for GPT
At the end, provide a concise list of the most relevant files I should pass to GPT for the next cycle. Prioritize files that are necessary to understand architecture, not every related file.

7. Suggested AR-AA Cycle Focus
Based on your findings, recommend the best focus for AR-AA:

- Subsystem map
- Runtime flow map
- Authority/ownership ledger
- Replay/evidence map
- Architectural tension audit
- Other

Deliverable:
Create a markdown file named:

AR-AA_architectural_mapping_discovery.md

Format it with clear headings and enough detail that GPT can generate the next implementation/reconciliation instruction block from it.

AR-AE — Architecture Conformance Assessment

Goal:
Evaluate how closely the current implementation conforms to the target architecture doctrine established during AR-AD.

Context:
AR-AA discovered the architecture.
AR-AB mapped ownership.

AR-AC reconciled permanent and transitional boundaries.
AR-AD established the target architectural doctrine.

This cycle measures implementation against that doctrine.

This remains an Architecture Reconciliation cycle.

Do not perform runtime refactors.

Inputs:

- AR-AA_architectural_mapping_discovery.md
- AR-AD_runtime_authority_and_replay_boundary_map.md
- AR-AC_architectural_ownership_reconciliation.md
- AR-AD_target_architecture_doctrine.md

Tasks

1. Architecture Conformance Matrix

For each major subsystem evaluate:

- Current implementation
- Target doctrine
- Degree of alignment

Classify each as:

- Fully aligned

- Mostly aligned

- Transitional

- Significant divergence

Cover at minimum:

- Runtime orchestration
- State authority
- CTR lifecycle
- Prompt construction
- GPT routing
- Retry/falback
- Opening realization
- Final emission
- Persistence
- Replay
- Provenance
- Governance

2. Dependency Conformance

Review whether implementation follows the dependency rules established in AR-AD.

Identify:

- Confirmed compliant relationships
- Transitional exceptions
- Architectural drift
- Potential future risks

Do not recommend implementation changes.

Simply evaluate architectural alignment.

3. Transitional Responsibility Review

Review every transitional responsibility identified in AR-AD.

For each determine:

- Still necessary
- Ready for future simplification
- Requires additional evidence before simplification

Do not propose code movement.

Only assess architectural readiness.

4. Architectural Strengths

Identify architecture that should be considered complete and stable.

Focus on systems that should now be treated as long-term foundations.

5. Remaining Architectural Risks

Identify remaining areas of architectural uncertainty.

These should be conceptual rather than implementation bugs.

Examples:

- Ownership ambiguity
- Layer ambiguity
- Evidence ambiguity

AR-AB — Runtime Authority and Replay Boundary Map

Goal:
Create a compact current-state architecture map for the second Architecture Reconciliation cycle.

Context:
AR-AA completed architectural discovery and identified that the architecture is stable but broad. "game api" has emerged as the runtime spine, while final emission, falback behavior, replay, and evidence systems are heavily governed. The objective now is to reconcile current implementation ownership with the architectural model that emerged during Foundation Stabilization.

Use:
- AR-AA_architectural_mapping_discovery.md

Do not refactor yet. This is still an architectural reconciliation cycle.

Tasks

1. Runtime Authority Ledger

Create a concise ownership ledger separating:

- Runtime owner
- Content author
- Selector / applicator
- Provenance packager
- Replay projection owner
- Protected replay acceptance owner
- Relevant files
- Architectural notes

Apply this to:

- Campaign start
- Shared resolved-turn pipeline
- CTR lifecycle
- Opening scene realization
- GPT retry pipeline
- Upstream falback handling
- Final emission
- Runtime lineage projection
- Protected replay projection

2. Start Campaign Runtime Flow

Trace "/api/start_campaign" from request to final response.

Include:

- Request entry
- Preflight validation
- Runtime state loading
- Campaign-start eligibility
- Opening scene bootstrap
- Authoritative state mutation
- CTR construction
- Prompt construction
- GPT execution
- Retry/falback handling
- Final emission
- Persistence
- Logging
- Response construction
- Replay/evidence surfaces touched

The goal is to identify the architectural ownership of each stage —not to redesign it.

3. Replay Boundary Map

Document the replay architecture, clearly separating:

- Runtime diagnostic projection
- Protected replay projection
- Governance surfaces
- Advisory/audit tooling

Explicitly document dependency direction.

Protected replay may consume runtime diagnostics.

Runtime should never depend on protected replay acceptance structures.

4. Falback Ownership Map

Map falback ownership without collapsing responsibilities.

Separate:

- Falback content author
- Falback selector
- Falback applicator
- Provenance packager
- Final emission recorder
- Replay observer

Document where these ownership boundaries intentionally overlap.

5. Architectural Tension Classification

For each major architectural tension, classify it as one of:

- Essential design
- Transitional design
- Accidental overlap
- Future cleanup candidate

At minimum evaluate:

- "game api" breadth
- Final emission semantic pressure
- Opening scene ownership
- Falback ownership fanout
- Runtime vs replay separation
- Evidence artifact fanout

Deliverable

Create:

AR-AB_runtime_authority_and_replay_boundary_map.md

- Executive summary
- Runtime authority ledger
- Start campaign runtime flow map
- Replay boundary map
- Falback ownership map
- Architectural tension classification
- Recommended focus for AR-AC

Validation

This remains a documentation and architectural reconciliation cycle.

Do not perform broad refactors.

Only run lightweight validation necessary to ensure no implementation was unintentionally modified.

AR-AF — Architecture Reconciliation Synthesis

Goal:
Synthesize the results of Campaign 1 into a concise architectural reference that future implementation campaigns can treat as the baseline architecture.

Context

Architecture Reconciliation has now completed:

- AR-AA — Architectural Discovery
- AR-AB — Runtime Authority Mapping
- AR-AC — Ownership Reconciliation
- AR-AD — Target Architecture Doctrine
- AR-AE — Architecture Conformance Assessment

The implementation has been assessed as substantially conforming to the doctrine.

The objective of this cycle is to consolidate everything learned into one coherent architectural baseline.

This remains a documentation-only cycle.

Do not modify runtime behavior.

Inputs

- AR-AA_architectural_mapping_discovery.md
- AR-AB_runtime_authority_and_replay_boundary_map.md
- AR-AC_architectural_ownership_reconciliation.md
- AR-AD_target_architecture_doctrine.md
- AR-AE_architecture_conformance_assessment.md

Tasks

1. Architectural Baseline

Produce a concise description of the architecture that should be assumed by future implementation work.

This should describe:

- the runtime transaction model
- the layer model
- permanent ownership boundaries
- replay boundaries
- evidence model
- governance model

Avoid historical discussion.

AR-AC — Architectural Ownership Reconciliation

Goal:
Reconcile the current architectural ownership model into a clearer target architecture while preserving all existing behavior.

Context:
AR-AA established what architecture emerged from Foundation Stabilization.

AR-AB identified the current runtime ownership model, replay boundaries, falback ownership, and major architectural tensions.

The architecture is not considered broken. The objective is to determine which ownership boundaries should become permanent architecture, which are transitional, and where future simplification should occur.

Inputs:

- AR-AA_architectural_mapping_discovery.md
- AR-AB_runtime_authority_and_replay_boundary_map.md

This remains an architectural reconciliation cycle.

Do not perform runtime refactors.

Tasks

1. Ownership Reconciliation Matrix

For every major architectural subsystem, produce a reconciliation table containing:

- Current owner
- Intended long-term owner
- Current implementation quality
- Architectural justification
- Recommendation

Classify each ownership boundary as:

- Permanent
- Transitional
- Candidate for consolidation
- Candidate for extraction
- Candidate for retirement

Cover at minimum:

- Runtime orchestration
- State authority
- CTR lifecycle
- Prompt construction
- GPT execution
- Retry/falback
- Opening realization
- Final emission
- Replay projection
- Protected replay
- Provenance
- Persistence
- Logging

2. Architectural Layer Model

Construct a layered view of the architecture.

Identify which modules belong primarily to:

- Request/API Layer
- Runtime Orchestration Layer
- Domain Simulation Layer
- Narrative Layer
- Emission Layer
- Persistence Layer
- Replay/Evidence Layer
- Governance Layer

For each layer:

- Primary responsibilities
- Allowed dependencies
- Forbidden dependencies
- Canonical ownership

3. Architectural Dependency Review

Identify architectural dependencies that appear:

- Appropriate
- Transitional
- Potentially circular
- Too broad
- Too tightly coupled

Focus on architectural relationships other than implementation details.

Do not recommend refactoring unless the architectural benefit is clear.

4. Permanent vs Transitional Design

Review every major architectural tension identified during AR-AB.

For each, determine whether it should become:

- Permanent architecture
- Transitional architecture
- Future simplification target

Explain why.

5. Candidate Simplification Opportunities

Without proposing implementation work, identify architectural simplifications that would improve clarity.

Examples include:

- Better ownership boundaries
- Cleaner layering
- Reduced conceptual overlap
- Clearer responsibility separation
- Improved documentation structure

Do not recommend code movement simply because something is large.

Only recommend simplification when it improves architectural understanding.

Deliverable

Create:

AR-AC_architectural_ownership_reconciliation.md

Include:

- Executive summary
- Ownership reconciliation matrix
- Architectural layer model
- Dependency review
- Permanent vs transitional architecture
- Candidate simplification opportunities
- Recommended focus for AR-AD

Validation

This is still a documentation and architecture cycle.

Do not modify runtime behavior.

If validation is performed, limit it to confirming no implementation files were unintentionally changed.

AR-AD — Target Architecture Doctrine

Goal:
Define the project's target architectural doctrine by consolidating the reconciled ownership model into a concise, durable architectural specification.

Context:

AR-AA mapped the architecture.

AR-AB mapped runtime ownership.

AR-AC reconciled permanent versus transitional ownership.

The next objective is to produce a concise architectural doctrine describing how the system is intended to function going forward.

This remains an Architecture Reconciliation cycle.

Do not perform runtime refactors.

Inputs

- AR-AA_architectural_mapping_discovery.md
- AR-AB_runtime_authority_and_replay_boundary_map.md
- AR-AC_architectural_ownership_reconciliation.md

Tasks

1. Target Architecture Doctrine

Produce a concise architectural doctrine answering:

- What are the permanent architectural principles?
- What responsibilities belong to each architectural layer?
- Which ownership boundaries should be considered stable?
- Which boundaries intentionally overlap?
- Which responsibilities remain transitional?

The doctrine should be understandable without reading previous reconciliation reports.

2. Canonical Layer Specification

For each architectural layer define:

- Purpose
- Canonical owner(s)
- Allowed responsibilities
- Responsibilities explicitly outside the layer
- Allowed dependencies
- Forbidden dependencies

Layers should include:

- Request/API
- Runtime Orchestration
- Domain Simulation
- Narrative
- Final Emission
- Persistence
- Replay/Evidence
- Governance

3. Multi-Axis Ownership Doctrine

Document the architectural ownership vocabulary.

Define each ownership axis:

- Runtime owner
- Domain owner
- Content author
- Selector
- Applicator
- Provenance packager
- Runtime diagnostic owner
- Protected replay owner

Explain why multiple ownership axes exist and when they should be used.

4. Canonicality Index

Classify every major architectural surface as one of:

- Runtime truth
- Runtime projection
- Protected acceptance
- Governance contract
- Generated artifact
- Advisory artifact

The objective is to identify which documents and systems are authoritative.

5. Transitional Architecture Register

Create a register of architectural elements expected to evolve.

For each:

- Current purpose
- Reason it is transitional
- Desired long-term direction
- Preconditions before simplification

Do not recommend implementation work.

Only identify architectural evolution.

6. Architectural Design Principles

Summarize the core principles that should guide future development.

Examples include:

- Runtime truth before narration
- Single transaction spine
- One-way replay dependency
- Evidence without ownership confusion
- Separation of runtime behavior and governance
- Documentation over premature refactoring

Produce a concise list suitable for long-term project documentation.

Deliverable

Create:

AR-AD_target_architecture_doctrine.md

Include:

- Executive summary
- Target architecture doctrine
- Layer specification
- Multi-axis ownership doctrine
- Canonicality index
- Transitional architecture register
- Architectural design principles
- Recommended focus for AR-AE

Validation

This is a documentation-only cycle.

Do not modify runtime behavior.

If validation is performed, limit it to ensuring no implementation files were unintentionally changed.

Identify remaining areas of architectural uncertainty.

These should be conceptual rather than implementation bugs.

Examples:

- Ownership ambiguity
- Layer ambiguity
- Evidence ambiguity
- Dependency ambiguity
- Terminology ambiguity

6. Campaign Readiness Assessment

Determine whether Campaign 1's central question has been answered.

Specifically answer:

- Do we now understand the architecture well enough to begin architectural refinement work?

If not:

- What architectural knowledge is still missing?

Deliverable

Create:

AR-AE_architecture_conformance_assessment.md

Include:

- Executive summary
- Architecture conformance matrix
- Dependency conformance review
- Transitional responsibility review
- Architectural strengths
- Remaining architectural risks
- Campaign readiness assessment
- Recommendation for AR-AE

Validation

This remains a documentation-only cycle.

Do not modify runtime behavior.

If validation is performed, limit it to confirming no implementation files were unintentionally changed.

- the runtime transaction model
- the layer model
- permanent ownership boundaries
- replay boundaries
- evidence model
- governance model

Avoid historical discussion.

Describe the architecture as it exists today.

2. Architectural Invariants

List the architectural rules that future work should preserve.

Examples include:

- Runtime truth before narration
- One transaction spine
- One-way replay dependency
- CTR ownership
- Final-emission boundary
- Multi-axis ownership

Only include principles that should remain stable for the foreseeable future.

3. Architectural Flexibility Register

Identify the architectural areas intentionally left open for future evolution.

For each identify:

- why flexibility exists
- what assumptions future work may safely change
- what assumptions should remain fixed

4. Campaign 1 Deliverables

Summarize the durable outputs produced during Architecture Reconciliation.

Identify which reports should become long-term references and which should simply remain historical artifacts.

5. Campaign Closeout Assessment

Answer explicitly:

- What did Campaign 1 teach us?
- What architectural decisions resulted?
- What question naturally comes next?
- Should the planned roadmap continue unchanged or adapt?

Base the recommendation on the evidence gathered throughout Campaign 1.

Deliverable

Create:

AR-AF_architecture_reconciliation_synthesis.md

Include:

- Executive summary
- Architectural baseline
- Architectural invariants
- Architectural flexibility register
- Durable campaign outputs
- Campaign closeout assessment
- Recommendation for the next campaign

Validation

This is documentation only.

Do not modify runtime behavior.

If validation is performed, limit it to ensuring no implementation files were unintentionally changed.

C2: Final Vision Compatibility

Tuesday, July 14, 2026 5:35 PM

You are beginning Campaign 2 of the Architecture Reconciliation era.

Campaign 2 — Final Vision Compatibility Campaign Goal

Evaluate how well the reconciled architecture supports the project's long-term vision.

Determine which vision goals are:

- Already enabled by the current architecture
- Partially supported and primarily require refinement
- Not yet supported and require meaningful architectural evolution

Treat the architecture as the intentional product of Foundation Stabilization and Campaign 1 — Architectural Mapping. Do not begin from the assumption that the architecture is inadequate, incomplete, or in need of redesign.

The objective of this cycle is evidence gathering and assessment preparation. Do not implement architectural changes unless a very small, clearly necessary documentation correction is required to make the evidence understandable.

Long-Term Vision Objectives

Assess the architecture against the following objectives:

1. Modular ruleset support
2. Multiple AI backend support
3. Clean repository handoff
4. Low-context runtime operation
5. Straightforward feature extensibility
6. UI and tooling readiness
7. Player trust, provenance, and explainability

Current Cycle Purpose

Perform a repository-wide discovery pass that gathers the evidence needed to evaluate each vision objective.

The resulting artifact should allow a later cycle to make defensible architectural judgments without needing to rediscover the repository.

This is not yet the final compatibility verdict. It is the evidence and issue-framing stage.

Required Inputs

Begin by locating and reviewing the outputs of Campaign 1 — Architectural Mapping, especially files equivalent to:

- Architectural inventory or system map
- Runtime authority and ownership mapping
- Dependency or boundary analysis
- Target architecture doctrine
- Architecture conformance assessment
- Architecture reconciliation synthesis

Expected recent files may include:

- AR-AA.*
- AR-AB.*
- AR-AC.*
- AR-AD_target_architecture_doctrine.md
- AR-AE_architecture_conformance_assessment.md
- AR-AF_architecture_reconciliation_synthesis.md

Also locate relevant Foundation Stabilization closeout artifacts, architecture doctrine, ownership ledgers, state authority documents, provenance documents, replay documentation, runtime flow documentation, and repository handoff or operational guidance.

Do not assume filenames are exact. Search by content and purpose.

Discovery Tasks

1. Establish the Architectural Baseline

Summarize the reconciled architecture that is being evaluated.

Identify:

- Major architectural layers
- Runtime flow
- Authority boundaries
- State ownership boundaries
- Core extension points
- Backend-facing interfaces
- Ruleset-facing interfaces
- Emission, replay, realization, and provenance boundaries
- Existing public or semi-public APIs
- Repository-level operational assumptions
- Areas intentionally constrained during Foundation Stabilization

Use Campaign 1 conclusions as the primary baseline. Do not reopen settled architectural questions unless current repository evidence materially contradicts those conclusions.

2. Build a Vision-to-Architecture Evidence Matrix

For each long-term vision objective, gather concrete supporting and limiting evidence.

Use the following classification scale provisionally:

- SUPPORTED
 - The architecture already contains the necessary boundaries, contracts, ownership model, and extension points.
 - Remaining work would mainly be implementation or documentation.
- PARTIALLY SUPPORTED
 - The architecture contains a viable foundation, but one or more boundaries, contracts, abstractions, or ownership rules need refinement.
 - The objective does not appear to require broad redesign.
- ARCHITECTURAL_EVOLUTION REQUIRED
 - The current architecture lacks a necessary structural capability.
 - Achieving the objective would require new architectural concepts, meaningful boundary changes, or changes to authority and ownership.
- INSUFFICIENT EVIDENCE
 - The repository does not currently provide enough evidence for a reliable classification.

For every objective, record:

- Provisional classification
- Supporting evidence
- Limiting evidence
- Relevant files and symbols
- Existing abstractions or extension points
- Missing abstractions or contracts
- Whether the remaining work appears to be:
 - Documentation
 - Implementation
 - Local architectural refinement
 - Cross-cutting architectural evolution
- Confidence level
- Open questions

3. Evaluate Modular Ruleset Support

Investigate whether game-system-specific behavior can be introduced, replaced, or isolated without destabilizing core runtime behavior.

Look for:

- Ruleset interfaces, adapters, registries, protocols, or service boundaries
- Domain logic embedded directly in orchestration or runtime infrastructure
- PF1e-specific assumptions in otherwise generic modules
- Rules data separated from runtime authority
- Combat, conditions, magic, action resolution, checks, inventory, progression, and similar subsystem boundaries
- Whether rulesets can be independently packaged, selected, initialized, and tested
- Whether multiple rulesets could coexist in the repository
- Whether adding a ruleset would require editing central orchestration code
- Whether ruleset identity and version are represented in runtime state, replay, or provenance

Distinguish between:

- Missing ruleset implementations
- Missing ruleset documentation
- Missing architectural seams required for ruleset modularity

4. Evaluate Multiple AI Backend Support

Investigate whether the runtime can support multiple model providers or inference backends without changing domain behavior.

Look for:

- Provider interfaces or backend abstractions
- Request and response normalization
- Backend-specific code leaking into orchestration
- Model capability assumptions
- Tool-call normalization
- Structured output handling
- Streaming assumptions
- Retry, timeout, fallback, and degradation behavior
- Backend identity captured in provenance
- Configuration-driven backend selection
- Support for local and remote inference
- *Whether determinism follows either demand or a specific provider*

You are continuing Campaign 2 of the Architecture Reconciliation era.

AR-AH — Final Vision Compatibility Assessment

Campaign Context

The previous cycle produced:

AR-AG_final_vision_compatibility_discovery.md

That report completed the repository-wide evidence discovery needed to evaluate the reconciled architecture against

the project's long-term vision.

Its provisional findings were:

- SUPPORTED
 - Low-context runtime operation
 - Player trust, provenance, and explainability
- PARTIALLY SUPPORTED
 - Modular ruleset support
 - Multiple AI backend support
 - Clean repository handoff
 - Straightforward feature extensibility
 - UI and tooling readiness
- No objective currently justified a finding of:
 - ARCHITECTURAL_EVOLUTION REQUIRED

The report also identified several cross-cutting gaps:

- No explicit ruleset packaging, selection, or version contract
- No provider-neutral AI backend interface
- No concise durable repository handoff index
- No versioned public application API or event model
- No consolidated versioning, migration, or capability-negotiation strategy
- No complete player-facing explanation and challenge workflow

This cycle must convert that evidence into the campaign's actual architectural assessment.

Cycle Goal

Issue a defensible final-vision compatibility assessment for each long-term vision objective.

Determine:

1. What the current architecture already enables
2. What requires only documentation or implementation
3. What requires local architectural refinement
4. What, if anything, requires genuine architectural evolution
5. Which remaining work should be prioritized before large-scale feature expansion
6. Whether the roadmap should continue as planned or be reordered

This is an architectural judgment cycle.

Do not implement the identified changes.

Do not redesign the architecture merely because some product capabilities are not yet implemented.

Required Inputs

Review:

- AR-AG_final_vision_compatibility_discovery.md
- AR-AF_architecture_reconciliation_synthesis.md
- AR-AD_target_architecture_doctrine.md
- AR-AE_architecture_conformance_assessment.md

Consult underlying repository files only where needed to:

- Verify a material AR-AG conclusion
- Resolve contradictory evidence
- Improve confidence in a classification
- Determine whether a limitation is architectural or merely implementational

Do not repeat the full repository discovery pass.

AR-AG is the primary evidence inventory for this cycle.

Long-Term Vision Objectives

Assess:

1. Modular ruleset support
2. Multiple AI backend support
3. Clean repository handoff
4. Low-context runtime operation
5. Straightforward feature extensibility
6. UI and tooling readiness
7. Player trust, provenance, and explainability

Assessment Standard

Use the following final classifications.

ARCHITECTURALLY ENABLED

Use when the current architecture already provides the structural capabilities needed to pursue the objective.

This does not mean the feature is fully implemented.

Remaining work may include:

- Feature implementation
- Documentation
- Tooling
- Optimization
- Product decisions
- Additional tests

A missing implementation alone must not prevent this classification.

ENABLED WITH LOCAL REFINEMENT

Use when the architecture provides a sound foundation, but a bounded architectural seam, contract, facade, registry, adapter, version field, or ownership clarification should be introduced before broad implementation.

The required work must be:

- Localized
- Compatible with the reconciled architecture
- Achievable without changing the system's fundamental authority model
- Achievable without broad runtime redesign

REQUIRES ARCHITECTURAL EVOLUTION

Use only when the objective cannot be responsibly pursued without meaningful structural change.

Examples include:

- New authority ownership
- Reversal of dependency direction
- Replacement of a central runtime model
- Broad changes to persistence or transaction semantics
- New cross-cutting architecture that cannot be introduced locally
- Fundamental conflict with the reconciled target architecture

Do not use this classification merely because:

- No implementation exists
- Product requirements are incomplete
- Documentation is weak
- A provider or ruleset interface has not yet been extracted
- A public API has not yet been formalized

DECISION DEPENDENT

Use only when materially different product decisions would produce different architectural conclusions and the repository cannot resolve the intended scope.

This classification should not stop the assessment.

Instead:

- State the most conservative reasonable assumption
- Give the classification under that assumption
- Explain what product decision could change it
- Identify whether the decision is needed before implementation or only before final design

Core Analytical Task

For every objective, answer:

1. What architectural capabilities already exist?
2. What evidence supports that judgment?
3. What limitation remains?
4. Is that limitation:
 - Documentation
 - Implementation
 - Product decision
 - Local architectural refinement
 - Genuine architectural evolution
5. Would implementation be safe before resolving it?
6. What is the minimum architectural action, if any?
7. What should explicitly not be redesigned?
8. What priority should the work receive?

Required Objective Assessments

1. Modular Ruleset Support

Determine whether the existing domain modules, state authority boundaries, result schemas, CTIR model, and non-combat contract are sufficient foundations for modular rulesets.

Judge whether the absence of:

- A Ruleset interface
- Ruleset registry
- Ruleset namespace layout

You are completing Campaign 2 of the Architecture Reconciliation era.

AR-AI — Final Vision Compatibility Closeout

Campaign Context

Campaign 2 has now completed both major phases:

- AR-AG — Final Vision Compatibility Discovery
- AR-AH — Final Vision Compatibility Assessment

Those documents collectively establish:

- the current reconciled architecture,
- its relationship to the project's long-term vision,
- the remaining architectural refinements,
- implementation dependencies,
- and the recommended roadmap.

This cycle is not another architectural assessment.

It is the formal closeout of Campaign 2.

Its purpose is to convert the assessment into a durable implementation roadmap that future development can follow without reopening architectural questions that have already been answered.

Goal

Produce the authoritative implementation transition document that marks the end of Final Vision Compatibility and the beginning of Controlled Feature Expansion.

The report should answer one central question:

Given everything now known about the architecture, what should the project actually build next?

Required Inputs

Review only:

- AR-AF_architecture_reconciliation_synthesis.md
- AR-AG_final_vision_compatibility_discovery.md
- AR-AH_final_vision_compatibility_assessment.md

Do not perform another repository-wide inspection.

Do not reopen Campaign 1 findings.

Do not reinterpret evidence already evaluated unless an obvious contradiction exists.

Primary Objectives

1. Close Campaign 2

State whether Campaign 2 achieved its stated goals.

Explicitly evaluate whether:

- the relationship between architecture and long-term vision is now understood,
- remaining architectural work has been identified,
- implementation ordering is clear,
- further architectural investigation would likely produce diminishing returns.

Provide a clear recommendation:

- Campaign Complete
 - or
 - One additional architecture cycle required
- If recommending another cycle, justify it with concrete evidence.

2. Produce the Final Vision Compatibility Summary

Summarize each vision objective in one concise section.

For each:

- final classification,
- why it received that classification,
- what work remains,
- what does not remain.

Keep this concise.

This should become the project's permanent executive summary.

3. Distinguish Three Categories of Remaining Work

Every remaining task should be placed into exactly one category.

Category A

Implementation

Meaning:

Architecture already supports this.

Only implementation remains.

Category B

Local Architectural Refinement

Meaning:

A bounded architectural seam should be completed before implementing a related feature.

Examples include:

- ruleset contract
- backend adapter
- public API facade

Category C

Future Product Evolution

Meaning:

Work that should intentionally wait until future product scope justifies it.

Examples may include:

- hosted multi-user support
- advanced capability negotiation
- semantic retrieval
- replay branching
- generic plugin systems

These should **not** appear as current project blockers.

4. Produce the Controlled Feature Expansion Readiness Assessment

Evaluate whether the architecture is ready for controlled feature expansion.

Consider:

- ownership
- replay
- CTIR
- persistence
- authority boundaries
- provenance
- extensibility
- documentation
- governance

Issue one recommendation:

READY

Controlled feature work may begin.

READY WITH PREREQUISITES

Feature work may begin after completing a very small prerequisite set.

NOT READY

Only use this if major architectural uncertainty still exists.

5. Produce the Implementation Ordering

Recommend the order in which implementation should occur.

Group work into phases.

For example:

Phase 1

Implementation prerequisites

Phase 2

Core feature expansion

Phase 3

Large capability expansion

Phase 4

Long-term evolution

The exact phases should be determined from repository evidence.

6. Define the First Recommended Feature Lane

Recommend the single best first implementation lane.

Justify why it should begin before other major efforts.

Possible examples:

- `!local-lets-not-handle-this!`
- Structured output handling
- Streaming assumptions
- Retry, timeout, fallback, and degradation behavior
- Backend identity captured in provenance
- Configuration-driven backend selection
- Support for local and remote inference
- Whether deterministic fallback paths depend on a specific provider
- Whether tests can substitute fake or deterministic backends

Determine whether future backend support is mainly an adapter implementation task or whether additional architecture is required.

5. Evaluate Clean Repository Handoff

Investigate whether another engineer or AI agent could receive the repository and operate it without relying on undocumented historical context.

Look for:

- Setup documentation
- Architecture documentation
- Environment and dependency declarations
- Configuration templates
- Test commands
- Entry points
- Repository layout clarity
- Generated or transient files
- Large or unnecessary assets
- Secret-management assumptions
- Local versus hosted operational modes
- Migration or initialization procedures
- Development and production distinctions
- Ownership or authority documentation
- Known limitations and invariants
- Whether critical knowledge exists only in campaign reports or historical notes

Identify whether handoff limitations are architectural, operational, or documentation-related.

6. Evaluate Low-Context Runtime Operation

Investigate how much historical or conversational context the runtime requires to operate correctly.

Look for:

- Explicit state models
- Durable world state
- Turn-local context construction
- Context assembly boundaries
- Replay projections
- Summarization or compression mechanisms
- Retrieval boundaries
- Prompt construction
- Hidden dependence on full conversation history
- State reconstruction behavior
- Provenance-backed context selection
- Token or context budgeting
- Whether different subsystems can request only the information they need
- Whether runtime decisions depend on implicit model memory
- Whether context can be regenerated deterministically from canonical state

Separate:

- Runtime architectural support
- Missing optimization
- Missing instrumentation
- Missing implementation

7. Evaluate Straightforward Feature Extensibility

Investigate the cost and risk of adding new capabilities such as:

- Combat systems
- Maps
- Portraits
- New rules subsystems
- Economy systems
- Investigation systems
- Research or construction systems
- New output formats
- Additional tools
- New state projections
- New realization or emission behavior

Look for:

- Stable extension points
- Registries
- Plugin-style boundaries
- Dependency direction
- Central switch statements
- God objects or orchestration hotspots
- Repeated cross-layer edits
- Test locality
- Ownership locality
- Feature flags
- Schema evolution practices
- Whether features can remain dormant without affecting the runtime
- Whether adding one feature requires modifications across unrelated modules

Use Campaign 1 hotspot and conformance findings rather than rediscovering them from scratch.

8. Evaluate UI and Tooling Readiness

Investigate whether the current architecture could support a user interface, external tools, debugging tools, campaign inspectors, or administrative controls.

Look for:

- Stable application APIs
- Command/query separation
- Serializable state
- Event or update models
- Read models and projections
- Streaming or incremental output
- Tool-call interfaces
- Idempotent commands
- Error models
- Session and campaign boundaries
- State inspection capabilities
- Save/load behavior
- Versioned schemas
- Whether UI concerns are mixed into domain execution
- Whether current APIs expose internal implementation details
- Whether headless operation is already cleanly supported

Do not treat the absence of an actual UI as an architectural failure. Evaluate whether the architecture can support one.

9. Evaluate Player Trust, Provenance, and Explainability

Investigate whether the system can explain what happened, why it happened, and which authority produced the result.

Look for:

- Provenance records
- Authority attribution
- Replay support
- Deterministic projections
- Rule-result traceability
- Model-output attribution
- Randomness recording
- State-change history
- Correction and reconciliation paths
- Fallback attribution
- Visibility into hidden versus player-visible state
- Audit logs
- Trust boundaries
- Whether generated narrative can be distinguished from canonical mechanical results
- Whether backend, ruleset, prompt, tool, and runtime versions can be recorded
- Whether players could challenge or inspect a ruling without exposing inappropriate hidden information

Determine which trust capabilities are already architectural strengths and which remain incomplete.

10. Identify Cross-Cutting Architectural Gaps

After reviewing all seven objectives, identify gaps that affect more than one objective.

Examples may include:

- Missing configuration architecture
- Missing plugin or adapter contract
- Missing versioning strategy
- Missing public application boundary
- Missing event model
- Missing serialization contract
- Missing capability negotiation
- Missing state migration policy

1. Modular Ruleset Support

Determine whether the existing domain modules, state authority boundaries, result schemas, CTIR model, and non-combat contract are sufficient foundations for modular rulesets.

Judge whether the absence of:

- A Ruleset interface
- Ruleset registry
- Ruleset package layout
- Runtime ruleset selection
- Ruleset version identity
- Alternate ruleset implementation

represents:

- Missing implementation
- Local refinement
- Genuine architectural evolution

Address at least two plausible scopes:

Scope A — One active ruleset per campaign or deployment

Assess whether the architecture can support selectable rulesets where one ruleset governs a campaign.

Scope B — Multiple rulesets or substantially different game systems

Assess whether the architecture can support systems that differ materially from PF1e-style assumptions.

Do not require the current repository to already contain a second ruleset.

Determine the minimum architecture needed before implementing one.

2. Multiple AI Backend Support

Determine whether model routing, normalized error handling, provenance metadata, retry/fallback behavior, and fake-client tests provide a sufficient foundation for multiple providers.

Judge whether extracting a provider-neutral backend protocol from `game.gm.call_gpt` and related preflight code is:

- A local adapter-boundary refinement
- A broader architectural evolution

Define the minimum expected backend contract conceptually, but do not implement it.

Consider:

- OpenAI-hosted models
- OpenAI-compatible APIs
- Other remote providers
- Local inference
- Deterministic fake/test backends
- Structured output capability
- Streaming
- Tool calling
- Capability negotiation
- Backend/provider provenance

Distinguish capabilities that must be in the first backend contract from capabilities that can be added later.

3. Clean Repository Handoff

Determine whether current limitations are architectural or primarily documentary and operational.

Assess whether the repository already contains the necessary runtime structure for handoff, even though its evidence is distributed.

Judge the need for:

- A durable “start here” document
- Current architecture index
- Canonical-versus-historical artifact index
- Setup and test checklist
- Deployment assumptions
- Secret/configuration guidance
- Data initialization and migration guidance
- Generated artifact retention policy

Do not classify document sprawl as architectural evolution unless it reflects a genuine inability to identify authority.

4. Low-Context Runtime Operation

Issue a firm judgment on whether low-context operation is architecturally enabled.

Assess:

- Explicit canonical state
- CTIR
- Bounded prompt context
- Persistence
- Snapshots
- Logs
- Replay projection
- State reconstruction
- Context-window helpers
- World progression slices

Determine whether missing semantic retrieval, token budgeting, or long-campaign instrumentation are:

- Necessary architectural prerequisites
- Optional scaling refinements
- Product-dependent implementation work

State the scale assumptions under which the current architecture is sufficient.

5. Straightforward Feature Extensibility

Determine whether the reconciled architecture permits controlled feature expansion without reopening Foundation Stabilization.

Consider several feature classes separately:

- New rules subsystem
- Economy or investigation system
- Map support
- Portrait/media support
- New output format
- Author/debug tool
- New state projection
- New model backend
- Player-facing explainability feature

For each class, identify the likely architectural path and expected touchpoints.

Assess whether:

- `game.api`
- Final-emission layers
- Fallback/provenance fields
- Replay protection
- State schemas

are acceptable coordination surfaces or evidence of an architectural blocker.

Do not propose a generic plugin system unless repository evidence shows it is necessary.

6. UI and Tooling Readiness

Determine whether FastAPI, state projection, UI modes, snapshots, logs, and debug channels already form an adequate headless application foundation.

Assess the difference between:

- Supporting the existing local browser UI
- Supporting an improved local UI
- Supporting author/debug tooling
- Supporting external integrations
- Supporting hosted or multi-user operation

Judge whether a versioned public API, command/query facade, event stream, or security model is:

- Required before any UI expansion
- Required only for external/hosted tooling
- A local architectural refinement

Do not treat the absence of streaming or multi-user support as a current failure unless the project vision explicitly requires it.

7. Player Trust, Provenance, and Explainability

Determine whether the existing architecture already supports trustworthy and inspectable operation.

Assess:

- State authority
- CTIR meaning
- Realization authority
- FEM
- Runtime lineage
- Protected replay
- Mutation traces
- Visibility separation
- Fallback attribution
- Model-route metadata
- Logs and snapshots

Distinguish between:

- Internal architectural explainability
- Operator-facing diagnostics
- Player-facing explanations
- Player challenge or correction workflows

Judge whether missing player-facing UX and redaction policy are:

- Product design and implementation work
- Local schema refinement
- Architectural evolution

Evaluate whether ruleset, backend, prompt, tool, and runtime version identity should be consolidated into a provenance/version bundle.

Product-Decision Dependencies

AR-AI identified several unresolved product questions.

Long-term evolution

The exact phases should be determined from repository evidence.

6. Define the First Recommended Feature Lane

Recommend the single best first implementation lane.

Justify why it should begin before other major efforts.

Possible examples:

- ruleset modularization
- economy systems
- investigation systems
- UI
- backend abstraction
- tooling
- maps
- explainability
- another feature if evidence supports it

Do not simply repeat AR-AH.

Choose one.

7. Identify What Should Explicitly NOT Be Done

Create a section titled:

Avoid These Mistakes

Include guidance such as:

- Do not reopen Foundation Stabilization.
- Do not redesign the transaction spine.
- Do not introduce generic abstractions before concrete need exists.
- Do not replace bounded ownership with convenience shortcuts.
- Do not weaken replay governance.

Add additional items supported by repository evidence.

8. Capture Permanent Architectural Invariants

Produce a concise list titled:

Architectural Invariants

These should represent principles that future implementation should preserve.

Examples include:

- domain simulation owns truth,
- CTIR owns resolved-turn meaning,
- prompt construction adapts rather than invents,
- replay observes rather than governs runtime,
- provenance explains behavior rather than determining behavior.

Only include principles strongly supported by Campaign 1 and Campaign 2.

9. Produce the Final Roadmap Recommendation

Select one:

Proceed Directly to Controlled Feature Expansion

Complete One Small Prerequisite Package Then Begin Controlled Feature Expansion

Return to Architecture Reconciliation

Explain the decision.

Deliverable

Create:

AR-AI_final_vision_compatibility_closeout.md

Include at least:

1. Executive Summary
2. Campaign Success Evaluation
3. Final Vision Compatibility Summary
4. Remaining Work Classification
5. Controlled Feature Expansion Readiness
6. Recommended Implementation Order
7. First Feature Lane Recommendation
8. Architectural Invariants
9. Avoid These Mistakes
10. Final Roadmap Recommendation
11. Campaign Closeout Statement

Campaign Closeout Statement

End the document with a formal statement indicating whether Campaign 2 is complete.

If complete, explicitly state that:

- Architecture Reconciliation is complete.
- Final Vision Compatibility is complete.
- Future work should assume the reconciled architecture as the project’s baseline.
- Architectural changes should be introduced only when justified by concrete implementation evidence rather than speculation.
- The project is now transitioning from architectural discovery into controlled implementation.

This statement should become the official conclusion of Campaign 2.

Scope Restrictions

Do not:

- inspect unrelated repository files,
- perform another discovery pass,
- redesign any subsystem,
- propose speculative abstractions,
- reopen Campaign 1 conclusions,
- begin implementation.

The goal is to permanently record what Campaign 2 accomplished and establish the baseline for the next era of development.

Final Response

When finished provide:

1. Whether Campaign 2 is officially complete.
2. The path to AR-AI_final_vision_compatibility_closeout.md.
3. The recommended first implementation lane.
4. The single most important prerequisite before implementation.
5. Whether the next campaign should transition into Controlled Feature Expansion.

- Missing plugin or adapter contract
 - Missing versioning strategy
 - Missing public application boundary
 - Missing event model
 - Missing serialization contract
 - Missing capability negotiation
 - Missing state migration policy
 - Missing observability or provenance fields
 - Excessive dependence on central orchestration
 - Documentation being used as a substitute for enforceable contracts
- Do not inflate the gap list. Only include gaps supported by concrete repository evidence.

11. Distinguish Architecture From Implementation Absence

- For every limitation, explicitly determine whether it represents:
- A missing implementation on top of an adequate architecture
 - Missing or outdated documentation
 - A local architectural refinement
 - A genuine architectural gap
 - An unresolved product decision rather than an architectural issue

This distinction is central to the campaign.

A feature not yet implemented must not automatically be classified as architecturally unsupported.

12. Identify Evidence That Must Be Supplied Externally

If the repository does not contain enough information to evaluate an objective, identify the exact missing information.

- Examples:
- Product vision documents
 - Intended deployment topology
 - Expected UI interaction model
 - AI provider requirements
 - Ruleset packaging expectations
 - Repository distribution constraints
 - Security or privacy requirements
 - Player-facing explainability expectations
 - Local-only operation requirements
- For every missing item, state:
- Why it is needed
 - Which vision objective it affects
 - Whether the repository may already contain it under an unknown filename
 - The exact files, decisions, or descriptions the project owner should provide

Required Deliverable

Create:
AR-AG_final_vision_compatibility_discovery.md

The report should contain at least the following sections:

1. Executive Summary
2. Scope and Method
3. Architectural Baseline
4. Evidence Sources Reviewed
5. Vision Compatibility Matrix
6. Modular Ruleset Support
7. Multiple AI Backend Support
8. Clean Repository Handoff
9. Low-Context Runtime Operation
10. Feature Extensibility
11. UI and Tooling Readiness
12. Player Trust, Provenance, and Explainability
13. Cross-Cutting Architectural Gaps
14. Architecture vs. Implementation Distinction
15. Missing Evidence and Requested Inputs
16. Candidate Priorities for Later Campaign Cycles
17. Recommended Next Cycle
18. Files Most Useful for External Review

Vision Compatibility Matrix Format

Include a table with columns equivalent to:

Vision Objective	Provisional Status	Strongest Supporting Evidence	Principal Limitation	Likely Work Type	Confidence
------------------	--------------------	-------------------------------	----------------------	------------------	------------

Use only the permitted provisional statuses:

- SUPPORTED
- PARTIALLY SUPPORTED
- ARCHITECTURAL EVOLUTION REQUIRED
- INSUFFICIENT EVIDENCE

Evidence Standards

Every important conclusion must identify concrete evidence, including where possible:

- File paths
- Module names
- Classes
- Functions
- Protocols or interfaces
- Schemas
- Tests
- Documentation sections
- Runtime call paths

Avoid statements such as “the system appears modular” without explaining which boundaries or contracts support that conclusion.

Where evidence conflicts, describe the conflict rather than silently selecting one interpretation.

Scope Control

Do not:

- Redesign the architecture
- Begin large-scale implementation
- Add speculative abstraction layers
- Create plugin systems merely because the vision mentions modularity
- Treat missing features as architectural defects
- Reopen resolved Campaign 1 findings without contradictory evidence
- Modify runtime behavior
- Perform broad refactoring
- Declare the entire vision compatible or incompatible based on superficial repository structure

Small documentation corrections are allowed only when necessary to prevent a material misunderstanding. Record any such changes in the report.

Final Response to the Project Owner

When finished, provide:

1. A concise summary of what was discovered
2. The path to AR-AG_final_vision_compatibility_discovery.md
3. A list of any files or decisions needed from the project owner
4. A short list of the most useful files to pass to an external reviewer
5. A statement of whether the repository contains enough evidence to proceed directly to the compatibility assessment cycle
6. Any blockers, uncertainties, or contradictory evidence

The report should prepare the next cycle to make the actual vision-compatibility judgments and prioritize any remaining architectural work.

- Product design and implementation work
- Local schema refinement
- Architectural evolution

Evaluate whether ruleset, backend, prompt, tool, and runtime version identity should be consolidated into a provenance/version bundle.

Product-Decision Dependencies

AR-AG identified several unresolved product questions.

Do not stop and request answers unless no meaningful architectural conclusion is possible.

Use these working assumptions:

Rulesets

Assume:

- One active ruleset per campaign
- Rulesets may be selected at campaign creation
- The long-term goal includes systems beyond PF1e
- Multiple unrelated rulesets do not need to execute simultaneously within one turn

AI Backends

Assume:

- At least OpenAI, one additional remote provider, and local inference should eventually be possible
- Streaming and tool calling are desirable but not required for the first provider-neutral boundary
- Deterministic fake backends are required for testing
- Backend and model identity should be preserved in provenance

Deployment

Assume:

- Local single-user operation remains the primary near-term target
- Hosted or multi-user operation is a possible future target, not an immediate requirement
- Local-only and online-backed modes should remain possible

UI and Tooling

Assume:

- The project will eventually need a player UI
- Author/debug tools are desirable
- External third-party API integrations are not yet mandatory
- A full hosted multi-user control plane is outside the immediate campaign scope

Explainability

Assume:

- Players should be able to inspect a safe explanation of mechanical outcomes
- Hidden information must remain protected
- Operators may access deeper provenance than players
- Formal dispute reversal or replay branching is a later product decision

Clearly label these as working assumptions.

Identify any conclusions that would change under different assumptions.

Cross-Cutting Gap Assessment

Evaluate each AR-AG cross-cutting gap:

1. Ruleset identity and packaging
2. Provider-neutral backend boundary
3. Versioning and migration strategy
4. Public application boundary
5. Evidence canonicity and handoff index
6. Capability negotiation
7. Player-safe explanation and challenge workflow

For each gap, determine:

- Objectives affected
- Whether it is truly architectural
- Whether it is local or cross-cutting
- Whether it blocks implementation
- Minimum resolution
- Recommended timing
- Risk of resolving it too early
- Risk of delaying it

Be conservative about adding abstraction.

A cross-cutting concern is not automatically a redesign.

Architectural Evolution Threshold

Before classifying anything as requiring architectural evolution, test it against these questions:

1. Can the limitation be solved by introducing one bounded interface, adapter, registry, facade, schema field, or document?
2. Can the change preserve current state authority and transaction flow?
3. Can current runtime behavior remain valid behind the new seam?
4. Can protected replay and provenance remain intact?
5. Can the change be introduced incrementally?
6. Does the target architecture already anticipate the relevant responsibility?

If most answers are yes, prefer:

ENABLED WITH LOCAL REFINEMENT

rather than:

REQUIRES ARCHITECTURAL EVOLUTION

Prioritization Framework

Assign every recommended work item:

Priority

- P0 — Required before controlled feature expansion
- P1 — Required before the affected feature class
- P2 — Valuable during implementation
- P3 — Deferred until scale or product scope requires it

Work Type

- Documentation
- Product decision
- Implementation
- Local architectural refinement
- Architectural evolution

Scope

- Objective-specific
- Cross-cutting
- Operational
- Governance

Risk if Deferred

- Low
- Moderate
- High
- Blocking

Do not give every item high priority.

The output should distinguish what is needed before the next feature from what can responsibly wait.

Roadmap Decision

Conclude with one of the following recommendations:

CONTINUE ROADMAP AS PLANNED

Use when no identified issue requires changing the sequence of upcoming work.

CONTINUE WITH TARGETED PREREQUISITES

Use when the architecture is broadly compatible, but a small number of bounded refinements or decisions should precede certain feature classes.

REORDER IMPLEMENTATION PRIORITIES

Use when the current roadmap would otherwise begin work that depends on unresolved boundaries.

RETURN TO ARCHITECTURE RECONCILIATION

Use only if material evidence shows Campaign 1’s reconciled architecture is not sufficient.

Explain:

- Why the recommendation was selected
- Which work should occur next
- Which work should not block progress
- Whether large-scale feature expansion is appropriate
- Whether controlled feature expansion is appropriate

Required Deliverable

Create:

AR-AH_final_vision_compatibility_assessment.md

Include at least these sections:

1. Executive Verdict
2. Assessment Scope and Working Assumptions
3. Final Vision Compatibility Matrix
4. Modular Ruleset Compatibility
5. Multiple AI Backend Compatibility
6. Repository Handoff Compatibility
7. Low-Context Runtime Compatibility
8. Feature Extensibility Compatibility
9. UI and Tooling Compatibility
10. Player Trust and Explainability Compatibility
11. Cross-Cutting Gap Assessment
12. Architecture vs. Implementation Findings

13. Prioritized Remaining Work
14. Implementation Dependency Map
15. Roadmap Recommendation
16. Campaign Closeout Question Answers
17. Recommended Next Cycle
18. Evidence and Confidence Notes

Final Vision Compatibility Matrix

Use a table equivalent to:

Vision Objective	Final Classification	Existing Architectural Strength	Remaining Gap	Work Type	Blocks What?	Confidence
------------------	----------------------	---------------------------------	---------------	-----------	--------------	------------

Use only:

- ARCHITECTURALLY ENABLED
- ENABLED WITH LOCAL REFINEMENT
- REQUIRES ARCHITECTURAL EVOLUTION
- DECISION DEPENDENT

Do not preserve AR-AG's provisional classification automatically.

Make the final judgment based on the distinction between architectural capability and absent implementation.

Prioritized Remaining Work Table

Use a table equivalent to:

Work Item	Priority	Work Type	Objectives Affected	Minimum Resolution	Blocks	Risk if Deferred
-----------	----------	-----------	---------------------	--------------------	--------	------------------

Group related work where appropriate.

Avoid turning every missing feature into a separate architectural work item.

Implementation Dependency Map

Identify which refinements must precede which implementation classes.

At minimum address:

- Alternate ruleset implementation
- Additional AI backend implementation
- UI expansion
- External tooling
- Maps/media
- Economy/investigation/research systems
- Player-facing explanation UI
- Hosted or multi-user deployment

Use language such as:

- Required before
- Recommended before
- Can proceed in parallel
- Does not block
- Defer until needed

Campaign Closeout Questions

Answer explicitly:

1. Which parts of the long-term vision are already enabled by the current architecture?
2. Which vision goals require only architectural refinement rather than redesign?
3. Which vision goals require genuine architectural evolution?
4. What implementation priorities naturally emerge?
5. Should the roadmap continue as planned, or should implementation priorities be reordered?

If no objective requires genuine evolution, say so plainly.

Do not invent one merely to make the assessment appear balanced.

Recommended Next Cycle

Recommend the next cycle based on the findings.

Possible outcomes include:

- A final Campaign 2 closeout and prioritized compatibility roadmap
- One bounded decision-record cycle
- One targeted prerequisite-design cycle
- Immediate transition to controlled feature expansion

Do not begin implementation in this cycle.

If a further cycle is recommended, define:

- Its exact purpose
- Why it is needed
- What decisions or artifacts it must produce
- Why it is not unnecessary architecture work

Evidence Standards

Every major judgment must cite:

- AR-AG evidence
- Campaign 1 doctrine or conformance findings where relevant
- Specific repository files or symbols when needed

Do not repeat the complete file inventory.

Use the strongest evidence only.

Clearly identify:

- Fact
- Architectural judgment
- Working assumption
- Product-dependent conclusion

Where evidence is insufficient, state the limitation without automatically lowering the entire objective to unsupported.

Scope Control

Do not:

- Modify runtime code
- Add interfaces or adapters
- Implement ruleset selection
- Implement additional AI providers
- Refactor game.api
- Create a plugin framework
- Add a public API
- Change protected replay fields
- Modify provenance taxonomy
- Begin UI redesign
- Treat hosted multi-user operation as an immediate requirement
- Reopen Campaign 1 without material contradictory evidence
- Equate absent implementation with architectural incompatibility

This cycle is complete when the relationship between the reconciled architecture and the long-term vision is clear enough to guide implementation ordering.

Final Response to the Project Owner

When complete, report:

1. The final classification of all seven vision objectives
2. Whether any objective genuinely requires architectural evolution
3. The most important prerequisite before controlled feature expansion
4. Which feature classes can proceed without additional architecture work
5. The roadmap recommendation
6. The path to AR-AH_final_vision_compatibility_assessment.md
7. Any product decisions that remain necessary
8. Whether Campaign 2 is ready for closeout or needs one additional cycle

Campaign 3 — Concept Reconciliation

Cycle 1: Concept Inventory and Responsibility Discovery

You are beginning Campaign 3: Concept Reconciliation for the Ashen Thrones architecture.

This campaign follows the completion of:

- Foundation Stabilization
- Campaign 1 — Architectural Mapping
- Campaign 2 — Final Vision Compatibility

The architecture has already been found broadly compatible with the project's long-term vision. Do not begin from the assumption that the system is overengineered or that concept reduction is inherently desirable. The purpose of this cycle is to gather the evidence needed to understand the conceptual architecture before recommending any clarification, consolidation, cleanup, or removal.

Campaign Context

Campaign 3 evaluates which architectural concepts are:

- Foundational
- Essential
- Transitional
- Overlapping
- Accidental
- Removable

The primary concepts currently include:

- Replay
- Realization
- Ownership
- Authority
- Routing
- Provenance
- Fallback
- Validators
- Repair
- Sanitizers
- Diagnostics
- Evidence

This list is not exhaustive. Identify additional architectural concepts where the repository, doctrines, tests, or prior reconciliation artifacts treat them as meaningful responsibilities.

Similar terminology must not automatically be treated as duplication. Concepts may intentionally protect different:

- Authority boundaries
- Ownership boundaries
- Lifecycle stages
- Runtime behaviors
- Governance responsibilities
- Evidence models
- Recovery paths
- Trust boundaries

Treat overlap as a hypothesis requiring evidence.

Objective

Produce a repository-grounded discovery report that answers:

- What architectural concepts currently exist?
 - Where is each concept defined, implemented, invoked, tested, or discussed?
 - What responsibility does each concept appear to protect?
 - Which invariants, boundaries, or failure modes depend on it?
 - Which concepts appear similar, and what evidence suggests they are intentionally distinct or potentially duplicative?
 - Which concepts appear permanent, transitional, historical, or unclear?
 - What additional evidence is required before the concepts can be classified confidently?
- Do not perform final concept reconciliation in this cycle unless the evidence is exceptionally conclusive. The primary result should be a reliable inventory and evidence map that a later cycle can use for classification.

Required Investigation

1. Locate Prior Architectural Conclusions

Find and collect the most relevant artifacts from the completed eras and campaigns, especially documents covering:

- Foundation Stabilization closure
- Architectural mapping
- Target architecture doctrine
- Architecture ownership
- Architecture conformance
- Architecture reconciliation synthesis
- Final Vision Compatibility
- Final Vision Compatibility closure
- State authority
- Replay boundaries
- Realization authority
- Provenance
- Routing
- Fallback behavior
- Validation, repair, sanitization, diagnostics, or evidence

Record which documents are normative, which are analytical, and which are historical.

Do not assume the newest document fully supersedes earlier material unless it explicitly says so.

2. Build a Concept Candidate Inventory

Search the repository for the primary concepts and closely related terms.

Include concepts found in:

- Architecture documents
- Doctrine files
- Ownership ledgers
- State models
- Runtime modules
- API boundaries
- Validators
- Fallback implementations
- Replay projections
- Realization code
- Repair pipelines
- Sanitizers
- Diagnostic systems
- Evidence or provenance structures
- Test names
- Comments describing responsibility boundaries

For each concept candidate, capture:

- Canonical name
- Alternative names or aliases
- Definition, if one exists
- Files where it is described
- Files where it is implemented
- Files where it is tested
- Main consumers
- Main producers
- Inputs and outputs
- Authority or ownership boundary
- Lifecycle stage
- Failure mode addressed
- Invariants apparently protected
- Whether the concept is externally visible or internal only

3. Trace Responsibilities Rather Than Words

A keyword match alone is insufficient.

For each major concept, inspect representative implementation paths and determine what the concept actually does.

Pay particular attention to distinctions such as:

- Replay versus reconstruction
- Replay versus projection
- Realization versus generation
- Realization versus emission
- Ownership versus authority
- Authority versus routing
- Routing versus dispatch
- Provenance versus evidence
- Evidence versus diagnostics
- Fallback versus repair
- Repair versus sanitization
- Validation versus sanitization
- Validation versus conformance checking
- Diagnostics versus operational logging
- Historical compatibility mechanisms versus intended permanent architecture

Where multiple concepts participate in one path, trace the order of operations.

4. Identify Concept Relationships

For each meaningful concept pair or cluster, identify whether the current evidence suggests:

- Dependency
- Composition
- Sequencing
- Delegation
- Governance
- Containment
- Orthogonality
- Intentional separation
- Partial overlap
- Unclear separation
- Possible duplication
- Historical residue

Do not classify similarity based only on naming.

Explain the different architectural questions each concept answers.

Examples:

- Ownership may answer "Which component is responsible for maintaining this state?"
- Authority may answer "Which entity may allow this to decide or mutate this value?"
- Provenance may answer "Where did this result come from?"
- Evidence may answer "What supports accepting this result?"
- Diagnostics may answer "How can a failure or decision be inspected afterward?"

Use repository evidence to determine whether distinctions like these actually exist.

5. Identify Protected Invariants

Locate tests, doctrines, assertions, contracts, or comments that indicate what would break if a concept were removed or merged.

Examples may include:

- Deterministic replay
- No unauthorized state mutation
- Single authoritative realization
- Stable ownership boundaries
- Traceable output origin
- Controlled fallback activation
- No silent repair
- No emission of malformed state
- Preservation of evidence across transformations
- Low context operation
- Backend independence
- Clean API handoff
- Player-visible trust controls

Do not invent invariants. Distinguish clearly between:

- Explicit invariant
- Implicit invariant

Campaign 3 — Concept Reconciliation

AR-BB — Defensive Runtime Boundary Reconciliation

You are continuing Campaign 3: Concept Reconciliation.

Campaign 3 is evaluating the conceptual architecture that emerged from:

- Foundation Stabilization
- Architecture Reconciliation
- Final Vision Compatibility

The previous cycle (AR-BA_concept_inventory_and_responsibility_discovery.md) completed the conceptual inventory and responsibility discovery phase.

That report concluded that the architecture already possesses a largely stable conceptual vocabulary, but identified one remaining high-density conceptual cluster requiring reconciliation before the campaign can safely classify concepts as permanent, transitional, or historical.

The cluster consists primarily of:

- Validators
- Repair
- Sanitizers
- Fallback
- Final Emission

This cycle reconciles those concepts.

Do not modify implementation.

Do not recommend refactoring.

Do not assume conceptual overlap implies conceptual duplication.

The objective is to determine why each concept exists, which architectural authority it protects, and whether any overlap is intentional or transitional.

Campaign Context

Previous discovery identified that:

- Final Emission owns player-visible legality and packaging.
- Validators, Repair, Sanitizers, and Fallback all participate in defensive runtime behavior.
- Several responsibilities intentionally overlap during execution.
- Some semantic repair responsibilities appear transitional.
- Final classification cannot safely proceed until these boundaries are understood.

This cycle should therefore reconcile the defensive-runtime concepts themselves rather than classify their permanence.

Objectives

Determine, using repository evidence:

- The precise architectural responsibility of each defensive-runtime concept.
- The lifecycle position occupied by each concept.
- The authority boundary of each concept.
- The ownership boundary of each concept.
- Which invariants depend upon each concept.
- Which concepts intentionally overlap.
- Which overlap exists only because of transitional implementation.
- Which terminology should become part of the permanent architectural vocabulary.

Do not evaluate concepts based on implementation size.

Evaluate responsibilities.

Required Investigation

1. Build Individual Responsibility Profiles

For each concept:

- Validators
- Repair
- Sanitizers
- Fallback
- Final Emission

determine:

- Architectural purpose
- Inputs
- Outputs
- Runtime owner
- Governance owner
- Lifecycle position
- Protected invariants
- Failure modes
- Downstream consumers
- Upstream dependencies

2. Trace Runtime Execution

Trace representative runtime paths.

Identify exactly where responsibility transfers occur.

Document:

- Validation begins here...
- Repair begins here...
- Sanitization begins here...
- Final Emission assumes ownership here...
- Fallback selection occurs here...
- Provenance recording occurs here...

If responsibility is shared, determine whether:

- one concept delegates,
- one concept orchestrates,
- or responsibilities are intentionally orthogonal.

3. Separate Responsibility from Execution Order

Do not conclude that concepts overlap merely because they execute consecutively.

Instead determine:

Which architectural question does each concept answer?

Examples:

Validators:

"Is this output acceptable?"

Repair:

"Can this known defect be corrected?"

Sanitizer:

"Can unsafe presentation artifacts be removed?"

Fallback:

"What should happen if acceptable output cannot be produced?"

Final Emission:

"What is ultimately allowed to reach the player?"

Repository evidence should determine the actual answers.

4. Reconcile Pairwise Relationships

Evaluate relationships between:

Validators ↔ Repair

Validators ↔ Sanitizers

Validators ↔ Final Emission

Repair ↔ Sanitizers

Repair ↔ Fallback

Repair ↔ Final Emission

Fallback ↔ Final Emission

Fallback ↔ Provenance

Fallback ↔ Repair

Sanitizers ↔ Final Emission

For every relationship determine whether it is:

- Sequential
- Hierarchical
- Orthogonal
- Delegated
- Intentional overlap
- Transitional overlap
- Possible redundancy

Support every conclusion with repository evidence.

5. PUBLISH Transitional Reconciliations

Campaign 3 — Concept Reconciliation

AR-BC — Permanent, Transitional, and Historical Concept Classification

You are continuing Campaign 3: Concept Reconciliation.

Previous cycles completed:

- AR-BA — Concept Inventory and Responsibility Discovery
- AR-BB — Defensive Runtime Boundary Reconciliation

The architecture has now been inventoried and its highest-density conceptual cluster has been reconciled.

This cycle performs the first complete architectural classification of the project's conceptual vocabulary.

Do not perform further repository discovery unless required to resolve a specific ambiguity.

Do not recommend implementation cleanup.

Do not recommend architectural simplification.

The purpose of this cycle is to determine the long-term architectural status of every major concept.

Campaign Context

Campaign 3 originally sought to determine:

- Which concepts are foundational.
- Which concepts remain essential implementation concepts.
- Which concepts exist only because of transitional implementation.
- Which concepts are historical residue.
- Which concepts should become part of the permanent architectural vocabulary.

AR-BA established the concept inventory.

AR-BB reconciled the defensive-runtime concepts and concluded that the remaining ambiguities are narrow enough that full concept classification can begin.

This cycle therefore classifies every major architectural concept.

Objectives

Using repository evidence and the findings from AR-BA and AR-BB, classify every significant architectural concept according to its long-term role.

For every concept determine:

- Long-term status
- Architectural responsibility
- Protected invariants
- Conceptual dependencies
- Architectural permanence
- Expected implementation lifetime
- Documentation priority

Do not classify concepts according to implementation size.

Classify architectural responsibility.

Classification Categories

Every concept should be assigned exactly one primary classification.

Foundational

Concepts that define the architecture itself.

Removing or replacing them would fundamentally change the architectural model.

Essential

Concepts that exist because the architecture requires them, even if implementations evolve.

Transitional

Concepts expected to shrink, migrate, or disappear as implementation matures.

Historical

Concepts retained primarily for compatibility or historical continuity.

Supporting

Concepts that provide governance, diagnostics, or operational support but are not central architectural primitives.

If a concept appears to span multiple categories, identify:

- Primary classification
- Secondary role

Concepts To Classify

At minimum evaluate:

- Runtime Transaction
- Domain Simulation
- State Authority
- Ownership
- Authority
- Routing
- CTIR
- Prompt / Adaptation
- Realization
- Provenance
- Evidence
- Validators
- Repair
- Sanitizers
- Fallback
- Diagnostics
- Telemetry / Lineage
- Governance
- Canonicity
- Compatibility Residue

Include any additional concepts discovered during AR-BA or AR-BB.

Required Investigation

1. Perform Individual Classification

For every concept determine:

- Primary classification
- Secondary role (if applicable)
- Why the concept exists
- Which architectural questions it answers
- Which invariants depend upon it
- Which other concepts depend upon it
- Expected long-term lifetime
- Whether implementation changes are expected to affect the concept itself or merely its implementation

2. Build Dependency Hierarchy

Determine which concepts exist because of others.

Examples:

Does Replay depend on Final Emission?

Does Provenance depend on Authority?

Does Validators depend on Final Emission?

Does Final Emission depend on Repair?

Represent these relationships explicitly.

3. Identify Permanent Vocabulary

Campaign 3 — Concept Reconciliation

AR-BD — Concept Map Synthesis and Campaign Closure

You are completing Campaign 3: Concept Reconciliation.

Previous cycles completed:

- AR-BA — Concept Inventory and Responsibility Discovery
- AR-BB — Defensive Runtime Boundary Reconciliation
- AR-BC — Permanent, Transitional, and Historical Concept Classification

Campaign 3 has completed its discovery, reconciliation, and classification phases.

This final cycle synthesizes those results into the permanent conceptual doctrine that future implementation should rely upon.

Do not perform additional repository discovery except where necessary to verify an unresolved ambiguity.

Do not recommend implementation refactoring.

Do not reopen previously reconciled conceptual boundaries.

The objective is to produce the project's stable conceptual vocabulary and formally close Campaign 3.

Campaign Context

Campaign 3 originally sought to determine:

- Which architectural concepts are foundational.
- Which concepts are essential.
- Which concepts are transitional.
- Which concepts are historical.
- Which concepts should become part of the permanent architectural vocabulary.

AR-BC concluded that the conceptual taxonomy is now sufficiently stable for synthesis.

This cycle consolidates those findings into a contributor-facing conceptual model.

Objectives

Produce a coherent conceptual architecture describing:

- the permanent vocabulary,
- conceptual hierarchy,
- responsibility boundaries,
- dependency relationships,
- transitional concepts,
- historical concepts,
- governance concepts,
- and the conceptual lifecycle of a runtime turn.

This report should become the conceptual reference point for future architecture work.

Required Investigation

1. Build the Permanent Concept Map

Construct a high-level conceptual map showing relationships among:

Core Runtime

Runtime Transaction

Domain Simulation

State Authority

CTIR

Prompt / Adaptation

Realization

Final Emission

Governance

Ownership

Authority

Canonicity

Governance

Evidence

Replay

Projection

Provenance

Evidence

Diagnostics

Telemetry

Defensive Runtime

Validators

Repair

Sanitizers

Fallback

Historical / Transitional

Compatibility Residue

Transitional repair

Transitional fallback vocabulary

Historical compatibility mechanisms

Represent dependency direction rather than implementation flow.

2. Build the Runtime Concept Lifecycle

Describe the conceptual lifecycle of a runtime turn from:

player intent

through

authoritative simulation

through

expression

- No silent repair
- No emission of malformed state
- Preservation of evidence across transformations
- Low-conflict operation
- Backed independence
- Clean API handoff
- Player-visible trust controls

Do not invent invariants. Distinguish clearly between:

- Explicit invariant
- Strongly inferred invariant
- Possible invariant requiring confirmation

6. Detect Transitional and Historical Mechanisms

Look for concepts or structures that may exist because of:

- Migration from an earlier architecture
 - Compatibility with old state formats
 - Temporary delegation
 - Incomplete ownership transfer
 - Transitional routing
 - Legacy fallback behavior
 - Duplicated validation during stabilization
 - Defensive repair around upstream uncertainty
 - Old terminology that survived implementation changes
 - Tests preserving behavior that may no longer be architecturally required
- Evidence of age alone is not sufficient for removal.
- For each candidate transitional or historical mechanism, determine:
- What condition originally required it?
 - Whether that condition still exists
 - Whether tests still depend on it
 - Whether documents describe an intended retirement condition
 - Whether removal would change behavior or merely vocabulary

7. Identify Missing or Ambiguous Documentation

Flag concepts that are implemented but not adequately documented, or documented differently across files.

- For each ambiguity, identify the smallest unresolved question, such as:
- Is this concept an architectural responsibility or only a module name?
 - Is this distinction normative or incidental?
 - Is one concept a subpage or phase of another?
 - Does a fallback own decisions or only execute decisions made elsewhere?
 - Does repair alter authoritative state or only prepare candidate data?
 - Is evidence persisted, derived, or diagnostic only?
 - Are validators advisory, gating, or authoritative?

8. Assess Test Coverage as Conceptual Evidence

Identify tests that strongly define conceptual boundaries.

- For each important test or test group, state:
- Concept protected
 - Behavior guaranteed
 - Boundary enforced
 - Whether the test reflects current doctrine or historical compatibility
 - Whether the test would help adjudicate a future merge or removal proposal
- Do not treat test existence as proof that the behavior should remain permanently. Tests are evidence of current dependency, not automatic evidence of permanent necessity.

Required Deliverable

Create:

- AR-BA_concept_inventory_and_responsibility_discovery.md
- If AR-BA conflicts with the repository's active naming sequence, use the next valid Architecture Reconciliation Identifier and explain the adjustment.
- The report should contain the following sections.

Do not treat test existence as proof that the behavior should remain permanently. Tests are evidence of current dependency, not automatic evidence of permanent necessity.

Required Deliverable

Create:

- AR-BA_concept_inventory_and_responsibility_discovery.md
- If AR-BA conflicts with the repository's active naming sequence, use the next valid Architecture Reconciliation Identifier and explain the adjustment.
- The report should contain the following sections.

1. Executive Summary

- Summarize:
- How many major concepts were identified
 - Which concepts appear best defined
 - Which concepts appear most ambiguous
 - Where the strongest apparent overlap exists
 - Whether any concepts already appear clearly transitional or historical
 - Whether the evidence is sufficient to begin classification
- Do not give final removal recommendations unless overwhelmingly supported.

2. Source and Evidence Register

File	Type	Status	Concepts Covered	Why It Matters
Suggested type values:				
• Normative doctrine				
• Architecture analysis				
• Closeout				
• Ownership model				
• Runtime implementation				
• Test				
• Historical artifact				
• Operational documentation				
Suggested status values:				
• Current				
• Possibly superseded				
• Historical				
• Unclear				

3. Concept Inventory

Provide one subsection per concept.

Use this structure:

Concept: [Name]

Working definition:
A repository-grounded definition.

Alternative terminology:
Aliases, older names, module-specific names, or related terms.

Primary architectural responsibility:
What this concept protects or enables.

Lifecycle position:
Where it acts in the system.

Authority and ownership relationship:
Which component owns or governs it.

Primary producers:
Components that create or initiate it.

Primary consumers:
Components that depend on it.

Inputs and outputs:
Relevant data or control flow.

Protected invariants:
Explicit, inferred, and uncertain invariants.

Failure modes addressed:
What goes wrong without it.

Implementation evidence:
Files, functions, classes, or modules.

Test evidence:
Relevant tests and assertions.

Documentation evidence:
Relevant architecture or doctrine files.

Current conceptual status:
Use only a preliminary status:

- Clearly permanent candidate
- Likely permanent candidate
- Transitional candidate
- Historical candidate
- Possible overlap
- Insufficient evidence

Open questions:
What must be resolved before final classification.

4. Concept Relationship Matrix

Create a matrix or structured list for the major concept pairs.

At minimum, evaluate:

- Reply / Realization
- Reply / Provenance
- Realization / Authority
- Ownership / Authority
- Authority / Routing
- Routing / Fallback
- Fallback / Repair
- Validators / Repair
- Validators / Sanitizers
- Repair / Sanitizers
- Provenance / Evidence
- Evidence / Diagnostics
- Diagnostics / Validators

For each relationship, classify the current evidence as:

- Internally distinct
- Hierarchical
- Sequential
- Partially overlapping
- Potentially redundant
- Historically entangled
- Unclear

Include a concise explanation and file references.

5. Invariant-to-Concept Map

Provide a table:

Invariant	Concepts Protecting It	Evidence	Explicit or Inferred	Confidence
-----------	------------------------	----------	----------------------	------------

This should make clear whether multiple concepts jointly protect the same invariant for legitimate reasons.

6. Transitional and Historical Candidate Register

Provide a table:

Candidate	Original Purpose	Current Dependency	Retirement Condition	Evidence	Confidence
-----------	------------------	--------------------	----------------------	----------	------------

Do not recommend removal solely because something is transitional.

7. Ambiguity and Documentation Gap Register

Provide a table:

Concept or Boundary	Ambiguity	Conflicting Evidence	Risk of Leaving Unclear	Recommended Next Investigation
---------------------	-----------	----------------------	-------------------------	--------------------------------

8. Preliminary Concept Map

Provide a text-based concept map showing:

- Core architectural concepts
- Governance concepts
- Runtime lifecycle concepts
- Recovery and defensive concepts
- Trust and evidence concepts
- Diagnostic or operational concepts
- Transitional or historical mechanisms
- Unclassified concepts

Use arrows or indentation to show relationships.

Do not force every concept into only one category where it legitimately spans multiple categories.

- Delegated
- Intentional overlap
- Transitional overlap
- Possible redundancy

Support every conclusion with repository evidence.

5. Evaluate Transitional Responsibilities

Determine whether repository evidence supports that any current responsibilities are expected to migrate elsewhere.

Examples:

- semantic repair
- fallback generation
- legality enforcement
- emergency recovery
- player-visible cleanup

For every transitional responsibility identify:

- current owner
- intended future owner
- evidence supporting migration
- implementation dependency
- retirement condition

6. Identify Stable Concept Boundaries

At the end of reconciliation determine the permanent conceptual boundary for each concept.

The goal is that future contributors should understand:

- where Validation ends,
- where Repair begins,
- where Repair ends,
- where Sanitization begins,
- where Final Emission becomes authoritative,
- and where Fallback participates.

Required Deliverable

Create:

AR-BB_defensive_runtime_boundary_reconciliation.md

The report should contain:

1 Executive Summary

Summarize:

- strongest conceptual distinctions
- strongest intentional overlap
- strongest transitional overlap
- unresolved ambiguity
- readiness for concept classification

2 Responsibility Profiles

One section per concept.

Include:

- Purpose
- Inputs
- Outputs
- Authority
- Ownership
- Lifecycle
- Protected invariants
- Failure modes
- Dependencies
- Current status

3 Responsibility Transfer Map

Show responsibility flow through the runtime pipeline.

Identify where authority changes hands.

4 Pairwise Relationship Analysis

Analyze every required concept pair.

State:

- relationship type
- evidence
- confidence

5 Invariant Mapping

Map every major defensive-runtime invariant to the concepts protecting it.

6 Transitional Responsibility Register

For every transitional responsibility document:

- current owner
- future owner
- migration evidence
- confidence

7 Permanent Boundary Recommendations

For each concept explain:

- "This concept permanently owns..."
- "This concept intentionally does not own..."
- "This concept should never become responsible for..."

8 Remaining Ambiguities

Document unresolved questions requiring future investigation.

Do not speculate.

9 Campaign Impact

Explain:

- How this reconciliation changes Campaign 3.
- State whether:
- Concept Classification can begin.
- Another reconciliation cycle is required.
- Additional repository evidence is needed.

Recommend the next Architecture Reconciliation block using the AR-BC naming sequence.

Files Recommended for External Review

Separate into:

Required

Optional

Explain why each file belongs in its category.

Evidence Standards

Every conclusion must distinguish between:

- documented doctrine
- implementation evidence
- tests
- architectural inference

Do not present inference as established architecture.

Constraints

Do not:

- modify runtime behavior
- rename concepts
- merge concepts

Does Validators depend on Final Emission?

Does Final Emission depend on Repair?

Represent these relationships explicitly.

3. Identify Permanent Vocabulary

Determine which concepts should become part of the permanent architectural language.

These should be concepts future contributors are expected to understand regardless of implementation evolution.

4. Identify Transitional Vocabulary

Determine which terms should gradually disappear as implementation converges.

For each transitional concept identify:

- Why it exists today
- What architectural condition still requires it
- What future event would eliminate it

5. Evaluate Documentation Priorities

Recommend which concepts deserve:

- Dedicated doctrine
- Architectural glossary entries
- Ownership documentation
- Runtime documentation
- No additional documentation

Required Deliverable

Create:

AR-BC_permanent_transitional_historical_concept_classification.md

The report should contain:

1 Executive Summary

Summarize the overall conceptual maturity of the architecture.

2 Concept Classification Table

Include:

- [Concept | Classification | Secondary Role | Confidence | Expected Lifetime]

3 Individual Concept Reviews

One section for every concept.

4 Dependency Hierarchy

Show concept dependencies.

5 Permanent Architectural Vocabulary

List concepts expected to remain stable across future implementation evolution.

6 Transitional Vocabulary

List concepts expected to disappear or substantially evolve.

7 Historical Vocabulary

Identify concepts retained primarily for compatibility.

8 Documentation Priority Matrix

Recommend future documentation priorities.

9 Architectural Observations

Explain:

- Which architectural concepts proved more fundamental than expected.
- Which concepts proved primarily implementation-oriented.
- Which concepts changed classification because of AR-BB.
- Whether any concepts should be renamed in future documentation (without recommending implementation changes).

10 Campaign Assessment

Evaluate whether Campaign 3 is approaching completion.

Files Recommended for External Review

Separate into:

Required

Optional

Do not introduce new architectural theories.

Do not reopen previously reconciled questions.

Constraints

This is the synthesis and closeout cycle.

Completion Standard

This cycle is complete when:

- the permanent conceptual vocabulary is established,
- the conceptual hierarchy is documented,
- the runtime conceptual lifecycle is explained,
- transitional and historical concepts are clearly separated,
- Campaign 3's original closeout questions have been explicitly answered,
- and future contributors can understand the architecture without revisiting earlier reconciliation work.

Return:

1. The completed synthesis report.
2. A concise Campaign 3 closeout summary.
3. A recommendation on whether Campaign 3 is complete.
4. If complete, recommend the first Architecture Reconciliation block for Campaign 4.

primary invariants

This should read like a contributor glossary rather than an audit.

5. Consolidate Transitional Concepts

For every transitional concept explain:

- why it exists
- why it should not yet disappear
- what future event would retire it

Do not recommend implementation work.

6. Evaluate Campaign Success

Answer the original Campaign 3 questions directly.

Determine whether:

- conceptual ambiguity has been eliminated,
- terminology is now stable,
- future implementation can rely upon this vocabulary.

Required Deliverable

Create:

AR-BD_concept_map_synthesis_and_campaign_closeout.md

The report should contain:

1 Executive Summary

Summarize the conceptual architecture as it exists today.

2 Permanent Concept Map

Present the project's permanent conceptual model.

3 Runtime Concept Lifecycle

Describe concept participation across a complete runtime turn.

4 Architectural Layer Model

Organize concepts into coherent architectural layers.

5 Permanent Vocabulary

Provide glossary-style definitions for every permanent concept.

6 Transitional Vocabulary

Document concepts expected to evolve or eventually disappear.

7 Historical Vocabulary

Document concepts retained primarily for compatibility or historical continuity.

8 Concept Dependency Map

Show which concepts depend upon others.

9 Campaign 3 Closeout Assessment

Explicitly answer:

1. Which architectural concepts are truly foundational?
2. Which concepts remain essential implementation concepts?
3. Which concepts are transitional?
4. Which concepts intentionally overlap?
5. Which concepts are historical?
6. What permanent vocabulary should future contributors use?
7. Should implementation priorities change because of Campaign 3?

10 Campaign Recommendation

Determine one of:

- Campaign 3 Complete
- One final refinement required

If complete, recommend transition into Campaign 4.

Files Recommended for External Review

Separate into:

Required

Strongly Recommended

Optional

Recommend the minimum document set needed for future architectural onboarding.

Evidence Standards

Support synthesis using:

- doctrine
- implementation
- tests
- prior Campaign 3 reports

Do not reopen previously reconciled questions.

Constraints

This is the synthesis and closeout cycle.

Completion Standard

This cycle is complete when:

- the permanent conceptual vocabulary is established,
- the conceptual hierarchy is documented,
- the runtime conceptual lifecycle is explained,
- transitional and historical concepts are clearly separated,
- Campaign 3's original closeout questions have been explicitly answered,
- and future contributors can understand the architecture without revisiting earlier reconciliation work.

Return:

1. The completed synthesis report.
2. A concise Campaign 3 closeout summary.
3. A recommendation on whether Campaign 3 is complete.
4. If complete, recommend the first Architecture Reconciliation block for Campaign 4.

C4: Boundary Reconciliation

Friday, July 17, 2026 2:29 PM

Campaign 4 — Boundary Reconciliation

The preceding campaigns established and reconciled:

- The current architectural structure
 - Compatibility with the intended long-term vision
 - The project's permanent, transitional, historical, overlapping, and removable concepts
 - The authoritative vocabulary and conceptual responsibility model
- Campaign 4 must now evaluate whether the resulting architecture expresses sufficiently durable **responsibility boundaries** for long term implementation and feature expansion.
- Do not approach this as a search for an undiscovered architecture.
- Assume the evolved boundary model is generally correct unless repository evidence demonstrates a genuine architectural inconsistency.

The purpose of this cycle is discovery and evidence collection. Do not perform broad architectural refactoring.

Goal

Produce a repository-grounded inventory of the project's major architectural boundaries and gather the evidence needed to determine:

- Which boundaries are already stable
- Which boundaries intentionally overlap
- Which overlaps are transitional or compatibility driven
- Which implementation responsibilities align naturally with their architectural boundary
- Which future changes are likely to affect only implementation
- Which areas may still require legitimate architectural evolution

The resulting document must make it possible to plan the remaining Boundary Reconciliation cycles without repeatedly rediscovering repository structure or requesting large numbers of source files.

Major Boundary Families

Investigate at least the following boundary families:

1. Runtime orchestration
2. Domain simulation
3. Ruleset architecture
4. AI interaction
5. Final emission
6. Replay and projection
7. Evidence and provenance
8. Diagnostics and telemetry
9. Persistence
10. Governance
11. Tooling
12. UI integration
13. Plugins and adapters
14. Content

Inspect whether boundary family only if repository evidence shows that it represents a distinct long-term architectural responsibility boundary or an implementation-specific component.

Required Investigation

1. Recover the Existing Architectural Doctrine

Locate and review the authoritative outputs from the previous campaigns, especially documents covering:

- Target architecture doctrine
- Architecture conformance
- Architecture reconciliation
- Final-vision compatibility
- Concept inventory and responsibility discovery
- Defensive runtime boundary reconciliation
- Permanent, transitional, and historical concept classification
- Concept-map synthesis and Campaign 3 closureout
- State authority
- Ownership or responsibility ledgers
- Replay, realization, projection, provenance, evidence, routing, validation, fallback, and final-emission doctrine

Treat these documents as prior architectural decisions, not merely commentary.

- Explicit boundary declarations
- Ownership rules
- Authority rules
- Long-dependency directives
- Temp-term concepts
- Transitional compatibility structures
- Historical terminology that should no longer govern architecture
- Previously identified unresolved questions that remain relevant to boundary placement

2. Map Repository Structure to Boundary Families

Inspect the repository and identify the modules, packages, services, interfaces, tests, documents, scripts, adapters, and entry points associated with each boundary family.

For every boundary family, record:

- Primary implementation locations
 - Public interfaces or entry points
 - Upstream callers
 - Downstream dependencies
 - State read or written
 - Decisions or transformations owned
 - Outputs produced
 - Adjacent boundaries
 - Tests that enforce or reveal the boundary
 - Documentation that defines the intended boundary
 - Compatibility or transitional layers associated with it
- Do not equate a directory boundary with an architectural boundary without supporting evidence.

3. Identify Boundary Ownership

For each boundary family, determine the strongest repository-supported answer to:

- What responsibility does this boundary permanently own?
- What must it not own?
- What inputs may it accept?
- What outputs may it produce?
- Which state may it authoritatively mutate?
- Which decisions may it make?
- Which decisions must already have been made before data reaches it?
- Which downstream systems may reintroduce, project, serialize, display, or transport its output?
- Which adjacent subsystem is allowed to reject, repair, validate, or annotate its work?
- Which responsibilities exist only because of current implementation constraints?

Separate architectural authority from operational participation.

A subsystem may participate in a workflow without owning the meaning of that workflow.

4. Analyze Cross-Boundary Flows

Trace representative end-to-end flows through the system, including where applicable:

- Campaign or session startup
- Player-input handling
- Domain-state transition
- Ruleset evaluation
- AI request construction
- AI response receipt
- Response validation, repair, or sanitization
- Realization
- Final emission
- Replay capture
- Replay projection
- Evidence and provenance generation
- Persistence
- Diagnostic or telemetry emission
- UI-facing projection
- Plugin or adapter invocation
- Tooling-only inspection or repair

For each flow, identify:

- Where authority begins
- Which responsibilities transfers
- Which boundaries transform data
- Which boundaries merely transport data
- Which boundaries validate or reject data
- Which boundaries annotate evidence
- Which boundaries project an authoritative result into another representation
- Whether any subsystem appears to make decisions belonging to another boundary

Focus on boundary crossings, not a general inventory of program execution.

5. Classify Boundary Condition

Assign each boundary family a preliminary classification using the following categories:

Stable
The long-term responsibility and authority are clear. Future work is expected to change implementation more than architectural placement.

Stable with Intentional Overlap
The responsibility is clear, but limited overlap with another boundary is required for validation, translation, projection, orchestration, evidence capture, governance, or compatibility. The overlap must have an identifiable reason and direction.

Transitional
The intended long-term boundary is understood, but current implementation still contains compatibility patches, duplicated participation, legacy entry points, migration scaffolding, or temporary ownership leakage.

Architecturally Unresolved
Repository evidence reveals a genuine unresolved question about long-term responsibility, authority, placement, or dependency direction.

Use this classification sparingly. Do not classify a boundary as unresolved merely because implementation is incomplete.

Insufficient Evidence
The repository does not currently provide enough evidence to classify the boundary confidently.

Explain exactly what evidence is missing.

6. Distinguish Types of Remaining Work

For each boundary family, classify likely remaining work as one or more of:

- Documentation
- Test coverage
- Interface clarification
- Dependency cleanup
- Compatibility retirement
- Implementation completion
- Architectural refinement
- Genuine architectural evolution
- No material boundary work presently indicated

Do not treat missing implementation as evidence of a missing boundary.

Do not treat documentation drift as architectural instability unless an observed behavior contradicts the intended model.

7. Identify Boundary Violations and Pressure Points

Search for concrete evidence of:

- Duplicate authority
- Circular architectural responsibility
- Domain decisions made in transport or presentation layers
- AI-specific behavior leaking into domain authority
- Final-emission code mutating authoritative state
- Replay or projection code becoming a second simulation authority
- Evidence or provenance code changing the result it records
- Diagnostics affecting runtime outcomes
- Persistence models dictating domain semantics
- Governance code performing subsystem implementation work
- Tooling paths becoming required production authorities
- UI models becoming authoritative runtime state
- Plugins or adapters bypassing governance

Produce a final report summarizing the findings of this cycle.

AR-BF — Ruleset, Backend, and Version-Provenance Boundary Inventory

Campaign Context

Campaign 4 repository established that the project's architectural boundary model is substantially mature.

The repository demonstrates a durable forward authority pipeline in which runtime orchestration, domain simulation, CTR, prompt adaptation, realization, final emission, persistence, replay, evidence, governance, tooling, UI projection, and content each possess well-defined long-term responsibilities.

AR-BF further validated that Final Emission is a mature architectural boundary whose remaining implementation work is primarily compatibility retirement and implementation refinement rather than new development.

The remaining architectural questions now shift away from runtime ownership and toward extensibility.

Previous campaigns repeatedly identified three closely-related future expansion seams:

- Ruleset architecture
- AI Backend abstraction
- Version and provenance identity

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

Do not design alternate rulesets or additional AI providers.

AR-BG — Ruleset, Backend, and Version-Provenance Boundary Inventory

Campaign Context

Campaign 4 has now established that the project's core runtime boundary model is architecturally stable.

AR-BG demonstrated that the major architectural responsibilities have durable long-term ownership.

AR-BF further validated that Final Emission is a mature architectural boundary whose remaining implementation work is primarily compatibility retirement and implementation refinement rather than new development.

The remaining architectural questions now shift away from runtime ownership and toward extensibility.

Previous campaigns repeatedly identified three closely-related future expansion seams:

- Ruleset architecture
- AI Backend abstraction
- Version and provenance identity

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

Do not design alternate rulesets or additional AI providers.

Instead, determine whether the existing architecture already defines sufficiently durable extension boundaries for those future capabilities.

- Final emission code mutating authoritative state
- Replay or projection code becoming a second simulation authority
- Evidence or provenance code changing the result it records
- Diagnostics affecting runtime outcomes
- Persistence models dictating domain semantics
- Governance code performing subsystem implementation work
- Tooling paths becoming required production authorities
- UI models becoming authoritative runtime state
- Plugins or adapters bypassing governance
- Content definitions embedding runtime orchestration
- Compatibility layers becoming permanent by accident
- Validators, repair paths, or sanitizers silently assuming authority they were intended only to defend

- For every suspected violation:
- Cite the exact file, symbol, or test
 - Describe the observed behavior
 - Identify the boundary apparently crossed
 - State whether the issue is architectural, transitional, defensive, or merely implementation-local
 - Avoid proposing a refactor until the classification is supported

8. Identify Stable Boundary Evidence

Do not focus only on problems.

For every major boundary family, actively identify evidence that the boundary is already working correctly, such as:

- Explicit interfaces
- Narrow dependency direction
- Ownership tests
- Authority guards
- Projection-only APIs
- Immutably handoff models
- Provenance records
- Final-emission isolation
- Replay non-authority
- Adapter contracts
- Plugin governance
- UI-facing DTOs or projections
- Domain-independent tooling
- Documentation aligned with implementation

The campaign assumes the architecture may already be substantially correct. The report must preserve that presumption unless contradicted.

Required Deliverable

Create:
AR-BE_boundary_reconciliation_discovery_and_evidence_inventory.md
The document must contain the following sections.

1. Executive Summary

Summarize:

- Overall apparent maturity of the boundary model
- Boundaries that already appear durable
- Boundaries with intentional overlap
- Boundaries that appear transitional
- Any areas showing evidence of genuine architectural uncertainty
- Whether the next cycles should primarily validate boundaries, reconcile overlaps, or investigate specific pressure points

2. Prior Architectural Decisions Recovered

Provide a concise table containing:

- Decision or doctrine
- Source document
- Boundary families affected
- Current implementation evidence
- Whether the decision still appears authoritative

3. Reproductive Boundary Map

For each major boundary family, provide:

- Purpose
- Primary modules and files
- Principal interfaces or symbols
- Upstream dependencies
- Downstream dependencies
- State authority
- Adjacent boundaries
- Relevant tests
- Relevant architectural documents

4. Preliminary Boundary Classification Matrix

Use one row per boundary family with these columns:

- Boundary family
- Preliminary classification
- Permanent responsibility
- Explicit exclusions
- Intentional overlaps
- Transitional overlap or compatibility support
- Implementation alignment
- Likely future change type
- Confidence
- Any evidence

5. Cross-Boundary Flow Traces

Document representative flows and their responsibility handoffs.

Use concise numbered states and identify the owning boundary at every stage.

6. Boundary Pressure Points

For each pressure point, document:

- Identifier
- Boundaries involved
- Files and symbols
- Observed condition
- Why it may matter
- Preliminary classification
- Evidence still needed
- Whether it warrants a later campaign cycle

7. Stable Boundary Evidence

Document the strongest examples showing that existing architectural boundaries are already durable and aligned with prior doctrine.

8. Remaining Evidence Gaps

List only information that could not be recovered from the repository.

- For each gap, specify:
- The exact question
 - Why the answer matters
 - Files, runtime evidence, or human decisions needed
 - Whether later work can proceed without it

9. Recommended Campaign Decomposition

Recommend the next Boundary Reconciliation cycles.

Use the established AR-BE sequence beginning after AR-BE.

Each proposed cycle must include:

- Proposed identifier and title
- Exact boundary problem being evaluated
- Why it requires a distinct cycle
- Primary evidence to examine
- Expected decision or artifact
- Dependencies on earlier cycles
- Whether the cycle is mandatory or conditional

Prefer a focused campaign of approximately 3–6 total cycles, including this discovery cycle.

Do not create unnecessary cycles merely to address every boundary family independently. Group boundaries where they participate in the same authority or handoff problem.

10. Files to Provide for External Review

Conclude with a minimal, prioritized list of files that should be passed back for generation of the next instruction block.

Separate them into:

Required

Files necessary to understand the campaign findings and generate the next cycle.

Conditional

Files needed only if a specific ambiguity or pressure point must be examined directly.

Not Needed Unless Requested

Large source files, repetitive tests, generated outputs, logs, or documents that the report already summarizes adequately.

The required list should normally include:

- AR-BE_boundary_reconciliation_discovery_and_evidence_inventory.md
- Any authoritative prior campaign closurest or synthesis document not adequately summarized in AR-BE
- Any source file whose exact implementation is necessary to understand the highest-priority boundary issue

Keep this list minimal. The AR-BE report should carry as much relevant evidence as practical.

Evidence Standards

Every material conclusion must be grounded in one or more of:

- Source code
- Tests
- Architectural documentation
- Ownership or authority ledgers
- Runtime entry points
- Public interfaces
- Dependency relationships
- Persisted schemas
- Existing campaign findings

Clearly distinguish:

- Confirmed fact
 - Strong inference
 - Tentative interpretation
 - Missing evidence
- Use exact repository paths and symbol names wherever possible.
- Do not make claims based solely on naming conventions.
- ### Constraints
- Do not perform broad refactoring.
 - Do not rename architectural concepts.
 - Do not introduce new architectural vocabulary unless an actual missing responsibility requires it.
 - Do not reopen decisions already settled by prior campaigns without contradictory evidence.
 - Do not treat transitional implementation as conceptual confusion.
 - Do not treat defensive overlap as duplicate ownership without analyzing its purpose.
 - Do not assume every overlap is undesirable.
 - Do not collapse orchestration, authority, validation, projection, evidence, and transport into a single concept.
 - Do not define UI, persistence, adapters, plugins, tooling, or content as core runtime responsibilities merely because the runtime currently invokes them.
 - Do not prescribe implementation changes before identifying the governing boundary.
 - Do not modify production code unless a tiny, non-behavioral change is strictly necessary to inspect or document the architecture.
 - Preserve all existing tests and behavior.

Completion Standard

This cycle is complete when:

- Every major boundary family has been mapped to repository evidence
- Each boundary has a preliminary stability classification
- Permanent responsibility has been distinguished from implementation support
- Intentional overlap has been distinguished from transitional leakage
- Representative cross-boundary flows have been traced
- Genuine pressure points have been separated from incomplete implementation
- The likely remaining campaign cycles are clearly justified
- The external-review file list is minimal and explicit

Provide:

- permanent responsibility
- explicit exclusions
- overlap type
- authority relationship
- long-term stability

4. Remaining Transitional Overlaps

For each include:

- files
- symbols
- classification
- justification
- expected retirement (if applicable)

5. Dependency Map

Document:

- incoming responsibilities
- outgoing responsibilities
- authority direction
- ownership transfers

6. Semantic Repair Inventory

Catalog every remaining semantic repair location.

Classify each occurrence.

7. Architectural Stability Assessment

Determine:

- whether Final Emission is architecturally complete
- remaining architectural questions
- remaining implementation work
- remaining documentation work

Clearly separate these categories.

8. Recommended Next Cycle

Recommend the next Boundary Reconciliation cycle.

Use the established AR identifier sequence.

Explain why the proposed cycle naturally follows this validation.

9. Files Required For External Review

Provide the smallest possible list of files required to generate the next campaign instruction block.

Separate into:

Required

Conditional

Not Needed Unless Requested

Evidence Standards

Every material conclusion must be supported by one or more of:

- source code
- tests
- architectural documentation
- ownership ledgers
- previous campaign findings

Clearly distinguish:

- confirmed fact
- strong inference
- tentative interpretation
- missing evidence

Use exact repository paths and symbol names wherever possible.

Constraints

- Do not redesign Final Emission.
- Do not rename concepts.
- Do not broaden subsystem responsibilities.
- Assume previous campaigns remain authoritative.
- Preserve all existing tests.
- Treat defensive overlap as intentional unless repository evidence demonstrates otherwise.
- Distinguish implementation concerns from architectural concerns.
- Do not propose refactoring until ownership has first been established.
- Modify production code only if a tiny, non-behavioral change is strictly necessary for inspection or documentation.

Completion Standard

This cycle is complete when:

- every Final Emission responsibility has been classified
- every neighboring subsystem has clearly defined ownership
- every remaining overlap has been categorized
- semantic repair has been completely inventoried
- dependency direction has been documented
- permanent architecture has been separated from transitional implementation
- the report clearly states whether Final Emission is architecturally stable
- the next campaign cycle has been justified
- the external-review file list is minimal and explicit

At completion, report:

1. The path to AR-BF_final_emission_boundary_validation.md
2. A concise summary of findings
3. The recommended next cycle
4. Any unresolved evidence gaps
5. The exact files required for generation of the next instruction block

AR-BE – Boundary Reconciliation Campaign Closeout

Campaign Context

Campaign 4 has completed its objective.

Across the preceding cycles, the repository has been evaluated from three complementary perspectives:

- architectural boundaries
- Final Emission ownership
- long-term extensibility

The campaign has progressively shifted from architectural discovery toward architectural certification.

No subsequent cycle identified instability requiring redesign.

Instead, the remaining work has consistently become:

- implementation
- documentation
- governance
- future feature realization

This closeout formally determines whether Boundary Reconciliation is complete and records the permanent architectural doctrine recovered throughout Campaign 4.

Goal

Produce the definitive closeout assessment for Campaign 4.

Determine whether:

- Boundary Reconciliation is complete.
- The recovered architecture is internally consistent.
- A future implementation can proceed without further architectural reconciliation.
- Any remaining architectural questions are mandatory or merely optional future enhancements.

This document becomes the permanent architectural reference for all subsequent implementation campaigns.

Primary Evidence

Review all Campaign 4 reports:

- AR-BE_minimal_extensibility_contract_definition.md
- AR-BG_rulest_backend_and_version_provenance_boundary_inventory.md
- AR-BF_final_emission_boundary_validation.md
- AR-BE_boundary_reconciliation_discovery_and_evidence_inventory.md

Review supporting doctrine as needed:

- AR-BD_concept_map_synthesis_and_campaign_closeout.md
- AR-AI_final_vision_compatibility_closeout.md
- docs/architecture_ownership_ledger.md

Do not perform new architectural discovery unless repository evidence directly contradicts prior conclusions.

Required Investigation

1. Validate Campaign Goals

Determine whether Campaign 4 successfully established:

- durable subsystem boundaries
- permanent ownership
- dependency direction
- extensibility boundaries
- implementation separation

governance boundaries

For every goal state whether it is:

- Complete
- Substantially Complete
- Partially Complete
- Incomplete

Provide repository evidence.

2. Validate Architectural Stability

Determine whether the architecture is now stable regarding:

- runtime orchestration
- state authority
- CTR
- realization
- prompt adaptation
- Final Emission
- replay
- persistence
- provenance
- diagnostics
- ruleset identity
- backend identity

Identify any remaining architectural instability.

3. Produce the Permanent Boundary Inventory

For every permanent subsystem document:

- permanent responsibility
- explicit exclusions

evolving prompts

- replay longevity
- provenance continuity

If architectural work remains, identify only the minimum contracts necessary before future expansion.

Required Deliverable

Create:

AR-BG_rulest_backend_and_version_provenance_boundary_inventory.md

The document must contain the following sections.

1. Executive Summary

Overall assessment of the project's extensibility boundaries.

State whether future expansion appears architecturally supported.

2. Ruleset Boundary Assessment

Document:

- permanent runtime responsibilities
- permanent ruleset responsibilities
- implementation responsibilities
- remaining architectural questions

3. AI Backend Boundary Assessment

Document:

- backend responsibilities
- routing responsibilities
- realization responsibilities
- provider responsibilities
- remaining architectural questions

4. Version Identity Matrix

For every architectural id entity include:

- owner
- purpose
- current implementation
- future evolution expectations
- provenance implications

5. Provenance Ownership Matrix

For each provenance category include:

- owner
- downstream consumers
- replay relationship
- evidence relationship
- permanence

6. Dependency Boundary Map

Document:

- dependency direction
- extension seams
- compatibility seams
- architectural coupling
- implementation coupling

7. Future Expansion Assessment

Determine whether future work is primarily:

- implementation
- interface creation
- documentation
- governance
- architecture

Separate these categories clearly.

8. Architectural Readiness Assessment

Determine whether the project now possesses sufficiently mature extensibility boundaries to support years of feature expansion.

Identify only genuine remaining architectural gaps.

9. Recommended Next Cycle

Recommend the next Boundary Reconciliation cycle using the established AR identifier sequence.

Explain why it naturally follows this assessment.

10. Files Required For External Review

Provide the smallest possible list of files required to generate the next campaign instruction block.

Separate into:

Required

Conditional

Not Needed Unless Requested

Evidence Standards

Every material conclusion must be supported by one or more of:

- source code
- tests
- architectural documentation
- ownership ledgers
- previous campaign findings

Clearly distinguish:

- confirmed fact
- strong inference
- tentative interpretation
- missing evidence

Use exact repository paths and symbol names wherever possible.

Constraints

- Do not design alternate rulesets.
- Do not implement new AI providers.
- Do not create a generic plugin architecture.
- Do not redesign runtime orchestration.
- Do not rename architectural concepts.
- Assume previous campaigns remain authoritative.
- Distinguish architectural contracts from implementation details.
- Preserve all existing tests.
- Modify production code only if a tiny, non-behavioral change is strictly necessary for inspection or documentation.

Completion Standard

This cycle is complete when:

- ruleset boundaries have been fully classified
- AI backend boundaries have been fully classified
- version identities have been inventoried
- provenance ownership has been mapped
- dependency directions have been validated
- extensibility seams have been documented
- architectural contracts have been separated from implementation details
- the report states whether future expansion requires architectural evolution or primarily implementation
- the next campaign cycle has been justified
- the external-review file list is minimal and explicit

At completion, report:

1. The path to AR-BG_rulest_backend_and_version_provenance_boundary_inventory.md
2. A concise summary of findings
3. The recommended next cycle
4. Any unresolved evidence gaps
5. The exact files required for generation of the next instruction block

Document:

- required fields
- ownership
- lifecycle
- exclusions
- architectural rationale

3. Backend Contract

Document:

- required fields
- ownership
- routing relationship
- provider relationship
- architectural rationale

4. Version Bundle Definition

For every version identity include:

- owner
- permanence
- consumers
- evolution expectations

5. Provenance Bundle Definition

For every provenance identity include:

- owner
- producer
- consumers
- replay relationship
- diagnostics relationship

6. Contract Placement Matrix

For every contract specify:

- permanent architectural owner
- prohibited owners
- dependency direction
- implementation expectations

7. Architectural Readiness Assessment

Determine whether these contracts complete the architecture required for long-term extensibility.

Clearly separate:

- remaining architecture
- remaining implementation
- remaining documentation
- remaining governance

8. Recommended Next Cycle

Recommend the next Boundary Reconciliation cycle using the established AR identifier sequence. If no additional mandatory architectural reconciliation remains, explicitly state that Campaign 4 is ready for closeout and recommend a closure cycle.

9. Files Required For External Review

Provide the smallest possible list of files required to generate the next campaign instruction block.

Separate into:

Required

Conditional

Not Needed Unless Requested

Evidence Standards

Every material conclusion must be supported by one or more of:

- source code
- tests
- architectural documentation
- ownership ledgers
- previous campaign findings

Clearly distinguish:

- confirmed fact
- strong inference
- tentative interpretation
- missing evidence

Use exact repository paths and symbol names wherever possible.

Constraints

- Do not implement alternate rulesets.
- Do not implement backend adapters.
- Do not redesign runtime orchestration.
- Do not create a generic plugin architecture.
- Do not expand subsystem responsibilities.
- Assume previous campaigns remain authoritative.
- Preserve all existing tests.
- Modify production code only if a tiny, non-behavioral change is strictly necessary for inspection or documentation.

Completion Standard

This cycle is complete when:

- the minimum ruleset contract has been defined
- the minimum backend contract has been defined
- the architectural version bundle has been defined
- the architectural provenance bundle has been defined
- contract ownership has been validated
- dependency placement has been justified
- implementation has been separated from architecture
- the report states whether Campaign 4 is ready for closeout
- the external-review file list is minimal and explicit

At completion, report:

- This cycle is complete when:
- Every major boundary family has been mapped to repository evidence
 - Each boundary has a preliminary stability classification
 - Permanent responsibility has been distinguished from implementation support
 - Intentional overlap has been distinguished from transitional leakage
 - Representative cross-boundary flows have been traced
 - Genuine pressure points have been separated from incomplete implementation
 - The likely remaining campaign cycles are clearly justified
 - The external-review file list is minimal and explicit
 - No broad architectural changes have been performed
- At completion, report:
1. The path to AR-BE_boundary_reconciliation_discovery_and_evidence_inventory.md
 2. A concise summary of the findings
 3. The recommended next cycle
 4. Any unresolved evidence gaps
 5. The exact files that should be provided for generation of the next instruction block

- diagnostics
 - relevant identity
 - backend identity
- Identify any remaining architectural instability.

3. Produce the Permanent Boundary Inventory

- For every permanent subsystem document:
- permanent responsibility
 - explicit exclusions
 - architectural owner
 - downstream consumers
 - expected long-term stability
- Include all major architectural boundaries.

4. Produce the Permanent Dependency Map

- Document the canonical dependency direction between:
- runtime
 - domain simulation
 - CTIR
 - prompt context
 - realization
 - Final Emission
 - persistence
 - replay
 - diagnostics
 - governance
- This becomes the project's canonical dependency diagram.

5. Separate Future Work

- Classify every remaining task into exactly one category:
- Implementation
 - Documentation
 - Governance
 - Product Decision
 - Optional Future Architecture
- No remaining item should remain classified as unresolved architecture unless repository evidence requires it.

6. Readiness Assessment

- Determine whether the repository is now architecturally prepared for:
- long-term implementation
 - future rulesets
 - additional AI providers
 - replay longevity
 - save compatibility
 - UI evolution
 - external tooling
- Support every conclusion with repository evidence.

Required Deliverable

- Create:
- AR-BI_boundary_reconciliation_campaign_closeout.md
- The document must contain the following sections.
- 1. Executive Summary**
- Overall assessment of Campaign 4.
- State clearly whether Boundary Reconciliation is complete.

2. Campaign Objective Assessment

Evaluate every original Campaign 4 objective.

3. Permanent Architectural Doctrine

Document the recovered architectural doctrine that should now be considered stable.

4. Permanent Boundary Inventory

- List every permanent subsystem together with:
- ownership
 - exclusions
 - stability
 - evidence

5. Canonical Dependency Model

Document the authoritative dependency direction for the runtime.

6. Remaining Work Classification

- Separate all remaining work into:
- Implementation
 - Documentation
 - Governance
 - Product Decisions
 - Optional Future Architecture

7. Campaign Closeout Decision

- Answer explicitly:
- Is Campaign 4 complete?
 - Is additional Boundary Reconciliation required?
 - Is implementation now the recommended focus?
- Support every answer with repository evidence.

8. Recommendations

Recommend the next major campaign after Boundary Reconciliation.

Explain why it follows naturally.

9. Files Required For Future Campaigns

- Provide the minimum file set required for future implementation-focused architectural reviews.
- Separate into:
- Core Doctrine**
 - Frequently Needed**
 - Rarely Needed**

Evidence Standards

- Every material conclusion must be supported by:
- source code
 - tests
 - architectural documentation
 - ownership ledgers
 - previous campaign findings
- Clearly distinguish:
- confirmed fact
 - strong inference
 - tentative interpretation
 - missing evidence

Constraints

- Do not redesign the architecture.
- Do not reopen previously reconciled concepts.
- Do not introduce speculative future systems.
- Treat previous Campaign 4 conclusions as authoritative unless contradicted by repository evidence.
- Preserve all existing tests.
- Focus on consolidation rather than discovery.

Completion Standard

- This cycle is complete when:
- Campaign 4 has been formally closed.
 - Permanent architectural doctrine has been documented.
 - Permanent subsystem boundaries have been recorded.
 - Canonical dependency direction has been finalized.
 - Remaining work has been categorized.
 - The report clearly states whether implementation should become the primary focus.
 - A minimal doctrine file set has been identified for future campaigns.

- At completion, report:
1. The path to AR-BI_boundary_reconciliation_campaign_closeout.md
 2. A concise summary of Campaign 4
 3. Whether Boundary Reconciliation is officially complete
 4. The recommended next major campaign
 5. The exact doctrine files needed going forward

C5: Coordination Cost Reduction

Friday, July 17, 2026 3:12 PM

AR-BI — Coordination Cost Inventory and Implementation Workflow Discovery

Goal
Begin Campaign 5 by producing a repository-grounded inventory of the coordination required to implement, validate, document, and maintain features within the architecture established during Campaigns 1–4.
This cycle determine where development effort is currently spent across ownership boundaries, contracts, adapters, compability layers, governance mechanisms, tests, documentation, and feature -delivery workflows.
The purpose is not to simplify the architecture, reap settled ownership decisions, or treat cross -module coordination as inherently undesirable.
Instead, establish an evidence base that distinguishes:

- coordination required to preserve correct architectural separation,
- coordination caused by incomplete contract publication or tooling,
- coordination caused by compatibility or transitional residue,
- coordination caused by duplicated implementation practices,
- coordination caused by test or documentation structure,
- and coordination that may be safely reduced without weakening the recovered doctrine.

Architectural Premise
Treat the architecture recovered and validated during Campaigns 1–4 as the governing baseline.

Assume that the following are intentional unless repository evidence demonstrates otherwise:

- canonical ownership,
- explicit boundary crossing,
- provenance preservation,
- replay safety realization,
- evidence-bearing decisions,
- defensive runtime boundaries,
- adapter separation,
- validator and repair distinctions,
- governance checks,
- and modular velocity/backed extensibility.

Do not recommend architectural consolidation merely because a feature requires edits in multiple modules.

A multi-file or multi-module change is not automatically excessive fan -out. Determine whether each edit reflects a legitimate responsibility or additive implementation friction.

- Primary Questions**
1. What does a representative feature implementation currently require from discovery through completion?
 2. Which modules, contracts, tests, documentation artifacts, registries, adapters, validators, provenance structures, or governance checks commonly require coordinated edits?
 3. Which recurring edits reflect legitimate ownership boundaries?
 4. Which recurring edits duplicate the same information or decision across multiple locations?
 5. Where must developers infer contracts from implementations rather than consume clearly published interfaces, schemas, protocols, registries, or documentation?
 6. Where do canonical owners experience excessive pressure because too many unrelated changes must pass through them?
 7. Which compability, migration, historical, or transitional paths still increase implementation effort?
 8. Which governance mechanisms provide meaningful protection, and which may impose repetitive manual work that could be automated or centralized without reducing enforcement?
 9. Which tests are coupled to implementation details rather than stable behavioral or boundary contracts?
 10. Which documentation must be updated manually in several places for the same conceptual change?
 11. Where do vocabulary, enums, field definitions, status names, provenance labels, routing concepts, or ownership declarations appear in multiple modules?
 12. Which coordination major or critical changes are required by the current architecture?
 13. Which costs are likely to disappear when compability layers mature or are retired?
 14. Which costs appear reducible through implementation improvements rather than architectural redesign?

Required Investigation

A. Representative Feature -Change Tracing

Identify several recent or structurally representative feature changes from repository history, documentation, tests, T000s, implementation notes, or current code structure.

Prefer examples that exercise different architectural path, such as:

- introducing or changing a ruselet capability,
- adding or changing an AI/backed integration,
- modifying realization behavior,
- extending replay or final emission behavior,
- adding provenance or evidence,
- changing routing or ownership declarations,
- modifying fallback behavior,
- adding a validator, repair, or sanitizer,
- changing an API-facing contract,
- and retiring or bypassing compability behavior.

For each selected example, reconstruct the likely implementation workflow:

1. entry point or initiating requirement,
2. canonical owner,
3. dependent contracts,
4. adapters or projections,
5. validation or repair paths,
6. provenance/evidence effects,
7. replay implications,
8. tests requiring modification,
9. documentation requiring modification,
10. compability considerations,
11. governance checks,
12. and final verification.

Record the files and responsibilities involved.

Do not judge edit count alone. Explain why each edit is or is not architecturally justified.

B. Edits Fan-Out Inventory

Search for groups or concepts that require coordinated edits across multiple files.

Look for:

- duplicated enums or literals,
 - repeated field names,
 - parallel registries,
 - manually synchronized mappings,
 - duplicated protocol descriptions,
 - repeated ownership declarations,
 - repeated routing labels,
 - repeated provenance labels,
 - repeated compability branches,
 - repeated test fixtures,
 - duplicated constructors or object assembly,
 - repeated serializer/deserializer logic,
 - mirrored validation rules,
 - and documentation that restates executable contracts.
- For each fan-out pattern, classify the edits as:
- intentional responsibility distribution,
 - contract propagation,
 - adapter maintenance,
 - compability impact,
 - duplicated implementation knowledge,
 - test duplication,
 - documentation duplication,
 - governance duplication,
 - or uncertain.

C. Canonical Owner Pressure

Identify modules or components that act as canonical owners for important state, decisions, contracts, schemas, replay data, realization, provenance, routing, or governance.

Determine:

- how many unrelated feature categories require modifications to each owner,
- whether the owner exposes adequate extension points,
- whether callers must understand internal implementation details,
- whether the owner contains broad conditional dispatch,
- whether registrations are centralized appropriately,
- whether the module is large because it owns a coherent responsibility,
- and whether pressure comes from architecture or missing implementation mechanisms.

Do not recommend splitting a canonical owner merely because it is frequently edited.

Determine whether its edit frequency is expected for an authoritative component or whether extension mechanisms, declarative registration, generated contracts, helpers, or clearer APIs could reduce unnecessary edits.

D. Contract Publication Assessment

Identify important contracts developers must understand to implement features correctly.

Include, where applicable:

- interfaces,
 - protocols,
 - schemas,
 - typed structures,
 - datamodels/models,
 - registries,
 - ownership ledgers,
 - routing declarations,
 - provenance requirements,
 - replay contracts,
 - realization contracts,
 - adapter contracts,
 - validator contracts,
 - repair contracts,
 - sanitizer contracts,
 - diagnostic contracts,
 - and evidence requirements.
- For each contract, determine whether it is:
- explicitly published and discoverable,
 - executable but poorly documented,
 - documented but not discoverable,
 - inferred from tests,
 - inferred from implementation,
 - duplicated across several locations,
 - transitional,
 - or absent.

Identify places where better publication could reduce developer coordination without changing responsibility or authority.

E. Compability and Transitional Residue

Locate compability paths, deprecated contracts, historical aliases, blisack shifts, migration adapters, legacy projections, defensive bridges, or transitional ownership accommodations that still affect feature work.

For each item, determine:

- what it protects,
 - whether it is still active,
 - whether evidence would permit retirement,
 - which files or workflows it complicates,
 - whether it forces duplicate implementation,
 - whether it increases test combinations,
 - and whether its cost is temporary, indefinite, or already unnecessary.
- Do not remove or recommend removal without repository evidence and explicit retirement criteria.
- F. Cross-Module Vocabulary Duplication
- Search for recurring architectural vocabulary represented independently across modules.
- Examples include:
- state names,
 - lifecycle phases,
 - result categories,
 - provenance labels,
 - replay modes,
 - realization statuses,
 - fallback reasons,
 - validator outcomes,
 - repair classifications,
 - routing destinations,

AR-BK — Coordination Cost Classification and Reduction Opportunity Analysis

Goal
Transform the coordination-cost inventory produced during AR-BI into a repository-grounded prioritization framework.
Do not search broadly for new coordination costs unless repository evidence contracts AR-BI.

Instead, classify, quantify, and rank the implementation effort associated with the coordination costs already identified, determining which represent:

- permanent architectural investment,
- implementation friction,
- compability residue,
- governance cost,
- documentation synchronization,
- test maintenance,
- or genuine implementation improvement opportunities.

The purpose of this cycle is not to redesign architecture.

The purpose is to identify where future implementation effort can safely decrease while preserving the architectural doctrine recovered during Campaigns 1–4.

Architectural Premise

Assume the architectural boundaries recovered during Campaigns 1–4 remain correct.

Do not recommend:

- ownership redistribution,
- authority movement,
- collapsing canonical owners,
- replay simplification,
- provenance simplification,
- governance weakening,
- adapter removal,
- validator/repair merging,
- or Final Emission responsibility reduction

unless repository evidence demonstrates that the existing architecture itself—not merely implementation—is incorrect.

Required Investigation

The objective is to determine whether implementation effort can decrease while architectural separation remains intact.

Primary Questions

1. How frequently is this coordination expected to occur?
2. How many implementation workflows require it?
3. How many files typically participate?
4. How much of the work exists because ownership is correct?
5. How much exists because implementation lacks better tooling or publication?
6. Which implementation improvements would reduce future effort without changing ownership?
7. Which improvements would permanently reduce effort?
8. Which improvements disappear naturally after compability retirement?
9. Which improvements would create architectural risk?
10. Which improvements provide the highest implementation ROI?

Required Investigation

A. Coordination Cost Quantification

For each coordination cost from AR-BI estimate repository-supported:

- implementation frequency,
- implementation surface,
- affected owners,
- expected edit fan-out,
- governance involvement,
- replay involvement,
- provenance involvement,
- documentation updates,
- testing impact,
- compability impact.

Use repository evidence rather than intuition.

B. Architectural vs Implementation Cost

For every coordination item determine approximately:

Architectural Portion

- preservation ownership
 - preserving authority
 - preserving replay
 - preserving provenance
 - preserving governance
- Implementation Portion**
- repetitive edits
 - manual synchronization
 - duplicated fixtures
 - missing helpers
 - weak contract publication
 - compability maintenance
 - generated artifacts
 - documentation duplication

Separate these explicitly.

C. Opportunity Classification

Every candidate improvement should be classified into categories such as:

- higher extraction
- workflow improvement
- builder/future improvement
- registry publication
- documentation generation
- code generation
- compability retirement
- adapter improvement
- contract publication
- governance automation
- reporting automation
- developer guidance
- no action recommended

D. Risk Assessment

For every opportunity determine:

Architectural Risk

- none
- low
- medium
- high

Replay Risk

Governance Risk

Compability Risk

Ownership Risk

Maintenance Risk

Only recommend opportunities whose implementation risk is clearly understood.

E. ROI Assessment

Estimate relative implementation benefit.

Possible categories:

Very High

High

Medium

Low

Negligible

ROI should consider:

- expected developer time saved,
- reduced coordination,
- lower cognitive load,
- reduced documentation burden,
- reduced test maintenance,
- preservation of architecture.

F. Dependency Mapping

Identify prerequisite relationships.

Examples:

Backend contract publication before backend expansion.

Ruselet contract publication before alternate ruselets.

Compability retirement before vocabulary reduction.

Registry publication before code generation.

Generated documentation before manual synchronization removal.

Do not recommend dependent work out of order.

G. Future Roadmap Influence

Determine whether implementation priorities should be reordered.

Do not reorder simply because work appears easier.

Only reorder if reducing implementation friction first would significantly improve future development.

Required Deliverable

Create:

AR-BK_coordination_cost_classification_and_reduction_opportunities.md

The document should contain:

1. Executive Summary

Summarize:

- overall coordination profile,
- dominant cost categories,
- architectural health,
- implementation health,
- highest-value opportunities,
- 2. Coordination Cost Ranking
- 3. [Rank] | Coordination Cost | Frequency | Impact | Classification | Architectural Portion | Implementation Portion |
- 4. Opportunity Matrix
- 5. [Opportunity] | Targets | Cost Category | Expected Benefit | Architectural Risk | Replay Risk | Governance Risk | ROI |
- 6. Architectural vs Implementation Breakdown

Provide a narrative explaining where implementation—not architecture—is responsible for remaining cost.

7. Compability Opportunity Assessment

For every compability residue identified in AR-BI:

- expected lifetime,
- retirement evidence,
- expected implementation savings,
- architectural impact,
- 6. Governance Opportunity Assessment
- Identify automation opportunities.
- Preserve governance strength.
- 7. Testing Opportunity Assessment

Focus on:

- builders,
 - futures,
 - generated assertions,
 - reusable verification,
 - owner-preserving improvements.
8. Documentation Opportunity Assessment
- Determine:
- executable contracts suitable for documentation generation,
 - documents that should remain handcrafted,
 - historical documents that should never become generated.
9. Dependency Graph

Present implementation improvements in dependency order.

AR-BL — Implementation Workflow Optimization Planning

Goal
Design a repository-grounded implementation workflow that reduces developer coordination effort while fully preserving the architectural doctrine validated during Campaigns 1–4 and classified during AR-BI and AR-BK.

This cycle should not implement improvements.

Instead, produce a practical implementation strategy describing how future repository work should be performed so that coordination costs naturally decrease without altering ownership, replay integrity, provenance, governance, or extensibility.

Architectural Premise

Treat the recovered architecture as stable.

Treat the implementation opportunities identified during AR-BK as the implementation backlog—not architectural defects.

Every repository item must preserve:

- canonical ownership,
- replay correctness,
- provenance,
- governance,
- adapter isolation,
- validator/repair separation,
- Final Emission authority,
- state authority,
- architectural boundaries.

Primary Questions

1. What should the standard workflow for implementing a new feature become?
2. Which reusable implementation patterns should developers consistently follow?
3. Which implementation activities should become standardized?
4. Which coordination steps can become predictable checklists?
5. Which implementation improvements should become future infrastructure projects?
6. Which implementation work should always occur before feature expansion?

Required Investigation

A. Feature Workflow Blueprint

Construct the ideal workflow for implementing:

- ordinary gameplay features,
- Final Emission changes,
- replay changes,
- provenance changes,
- ruselet extensions,
- backend extensions,
- compability retirement,
- governance changes,
- required contracts,
- required adapters,
- required replay changes,
- required tests,
- required governance,
- required code.

For each workflow identify:

- canonical owner,
- required contracts,
- required adapters,
- required replay changes,
- required tests,
- required governance,
- required code.

B. Standard Coordination Checklists

Develop reusable implementation checklists for common feature categories.

Examples:

Protected replay field

Final Emission metadata

Backend compability

Ruselet capability

Fallback changes

Testing impact

Repair additions

Compability retirement

Governance additions

Documentation updates

These should become repeatable developer workflows.

Architectural Pattern Catalog

Identify implementation patterns that should become repository standards.

Examples:

- builder patterns,
- future patterns,
- registry publication,
- compability documentation,
- adapter patterns,
- verification sequences,
- compability evidence collection,
- governance refresh procedures.

D. Workflow Simplification Opportunities

Without changing architecture identity.

Examples:

- repeated manual work,
- repeated verification,
- repeated documentation,
- repeated feature creation,
- repeated evidence gathering,
- repeated compability analysis,
- repeated manual work,
- repeated verification,
- repeated documentation,
- repeated feature creation,
- repeated evidence gathering,
- repeated compability analysis.

Describe how each could become standardized.

E. Implementation Infrastructure Roadmap

Organize future implementation improvements into independent infrastructure initiatives.

Examples:

- Backend contract publication
- Ruselet contract publication
- Test builders
- Governance documentation
- Compability register
- Verification workflow
- Governance automation

Each initiative should have:

purpose

dependencies

architectural constraints

expected benefit

implementation risk

verification requirements

F. Developer Experience Assessment

How difficult is it for a new contributor to correctly implement a feature?

Which knowledge currently depends upon repository familiarity?

Which knowledge could become discoverable through tooling, documentation, registries, or workflow guides?

Required Deliverable

Create:

AR-BL_implementation_workflow_optimization_planning.md

The document should contain:

1. Executive Summary

Summarize the overall implementation strategy.

2. Standard Feature Workflow

Document the preferred implementation lifecycle.

3. Workflow Variants

Separate workflows for:

- gameplay features
- Final Emission
- replay
- provenance
- backend
- ruselets
- compability
- governance

Backend contract Checklists

Provide reusable implementation checklists.

5. Repository Implementation Standards

Only recommend implementation conventions.

6. Infrastructure Initiative Catalog

Create a table:

[Initiative | Purpose | Dependencies | Architectural Constraints | Benefit | Risk | Priority]

7. Developer Experience Assessment

Describe current onboarding friction and opportunities.

8. Implementation Readiness Assessment

Evaluate whether future feature work can proceed efficiently under the current doctrine.

9. Campaign Assessment

Determine whether Campaign 5 has sufficiently transitioned from analysis into actionable implementation planning.

10. Recommended Next Cycle

Recommend the next Campaign 5 cycle.

Expected direction:

AR-BM — Infrastructure Initiative Prioritization and Sequencing

Only recommend a different direction if repository evidence strongly supports it.

11. Files Required for External Review

Separate into:

Required

Optional

Execution Constraints

• Do not modify production code.

• Do not refactor architecture.

• Do not merge owners.

• Do not simplify replay boundaries.

• Do not simplify provenance.

• Do not weaken governance.

• Preserve compability unless retirement evidence already exists.

• Produce implementation plans only.

Success Criteria

This cycle succeeds when it provides a repeatable implementation methodology that future development can follow while naturally reducing coordination costs without architectural redesign.

Stop Condition

Stop after producing the planning document.

Do not begin implementing infrastructure initiatives.

Do not retire compability.

Do not automate governance.

Do not modify repository structure.

Implementation Scope

AR-BD — Package 1 Implementation: Feature-Lane Verification Goal

Goal

Implement Package 1 exactly as defined in AR-BN.

This is the first implementation cycle of Campaign 5.

Architectural Premise

Treat AR-BN as the implementation specification.

Do not reinterpret architecture.

Do not expand scope.

Implement only the approved documentation package.

Implementation Scope

- lifecycle phases,
 - result categories,
 - provenance labels,
 - replay modes,
 - realization statuses,
 - fallback reasons,
 - validator outcomes,
 - repair classifications,
 - routing destinations,
 - ownership identifiers,
 - evidence types,
 - diagnostic codes,
 - backend identifiers,
 - ruleset identifiers,
 - and compatibility modes.
- Determine whether each duplication represents:
- a shared concept that should have one unified definition,
 - intentionally distinct concepts with similar names,
 - evidence-level translation,
 - compatibility residue,
 - or accidental duplication.
- Do not merge vocabulary based only on naming similarity.

- G. Adapter-Boundary Cost
- Inspect adapters between canonical owners and external or alternate representations.
- Include:
- API adapters,
 - backend adapters,
 - ruleset adapters,
 - and serialization boundaries,
 - replay projections,
 - final emission projections,
 - compatibility adapters,
 - UI-facing projections,
 - persistence adapters,
 - and diagnostic/reporting projections.
- Determine:
- whether adapters isolate change effectively,
 - whether they duplicate business decisions,
 - whether mappings are declarative or handwritten,
 - whether ownership is clear,
 - and whether new feature fields must be manually threaded through many adapters,
 - and whether tooling or contract generation could reduce effort.

- Preserve adapters that present external formats from becoming authoritative.
- H. Governance Overhead
- Inventory governance mechanisms that developers must satisfy.
- Examples include:
- architectural ownership tests,
 - import restrictions,
 - conformance tests,
 - provenance requirements,
 - replay checks,
 - evidence checks,
 - linting rules,
 - invariant assertions,
 - documentation requirements,
 - boundary inventories,
 - registries,
 - review checklists,
 - and closeout artifacts.
- For each mechanism, determine:
- the failure it prevents,
 - whether that failure remains plausible,
 - whether enforcement is automated,
 - whether developers must duplicate evidence manually,
 - and whether error messages identify the corrective action,
 - whether checks overlap,
 - and whether the same protection could be retained with less manual coordination.
- Do not weaken governance merely to improve convenience.
- I. Test Maintenance Cost
- Identify test patterns that amplify coordination effort.
- Look for:
- widespread future changes from small contract extensions,
 - repeated fixture construction,
 - snapshot churn,
 - tests coupled to internal call ordering,
 - duplicate boundary tests,
 - compatibility matrix expansion,
 - repeated mock configuration,
 - large integration setup,
 - brittle import-path assumptions,
 - repeated expected payloads,
 - and tests that restate implementation details.

- Classify each cost as:
- necessary behavioral coverage,
 - necessary boundary/governance coverage,
 - compatibility coverage,
 - future/tooling deficiency,
 - excessive implementation coupling,
 - duplicated coverage,
 - or uncertain.
- Identify opportunities such as reusable builders, contract-focused fixtures, shared assertions, generated cases, or clearer test layers only when they preserve explicit protection.
- J. Documentation Burden
- Identify documentation that must change during implementation.
- Determine:
- which documents are authoritative,
 - which documents summarize executable truth,
 - which repeat information already encoded elsewhere,
 - which are historical records,
 - which are required governance artifacts,
 - which are stale,
 - and which could be generated or linked rather than manually synchronized.
- Preserve historical campaign records and architectural decision evidence.
- Do not treat historical documentation as duplication merely because newer summaries exist.
- K. Feature Implementation Workflow
- Construct a repository-grounded description of the current expected workflow for implementing a nontrivial feature.
- The workflow should cover:
- discovering the canonical owner,
 - identifying affected contracts,
 - determining replay/provenance/evidence implications,
 - implementing authoritative behavior,
 - updating adapters and projections,
 - handling compatibility,
 - updating tests,
 - satisfying governance,
 - updating documentation,
 - and verifying the completed change.

- Identify where the workflow is:
- clear and efficient,
 - intentionally careful,
 - dependent on expert memory,
 - repetitive,
 - weakly documented,
 - insufficiently automated,
 - or obstructed by residue.
- Evidence Requirements
- Use direct repository evidence.
- Evidence may include:
- source files,
 - tests,
 - architectural ledgers,
 - campaign artifacts,
 - ownership documentation,
 - conformance reports,
 - schema,
 - registries,
 - interfaces/protocols,
 - adapters,
 - compatibility modules,
 - git history if available,
 - TODO/FIXME notes,
 - developer documentation,
 - and representative feature implementations.
- For every major claim, provide:
- file path,
 - relevant symbol, section, test, or declaration,
 - observed coordination requirement,
 - responsible architectural boundary,
 - and preliminary classification.
- Avoid speculative recommendations unsupported by repository evidence.
- Required Classification Mode
- Classify each discovered coordination cost using one primary category:
1. INTENTIONAL_ARCHITECTURAL_COORDINATION
- Coordination required to preserve distinct responsibilities, canonical ownership, evidence, replay integrity, provenance, extensibility, or governance.
2. ALREADY_MINIMIZED
- A project has an expensive concern that the current architecture already contains effectively through a canonical owner, adapter, registry, contract, helper, or automated check.
3. IMPLEMENTATION_FRICTION
- Effort caused primarily by tooling, repetitive wiring, unclear APIs, missing helpers, weak contract publication, fixture design, manual synchronization, or workflow deficiencies.
4. COMPATIBILITY_RESIDUE
- Effort caused by supported historical forms, migration paths, aliases, shims, defensive bridges, or transitional behavior.
5. GOVERNANCE_OVERHEAD
- Effort imposed by enforcement, evidence production, review requirements, or conformance mechanisms.
6. DOCUMENTATION_SYNCHRONIZATION
- Effort caused by maintaining several descriptions of the same current contract or workflow.
7. TEST_MAINTENANCE_FRICTION
- Effort caused by fixture structure, implementation coupling, duplicated setup, compatibility matrices, or weak test abstractions.
8. GENUINE_SIMPLIFICATION_OPPORTUNITY
- A repository supported opportunity to reduce implementation effort without weakening ownership, replay correctness, provenance, extensibility, evidence quality, or governance.
9. UNCERTAIN
- Insufficient evidence exists to classify the cost safely.
- Also assign:
- permanence: permanent, transitional, historical, or uncertain,
 - reduction potential: none, low, medium, high, or uncertain,
 - architectural risk if changed: low, medium, high, or unknown,
 - rely mechanism: documentation, tooling, automation, helper API, contract publication, future improvement, code generation, adapter improvement, compatibility retirement, workflow clarification, or none.

- Required Deliverable
- reusable verification,
- owner-preserving improvements,
8. Documentation Opportunity Assessment
- Determine:
- executable contracts suitable for documentation generation,
 - documents that should remain handcrafted,
 - historical documents that should never become generated.
9. Dependency Graph
- Present implementation improvements in dependency order.
10. Prioritized Implementation Roadmap
- Group into:
- Immediate
 - New-Term
 - Long-Term
- Future Compatibility Retirement
- 1. Items Explicitly Rejected
- List ideas that appear attractive but would weaken:
- ownership,
 - replay,
 - provenance,
 - governance,
 - extensibility,
 - architectural doctrine.
- Explain why they should not be pursued.
12. Campaign Assessment
- Answer:
- Is Campaign 5 progressing as expected?
 - Have any architectural concerns emerged?
 - Are remaining costs primarily implementation concerns?
 - Is roadmap reordering warranted?
13. Recommended Next Cycle
- Recommend the next Campaign 5 cycle.
- Expected direction:
- AR-BL — Implementation Workflow Optimization Planning
- Only recommend another direction if repository evidence strongly justifies it.
14. Files Required for External Review
- Separate into:
- Required
- Optional
- Execution Constraints
- Do not modify production code.
 - Do not recommend architectural simplification without repository evidence.
 - Preserve canonical ownership.
 - Preserve replay integrity.
 - Preserve provenance.
 - Preserve Final Emission authority.
 - Preserve architectural doctrine from Campaigns 1-4.
- Success Criteria
- This cycle succeeds when it produces a defensible prioritization of implementation improvements while clearly demonstrating that architectural quality and implementation efficiency are separate concerns.
- Stop Condition
- Stop after producing the classification document.
- Do not begin implementing improvements.
- Do not begin compatibility retirement.
- Do not begin governance automation.
- Do not begin refactoring.

- AR-BN — Implementation Package Definition
- Goal
- Define the first implementation-ready infrastructure package identified during Campaign 5 planning.
- Do not implement the package.
- Instead, produce a complete execution specification that can be handed to a future implementation campaign with minimal additional planning.
- Architectural Premise
- Assume the following have been completed:
- Campaigns 1-4 recovered and validated the architecture.
 - AR-BJ inventoried coordination costs.
 - AR-BK classified reduction opportunities.
 - AR-BL established implementation methodology.
 - AR-BM prioritized and sequenced implementation initiatives.
- No additional architectural discovery is required unless repository evidence contradicts previous findings.
- Primary Questions
1. What is the exact scope of the first implementation package?
 2. Which files should be modified?
 3. Which files must remain untouched?
 4. What constitutes successful completion?
 5. Which verification tests prove the package is complete?
 6. Which risks must be monitored during implementation?
- Required Investigation
- A. Package Scope Definition
- Define the implementation boundaries for the first initiative: Feature-Lane Verification Guide
- Specify:
- exact objectives,
 - expected repository artifacts,
 - excluded work,
 - architectural assumptions,
 - ownership boundaries.
- B. File Impact Analysis
- Produce three categories:
- Expected Changes
 - Supporting References
 - Must Not Change
- Include justification for every classification.
- C. Acceptance Criteria
- Define objective completion criteria.
- Each criterion should be independently verifiable.
- Separate:
- Required
- Options
- Future Enhancement
- D. Verification Procedure
- Specify:
- documentation reviews,
 - cross-reference validation,
 - verification,
 - consistency checks,
 - repository searches,
 - governance review,
 - any existing automated verification that applies.
- Do not invent new governance unless justified.
- E. Risk Review
- Evaluate:
- architectural risk,
 - documentation drift,
 - workflow ambiguity,
 - future maintenance,
 - developer onboarding,
 - governance consistency.
- Provide mitigation strategies.
- F. Success Package
- Describe exactly what a reviewer should receive after implementation.
- Examples:
- updated documentation,
 - new guide,
 - cross-links,
 - repository index updates,
 - review checklist,
 - verification log.
- Required Deliverable
- Create:
- AR-BN_feature_lane_verification_package_definition.md
- The document should contain:
1. Executive Summary
 2. Package Scope
- Describe the package and its purpose.
3. Repository Impact
- Tables:
- Expected Changes
- Supporting References
- Protected Files- 4. Acceptance Criteria

List measurable completion requirements.

4. Verification Plan

Provide implementation verification steps.

6. Risk Assessment

Summarize implementation risks and mitigations.

7. Deliverables

Enumerate all expected implementation outputs.

8. Package Closeout Criteria

Define when the package is considered complete.

9. Campaign Assessment

Answer:

Is this package ready for implementation?

Are additional planning cycles necessary?

Can implementation begin after approval?

10. Recommended Next Cycle

Recommend one of:

Proceed to implementation of Package 1

Define Package 2 (Compatibility-Residue Register)

Return to planning (only if justified by new evidence)

Support the recommendation with repository evidence.

11. Files Required for External Review

Separate into:

Required

Optional

Execution Constraints

 - Do not modify production code.
 - Do not implement documentation changes.
 - Do not update governance.
 - Do not publish contracts.
 - Do not introduce builders.
 - Do not automate tooling.
 - Produce only an implementation specification.

Success Criteria

This cycle succeeds when the first implementation initiative is demonstrably executed, independently

- Goal
- Implement Package 1 exactly as defined in AR-BN.
- This is the first implementation cycle of Campaign 5.
- Architectural Premise
- Treat AR-BN as the implementation specification.
- Do not reinterpret architecture.
- Do not expand scope.
- Implement only the approved documentation package.
- Implementation Scope
- Create the Feature-Lane Verification Guide.
- Expected implementation:
- Create docs/feature_lane_verification.md (or an equivalent repository-approved location if justified).
 - Add discoverability links from:
 - o tests/README_TESTS.md
 - o docs/convergence_ci_inventory.md
 - Include:
 - o feature-lane overview,
 - o verification lane table,
 - o owner categories,
 - o replay/provenance considerations,
 - o governance considerations,
 - o documentation expectations,
 - o reviewer checklist.
- Out of Scope
- Do not:
- modify production code,
 - change tests,
 - publish backend contracts,
 - publish ruleset contracts,
 - introduce builders,
 - automate governance,
 - update generated artifacts,
 - modify replay schemas,
 - alter compatibility behavior.
- Required Verification
- Verify that:
- all required lanes exist,
 - documentation links resolve,
 - no protected files changed,
 - no production code changed,
 - no governance behavior changed,
 - package matches AR-BN acceptance criteria.
- Required Deliverable
- Produce:
- AR-B0_feature_lane_verification_package_implementation.md
- Include:
1. Executive Summary
 2. Files Modified
 3. Implementation Summary
 4. Acceptance Criteria Checklist
 5. Verification Results
 6. Deviations (if any)
 7. Risks Encountered
 8. Package Closeout
 9. Campaign Assessment
- Answer:
- Was Package 1 successfully implemented?
 - Were architectural boundaries preserved?
 - Is Package 2 now ready?
 - 10. Recommendation
- Recommended one of:
- Proceed to Package 2 (Compatibility-Residue Register)
 - Additional files required before Package 2
11. Files Required for External Review
- Separate into:
- Required
- Optional
- Success Criteria
- Package 1 is fully implemented exactly as specified by AR-BN.
- No architectural behavior changes.
- No production code modifications.
- No expansion beyond approved scope.
- Stop Condition
- Stop immediately after Package 1 implementation and verification.
- Do not begin Package 2.

- AR-B0 — Package 3 Implementation: Governance Refresh Workflow
- Goal
- Implement the Governance Refresh Workflow identified during Campaign 5.
- The objective is to make governance maintenance predictable by documenting when repository governance artifacts must be refreshed, which artifacts are authoritative, and how contributors determine whether a change requires governance updates.
- This package improves governance workflow.
- It does not change governance behavior.
- Architectural Premise
- Assume:
- Campaigns 1-4 recovered the architecture.
 - Package 1 implemented the Feature-Lane Verification Guide.
 - Package 2 implemented the Compatibility-Residue Register.
 - Existing governance checks, inventories, manifests, and reports remain architecturally correct.
- This package documents how governance artifacts are maintained.
- It does not alter governance policy.
- Implementation Scope
- Create a Governance Refresh Workflow document that explains:
- how governance artifacts require refresh,
 - which artifacts are generated,
 - which artifacts are manually maintained,
 - which executable sources are authoritative,
 - refresh order,
 - reviewer expectations,
 - common mistakes,
 - escalation rules,
 - stop conditions.
- Document governance maintenance for all areas including (where applicable):
- test inventory,
 - convergence CI inventory,
 - protected replay manifests,
 - split-owner reports,
 - generated documentation,
 - ownership ledgers,
 - compatibility register,
 - future contract registries.
- Repository Impact
- Expected additions:
- Governance Refresh Workflow documentation.
 - Cross-links from existing governance entry points where appropriate.
- Do not modify:
- production code,
 - governance tests,
 - CI behavior,
 - generated artifacts,
 - replay schemas,
 - compatibility implementations,
 - ownership semantics,
 - runtime behavior.
- Verification
- Verify that:
- every governed artifact has a documented refresh trigger,
 - authoritative sources are identified,
 - generated artifacts are distinguished from canonical sources,
 - no governance behavior changed,
 - documentation is discoverable from existing governance references.
- Required Deliverable
- Produce:
- AR-B0_governance_refresh_workflow_implementation.md
- Include:
1. Executive Summary
 2. Governance Workflow Overview
 3. Governed Artifact Inventory
 4. Refresh Triggers
 5. Authority Mapping
 6. Refresh Sequence
 7. Reviewer Checklist
 8. Verification Results
 9. Risks
 10. Package Closeout
- Answer:
- Was the governance workflow successfully implemented?
 - Are governance refresh expectations now centrally documented?
 - Can future contributors determine refresh requirements without repository-wide investigation?
11. Campaign Assessment
- Determine whether Campaign 5 implementation should continue with the next infrastructure package or whether governance workflow requires refinement before proceeding.
12. Recommended Next Cycle
- Recommended one of:
- Proceed to Package 4
 - Refine Governance Workflow
 - Pause implementation pending new evidence
- Support the recommendation using repository evidence.
13. Files Required for External Review
- Separate into:
- Required
- Optional
- Success Criteria
- The repository has a centralized governance refresh workflow describing how governance artifacts are maintained without changing governance enforcement or runtime behavior.
- Future contributors should be able to determine governance maintenance requirements from this workflow rather than rediscovering repository practices.
- Stop Condition
- Stop immediately after:
- implementing the Governance Refresh Workflow,
 - verifying documentation,
 - producing the implementation closeout.
- Do not automate governance.
- Do not modify governance behavior.
- Do not alter runtime architecture.

- AR-BU — Package 7 Implementation: Ruleset Contract Registry
- Goal
- Implement the Governance Refresh Workflow identified during Campaign 5.
- The objective is to make governance maintenance predictable by documenting when repository governance artifacts must be refreshed, which artifacts are authoritative, and how contributors determine whether a change requires governance updates.
- This package improves governance workflow.
- It does not change governance behavior.
- Architectural Premise
- Assume:
- Campaigns 1-4 recovered the architecture.
 - Package 1 implemented the Feature-Lane Verification Guide.
 - Package 2 implemented the Compatibility-Residue Register.
 - Existing governance checks, inventories, manifests, and reports remain architecturally correct.
- This package documents how governance artifacts are maintained.
- It does not alter governance policy.
- Implementation Scope
- Create a Governance Refresh Workflow document that explains:
- how governance artifacts require refresh,
 - which artifacts are generated,
 - which artifacts are manually maintained,
 - which executable sources are authoritative,
 - refresh order,
 - reviewer expectations,
 - common mistakes,
 - escalation rules,
 - stop conditions.
- Document governance maintenance for all areas including (where applicable):
- test inventory,
 - convergence CI inventory,
 - protected replay manifests,
 - split-owner reports,
 - generated documentation,
 - ownership ledgers,
 - compatibility register,
 - future contract registries.
- Repository Impact
- Expected additions:
- Governance Refresh Workflow documentation.
 - Cross-links from existing governance entry points where appropriate.
- Do not modify:
- production code,
 - governance tests,
 - CI behavior,
 - generated artifacts,
 - replay schemas,
 - compatibility implementations,
 - ownership semantics,
 - runtime behavior.
- Verification
- Verify that:
- every governed artifact has a documented refresh trigger,
 - authoritative sources are identified,
 - generated artifacts are distinguished from canonical sources,
 - no governance behavior changed,
 - documentation is discoverable from existing governance references.
- Required Deliverable
- Produce:
- AR-B0_governance_refresh_workflow_implementation.md
- Include:
1. Executive Summary
 2. Governance Workflow Overview
 3. Governed Artifact Inventory
 4. Refresh Triggers
 5. Authority Mapping
 6. Refresh Sequence
 7. Reviewer Checklist
 8. Verification Results
 9. Risks
 10. Package Closeout
- Answer:
- Was the governance workflow successfully implemented?
 - Are governance refresh expectations now centrally documented?
 - Can future contributors determine refresh requirements without repository-wide investigation?
11. Campaign Assessment
- Determine whether Campaign 5 implementation should continue with the next infrastructure package or whether governance workflow requires refinement before proceeding.
12. Recommended Next Cycle
- Recommended one of:
- Proceed to Package 4
 - Refine Governance Workflow
 - Pause implementation pending new evidence
- Support the recommendation using repository evidence.
13. Files Required for External Review
- Separate into:
- Required
- Optional
- Success Criteria
- The repository has a centralized governance refresh workflow describing how governance artifacts are maintained without changing governance enforcement or runtime behavior.
- Future contributors should be able to determine governance maintenance requirements from this workflow rather than rediscovering repository practices.
- Stop Condition
- Stop immediately after:
- implementing the Governance Refresh Workflow,
 - verifying documentation,
 - producing the implementation closeout.
- Do not automate governance.
- Do not modify governance behavior.
- Do not alter runtime architecture.

- The register becomes the planning surface for all future compatibility cleanup.
- Implementation Scope
- Create a Compatibility-Residue Register.
- For every compatibility surface identified throughout Campaigns 1-5, record:
- compatibility surface,
 - canonical owner,
 - repository location,
 - current purpose,
 - downstream consumers,
 - replay implications,
 - provenance implications,
 - governance implications,
 - known tests,
 - supporting architectural evidence,
 - retirement prerequisites,
 - estimated lifetime,
 - current status.
- Compatibility categories should include (where applicable):
- legacy vocabulary,
 - compatibility aliases,
 - compatibility adapters,
 - temporary bridges,
 - historical payload formats,
 - import re-exports,
 - fallback compatibility,
 - replay compatibility,
 - serialization compatibility,
 - API compatibility,
 - test compatibility.
- Repository Impact
- Expected additions:
- Compatibility-Residue Register document.
 - Cross-reference from existing architecture or maintenance documentation if appropriate.
- Do not modify:
- production code,
 - compatibility implementations,
 - replay schemas,
 - provenance behavior,
 - governance logic,
 - tests,
 - generated artifacts,
 - runtime behavior.
- Verification
- Verify that:
- every compatibility item has an owner,
 - every item has evidence,
 - every item has a retirement decision state,
 - every item has replay/provenance classification,
 - no compatibility behavior changed.
- Required Deliverable
- Produce:
- AR-B0_compatibility_residue_register_implementation.md
- Include:
1. Executive Summary
 2. Register Overview
 3. Compatibility Inventory
 4. Ownership Mapping
 5. Consumer Mapping
 6. Replay / Provenance Assessment
 7. Retirement Readiness
- Classify each item:
- Active
 - Transitional
 - Awaiting Evidence
 - Retirement Candidate
 - Permanent Compatibility
 - 8. Verification Results
 - 9. Risks
 - 10. Package Closeout

Answer:

 - Was the register successfully implemented?
 - Are compatibility surfaces now centrally discoverable?
 - Is compatibility retirement now easily searchable?

11. Campaign Assessment

Determine whether Campaign 5 implementation should continue with Campaign 6 or whether additional refinement of Package 5 is required.

12. Recommended Next Cycle

Recommended one of:

 - Proceed to Package 3
 - Expand Compatibility Register
 - Pause implementation pending new evidence

Support the recommendation using repository evidence.

13. Files Required for External Review

Separate into:

Required

Optional

Success Criteria

The repository now has one authoritative inventory describing every active compatibility surface without changing compatibility behavior.

Future compatibility retirement should be possible using this register without requiring another repository-wide discovery effort.

Stop Condition

Stop immediately after:

 - creating the Compatibility-Residue Register,
 - verifying completeness,
 - producing the implementation closeout.

Do not retire compatibility.

Do not refactor compatibility.

Do not modify runtime behavior.

- AR-BR — Package 4 Implementation: Generated Documentation Appendices
- Goal
- Implement the Generated Documentation Appendices package by defining and introducing repository documentation that is generated from authoritative executable registries wherever appropriate.
- The objective is to reduce documentation duplication while reinforcing the existing ownership doctrine that executable registries remain the canonical source of truth.
- This package documents generation strategy.
- It does not automate generation.
- Architectural Premise
- Assume:
- Campaigns 1-4 recovered the architecture.
 - Package 1 implemented the Feature-Lane Verification Guide.
 - Package 2 implemented the Compatibility-Residue Register.
 - Package 3 implemented the Governance Refresh Workflow.
 - Existing executable registries remain authoritative.
- Documentation should increasingly point to executable sources instead of duplicating information.
- Implementation Scope
- Document and introduce generated-document appendices for repository surfaces where executable registries already exist.
- For each candidate appendix identify:
- canonical executable source,
 - generated documentation destination,
 - ownership,
 - refresh trigger,
 - verification method,
 - reviewer expectations,
 - maintenance status.
- Candidate areas include (where applicable):
- Final Emission taxonomy,
 - protected replay field registry,
 - split-owner acceptance matrix summaries,
 - governance inventories,
 - compatibility summaries,
 - future backend contract registry,
 - version/provenance registries.
- Repository Impact
- Expected additions:
- Generated Documentation Appendices document.
 - Cross-links from Governance Refresh Workflow where appropriate.
 - Documentation templates or appendix placeholders where justified.
- Do not modify:
- production code,
 - executable registries,
 - generation,
 - CI behavior,
 - governance behavior,
 - replay schemas,
 - runtime behavior,
 - generated outputs.
- Verification
- Verify that:
- every appendix identifies its authoritative executable source,
 - generated documentation is clearly distinguished from manually maintained documentation,
 - refresh responsibilities are documented,
 - reviewer expectations are defined,
 - no executable authority is moved into documentation.
- Required Deliverable
- Produce:
- AR-BR_generated_documentation_appendices_implementation.md
- Include:
1. Executive Summary
 2. Appendix Inventory
 3. Authority Mapping
 4. Generated vs Manual Documentation Classification
 5. Refresh Responsibilities
 6. Verification Strategy
 7. Reviewer Guidelines
 8. Verification Results
 9. Risks
 10. Package Closeout
- Answer:
- Were generated documentation appendices successfully introduced?
 - Are executable registries still the canonical authority?
 - Can future generated documentation be expanded without architectural change?
11. Campaign Assessment
- Determine whether Campaign 5 should proceed to the next implementation package or

9. **UNCERTAIN**
Insufficient evidence exists to classify the cost safely.
Also assign:
• permanence: permanent, transitional, historical, or uncertain,
• reduction potential: none, low, medium, high, or uncertain,
• architectural risk if changed: low, medium, high, or unknown,
• likely mechanism: documentation, tooling, automation, helper API, contract publication, future improvement, code generation, adapter improvement, compatibility retirement, workflow clarification, or none.

Required Deliverable
Create:
AR-BJ_coordination_cost_inventory_and_workflow_discovery.md
The document must contain:
1. Executive Summary
Summarize:
• the overall shape of coordination cost,
• whether the architecture appears to be the primary source of friction,
• the largest implementation-cost concentrations, and
• and whether any immediate architectural concern was discovered.

2. Architectural Baseline
Briefly restate the Campaigns 1–4 doctrine that constrains this investigation. Explicitly identify architectural decisions that must not be casually repeated.
3. Repository Areas Examined
List:
• source directories,
• major modules,
• tests,
• architecture documents,
• compatibility areas,
• adapters,
• registries/contracts,
• and any repository history examined.
4. Representative Feature-Change Traces
For each representative feature:
[Feature or Change | Canonical Owner | Files/Modules Touched | Contracts Affected | Replay/Provenance Impact | Tests | Documentation | Compatibility/Governance | Initial Assessment]
5. Coordination Cost Inventory
Use a table with at least:
[ID | Coordination Cost | Repository Evidence | Current Workflow Impact | Responsible Boundary | Primary Classification | Permanence | Reduction Potential | Risk if Changed | Notes]
6. Edit-Fan-Map
Document repeated multi-file change patterns.
For each pattern, distinguish necessary propagation from duplicated implementation knowledge.
7. Canonical-Owner Pressure Assessment
Use a table:
[Canonical Owner | Owned Responsibility | Common Change Triggers | Extension Surface | Pressure Source | Anticipatory Expected? | Possible Implementation | Risk |
8. Contract Publication Assessment
Use a table:
[Contract | Current Location | Publication Form | Discoverability | Duplicated/Inferred Elsewhere? | Coordination Effect | Improvement Potential]
9. Compatibility and Transitional Residue Inventory
Use a table:
[Residue | Protected Behavior | Active Callers/Tests | Coordination Cost | Retirement Evidence Required | Expected Lifetime | Preliminary Priority]
10. Vocabulary Duplication Assessment
Use a table:
[Vocabulary/Concept | Locations | Same Meaning? | Translation Boundary? | Duplication Type | Canonical Definition Present? | Action Needed?]
11. Adapter-Boundary Assessment
Use a table:
[Adapter | Source Authority | Target Representation | Mapping Style | Feature-Change Cost | Duplicated Decisions? | Boundary Value | Improvement Potential]
12. Governance Cost Assessment
Use a table:
[Governance Mechanism | Failure Prevented | Enforcement Method | Manual Effort | Overlap | Current Value | Possible Efficiency Improvement]
13. Test Maintenance Assessment
Use a table:
[Test Area | Maintenance Trigger | Coverage Value | Friction Source | Compatibility-Driven? | Safer Improvement Opportunity]
14. Documentation Burden Assessment
Use a table:
[Document/Artifact | Purpose | Authority Level | Update Trigger | Duplicates Executable Truth? | Historical Value | Coordination Cost | Possible Treatment]
15. Current Feature Implementation Workflow
Describe the observed end-to-end workflow.
Clearly mark:
• required architectural steps,
• manual or repetitive steps,
• compatibility steps,
• governance steps,
• and end points dependent on developer memory.

16. Preliminary Coordination Cost Profile
Group findings under:
• intentional architectural tradeoffs,
• already-minimized concerns,
• implementation inefficiencies,
• compatibility residue,
• governance overhead,
• test-maintenance friction,
• documentation synchronization,
• and candidate simplification opportunities.

17. Evidence Gaps
List anything that cannot yet be determined.
For each gap, identify:
• missing repository evidence,
• why the gap matters,
• exact files, history, examples, or runtime evidence needed,
• and whether the next cycle can proceed without it.

18. Recommended Next Cycle
Recommend the next Campaign 5 cycle based on evidence.
Do not automatically propose implementation.
The next cycle should likely deepen classification and quantify the highest-cost coordination paths before prescribing changes.
Candidate direction:
AR-BK — Coordination Cost Classification and Reduction Opportunity Analysis
Only recommend a different direction if the repository evidence strongly justifies it.

19. Files Required for External Review
End with a concise list of the smallest useful file set that should be passed back for generation of the next instruction block.

Separate the list into:
Required:
• the completed AR-BJ document,
• any newly created supporting inventory or trace artifacts,
• and files containing evidence that materially affects the next-cycle recommendation.

Optional:
• large source files already quoted sufficiently in AR-BJ,
• historical campaign documents used only as background,
• and supplemental test output.

Evaluation Constraints
• Do not modify production code.
• Do not refactor architecture.
• Do not remove compatibility behavior.
• Do not weaken governance.
• Do not rename concepts merely to reduce vocabulary count.
• Do not equate file count with poor design.
• Do not recommend collapsing canonical owners into callers.
• Do not move authoritative decisions into adapters, APIs, projections, tests, or UI-facing layers.
• Do not test replay, provenance, evidence, or ownership checks as incidental overhead.
• Do not assume every repeated value should be globally centralized.
• Do not propose code generation unless the generated artifact would have a clear canonical source and verification path.
• Do not claim simplification is safe without explaining which architectural guarantees remain intact.
• Prefer classification and evidence gathering over premature recommendations.
• Preserve uncertainty where repository evidence is insufficient.

Success Criteria
This cycle succeeds when it provides:
• a repository-grounded map of where coordination occurs,
• several representative feature-change traces,
• a defensible distinction between necessary coordination and implementation friction,
• an inventory of compatibility and governance costs,
• an assessment of contract publication and vocabulary duplication,
• and a clear account of test and documentation maintenance burden,
• and a sufficiently precise evidence package to support the next Campaign 5 cycle.

Stop Condition
Stop after producing the discovery document and any minimal supporting inventories required by it.
Do not begin broad refactoring, tooling implementation, contract consolidation, compatibility retirement, or governance changes during this cycle.

Execution Constraints
• Do not modify production code.
• Do not implement documentation changes.
• Do not update governance.
• Do not publish contracts.
• Do not introduce builders.
• Do not automate tooling.
• Produce only an implementation specification.

Success Criteria
This cycle succeeds when the first implementation initiative is completely specified, independently reviewable, and ready for execution without requiring additional architectural planning.

Stop Condition
Stop after producing the package-definition document.
Do not begin implementation.
Do not define Package 2 unless it is necessary to justify Package 1.
Do not reopen Campaigns 1–4 architectural decisions.

AR-BT — Package 6 Implementation: Backend Contract Registry
Goal
Implement the Backend Contract Registry by creating a centralized inventory of backend-facing architectural contracts.
The objective is to document the canonical backend interfaces, ownership boundaries, compatibility expectations, versioning status, and verification requirements without changing backend behavior.
This package establishes backend contract governance.
It does not introduce new backends or modify existing backend implementations.

Architectural Premise
Assume:
• Campaigns 1–4 recovered the architecture.
• Packages 1–5 established workflow guidance, governance maintenance, compatibility inventory, generated documentation strategy, and reduced split-owner maintenance.
• Backend behavior remains stable.
• Future backend extensibility depends on explicit contract definitions rather than implicit conventions.

Implementation Scope
Create a Backend Contract Registry documenting every backend-facing contract currently present in the repository.
For each contract identify:
• contract identifier,
• canonical owner,
• repository location,
• purpose,
• consumers,
• compatibility requirements,
• replay implications,
• provenance implications,
• versioning status,
• verification method,
• future extensibility notes.

Document backend contract categories where applicable, including:
• model routing,
• backend adapters,
• provider interfaces,
• response normalization,
• backend configuration,
• backend capability negotiation,
• backend diagnostics,
• backend fallback boundaries.

Repository Impact
Expected additions:
• Backend Contract Registry document.
• Cross-links from governance and architecture documentation where appropriate.

Do not modify:
• backend implementations,
• runtime routing,
• provider logic,
• replay schemas,
• provenance behavior,
• governance policy,
• CI behavior,
• production code unrelated to documentation.

Verification
Verify that:
• every backend contract has a canonical owner,
• every contract identifies its authoritative implementation,
• version status is documented,
• compatibility expectations are recorded,
• no backend behavior changed.

Required Deliverable
Produce:
AR-BT_backend_contract_registry_implementation.md
Include:
1. Executive Summary
2. Backend Contract Inventory
3. Ownership Mapping
4. Consumer Mapping
5. Compatibility Assessment
6. Version Status
7. Verification Strategy
8. Verification Results
9. Risks
10. Package Closeout

Answer:
• Was the Backend Contract Registry successfully implemented?
• Are backend contracts now centrally discoverable?
• Can future backend expansion proceed using explicit architectural contracts?

11. Campaign Assessment
Determine whether Campaign 5 should proceed to the next implementation package or whether backend contract coverage requires refinement.

12. Recommended Next Cycle
Recommend one of:
• Proceed to Package 7
• Expand Backend Contract Coverage
• Pause pending repository evidence

Support the recommendation using repository evidence.
13. Files Required for External Review
Separate into:
Required:
Optional:
Success Criteria
The repository contains a centralized Backend Contract Registry that documents existing backend contracts without altering backend behavior or architectural boundaries.
Future backend implementations should be able to integrate using this registry rather than rediscovering repository conventions.

Stop Condition
Stop immediately after:
• implementing the Backend Contract Registry,
• verifying documentation,
• producing the implementation closeout.

Do not implement new backends.
Do not modify backend routing.
Do not alter runtime behavior.

• verifying documentation,
• producing the implementation closeout.
Do not automate governance.
Do not modify governance behavior.
Do not alter runtime architecture.

AR-BU — Package 7 Implementation: Ruleset Contract Registry
Goal
Implement the Ruleset Contract Registry by creating a centralized inventory of ruleset-facing architectural contracts.
The objective is to document the canonical interfaces, ownership boundaries, lifecycle expectations, compatibility requirements, and verification strategy for ruleset integration without modifying gameplay behavior.
This package establishes ruleset contract governance.
It does not introduce new rulesets or change existing gameplay logic.

Architectural Premise
Assume:
• Campaigns 1–4 recovered the architecture.
• Packages 1–6 established workflow guidance, governance maintenance, compatibility inventory, generated documentation strategy, split-owner maintenance improvements, and backend contract governance.
• Existing gameplay behavior is stable.
• Future ruleset extensibility depends on explicit contracts rather than implicit conventions.

Implementation Scope
Create a Ruleset Contract Registry documenting every ruleset-facing contract currently present in the repository.
For each contract identify:
• contract identifier,
• canonical owner,
• authoritative implementation,
• repository location,
• purpose,
• consumers,
• lifecycle expectations,
• compatibility requirements,
• replay implications,
• provenance implications,
• version status,
• extensibility method,
• extensibility notes.

Document ruleset contract categories where applicable, including:
• ruleset discovery,
• ruleset loading,
• rule evaluation,
• state authority,
• realization boundaries,
• validation interfaces,
• repair interfaces,
• diagnostics,
• compatibility boundaries.

Repository Impact
Expected additions:
• Ruleset Contract Registry document.
• Cross-links from governance and architecture documentation where appropriate.

Do not modify:
• gameplay behavior,
• ruleset implementations,
• replay schemas,
• provenance behavior,
• runtime routing,
• governance policy,
• CI behavior,
• production code unrelated to documentation.

Verification
Verify that:
• every ruleset contract has a canonical owner,
• authoritative implementations are identified,
• compatibility expectations are documented,
• replay/provenance implications are classified,
• version status is recorded,
• no gameplay behavior changed.

Required Deliverable
Produce:
AR-BU_ruleset_contract_registry_implementation.md
Include:
1. Executive Summary
2. Ruleset Contract Inventory
3. Ownership Mapping
4. Consumer Mapping
5. Compatibility Assessment
6. Lifecycle and Version Status
7. Verification Strategy
8. Verification Results
9. Risks
10. Package Closeout

Answer:
• Was the Ruleset Contract Registry successfully implemented?
• Are rule contracts now centrally discoverable?
• Can future rulesets integrate using explicit architectural contracts?

11. Campaign Assessment
Determine whether Campaign 5 should proceed to the next implementation package or whether ruleset contract coverage requires refinement.

12. Recommended Next Cycle
Recommend one of:
• Proceed to Package 8
• Expand Ruleset Contract Coverage
• Pause pending repository evidence

Support the recommendation using repository evidence.
13. Files Required for External Review
Separate into:
Required:
Optional:
Success Criteria
The repository contains a centralized Ruleset Contract Registry documenting existing ruleset contracts without altering gameplay behavior or architectural boundaries.
Future rulesets should be able to integrate using the registry rather than rediscovering repository conventions.

Stop Condition
Stop immediately after:
• implementing the Ruleset Contract Registry,
• verifying documentation,
• producing the implementation closeout.

Do not implement new rulesets.
Do not modify gameplay logic.
Do not alter runtime behavior.

8. Verification Results
9. Risks
10. Package Closeout
Answer:
• Were generated documentation appendices successfully introduced?
• Are executable registries still the canonical authority?
• Can future generated documentation be expanded without architectural change?

11. Campaign Assessment
Determine whether Campaign 5 should proceed to the next implementation package or whether documentation generation requires refinement.

12. Recommended Next Cycle
Recommend one of:
• Proceed to Package 5
• Expand Generated Documentation Coverage
• Pause pending repository evidence

Support the recommendation using repository evidence.
13. Files Required for External Review
Separate into:
Required:
Optional:
Success Criteria
The repository contains a centralized Version and Provenance Contract Registry documenting existing versioning and provenance contracts without altering runtime behavior or architectural boundaries.
Future architectural evolution should be able to rely on explicit version and provenance governance rather than rediscovering repository conventions.

Stop Condition
Stop immediately after:
• implementing the Version and Provenance Contract Registry,
• verifying documentation,
• producing the implementation closeout.

Do not introduce new versioning behavior.
Do not modify provenance generation.
Do not alter runtime behavior.

8. Verification Results
9. Risks
10. Package Closeout
Answer:
• Were generated documentation appendices successfully introduced?
• Are executable registries still the canonical authority?
• Can future generated documentation be expanded without architectural change?

11. Campaign Assessment
Determine whether Campaign 5 should proceed to the next implementation package or whether documentation generation requires refinement.

12. Recommended Next Cycle
Recommend one of:
• Proceed to Package 5
• Expand Generated Documentation Coverage
• Pause pending repository evidence

Support the recommendation using repository evidence.
13. Files Required for External Review
Separate into:
Required:
Optional:
Success Criteria
The repository clearly distinguishes executable authority from generated documentation.
Future documentation can be expanded from executable registries without duplicating repository knowledge or changing governance behavior.

Stop Condition
Stop immediately after:
• implementing the Generated Documentation Appendices package,
• verifying documentation,
• producing the implementation closeout.

Do not automate documentation generation.
Do not modify executable registries.
Do not change runtime or governance behavior.

AR-BV — Package 8 Implementation: Version and Provenance Contract Registry
Goal
Implement the Version and Provenance Contract Registry by creating a centralized inventory of repository versioning and provenance contracts.
The objective is to document how versioned artifacts, provenance metadata, replay-sensitive outputs, and architectural lineage are governed without modifying runtime behavior.
This package establishes version and provenance governance.
It does not introduce new versioning behavior or alter provenance generation.

Architectural Premise
Assume:
• Campaigns 1–4 recovered the architecture.
• Packages 1–7 established workflow guidance, governance maintenance, compatibility inventory, generated documentation strategy, split-owner maintenance improvements, backend contracts, and ruleset contracts.
• Existing provenance and replay systems remain architecturally correct.
• Future architectural evolution should rely on explicit provenance contracts rather than implicit repository conventions.

Implementation Scope
Create a Version and Provenance Contract Registry documenting all versioned and provenance-sensitive architectural surfaces.
For each contract identify:
• contract identifier,
• canonical owner,
• authoritative implementation,
• repository location,
• purpose,
• consumers,
• versioning policy,
• provenance requirements,
• replay implications,
• compatibility expectations,
• verification method,
• future extensibility notes.

Document contract categories where applicable, including:
• replay versioning,
• realization provenance,
• final outcome provenance,
• schema versioning,
• contract publication,
• generated artifact lineage,
• compatibility lineage,
• diagnostic lineage,
• future registry versioning.

Repository Impact
Expected additions:
• Version and Provenance Contract Registry document.
• Cross-links from governance documentation where appropriate.

Do not modify:
• provenance generation,
• replay schemas,
• runtime behavior,
• backend behavior,
• gameplay behavior,
• governance enforcement,
• CI behavior,
• production code unrelated to documentation.

Verification
Verify that:
• every provenance-sensitive contract has a canonical owner,
• authoritative implementations are identified,
• version status is documented,
• replay implications are classified,
• compatibility expectations are recorded,
• no runtime behavior changed.

Required Deliverable
Produce:
AR-BV_version_and_provenance_contract_registry_implementation.md
Include:
1. Executive Summary
2. Contract Inventory
3. Ownership Mapping
4. Consumer Mapping
5. Versioning Assessment
6. Provenance Assessment
7. Verification Strategy
8. Verification Results
9. Risks
10. Package Closeout

Answer:
• Was the Version and Provenance Contract Registry successfully implemented?
• Are provenance contracts now centrally discoverable?
• Can future versioned architectural changes proceed using explicit contracts?

11. Campaign Assessment
Determine whether Campaign 5 should proceed toward implementation closeout or whether additional registry work is required.

12. Recommended Next Cycle
Recommend one of:
• Proceed to Campaign 5 Closeout
• Expand Version and Provenance Coverage
• Pause pending repository evidence

Support the recommendation using repository evidence.
13. Files Required for External Review
Separate into:
Required:
Optional:
Success Criteria
The repository contains a centralized Version and Provenance Contract Registry documenting existing versioning and provenance contracts without altering runtime behavior or architectural boundaries.
Future architectural evolution should be able to rely on explicit version and provenance governance rather than rediscovering repository conventions.

Stop Condition
Stop immediately after:
• implementing the Version and Provenance Contract Registry,
• verifying documentation,
• producing the implementation closeout.

Do not introduce new versioning behavior.
Do not modify provenance generation.
Do not alter runtime behavior.

AR-BW — Campaign 5 Implementation Closeout
Goal
Perform the formal closeout of Campaign 5 (Coordination Cost Reduction).
Determine whether the implementation packages collectively achieved the campaign objectives established during AR-BJ through AR-BU.
Do not perform additional implementation.
Do not introduce new infrastructure.
Evaluate the completed work.

Architectural Premise
Campaign 5 consisted of:
Planning:
• AR-BJ — Coordination Cost Inventory
• AR-BK — Classification
• AR-BL — Workflow Optimization
• AR-BM — Initiative Prioritization
• AR-BN — Package Definition

Implementation:
• AR-BQ — Feature-Lane Verification Guide
• AR-BP — Compatibility Residue Register
• AR-BR — Governance Refresh Workflow
• AR-BS — Generated Documentation Appendices
• AR-BT — Split-Owner Count & Report Automation
• AR-BU — Backend Contract Registry
• AR-BV — Ruleset Contract Registry
• AR-BW — Version & Provenance Contract Registry

Campaign Closeout Objectives
Evaluate whether the implementation accomplished the original campaign goals.
Specifically determine:
• Have coordination costs been measurably reduced?
• Have duplicated architectural responsibilities been reduced?
• Are canonical ownership boundaries clearer?
• Are canonical ownership boundaries clearer?
• Are extension points now explicitly documented?

- Can contributors navigate repository governance without rediscovery?
- Have future backend/ruleset/version extensions become substantially easier?
- Did implementation preserve runtime architecture?
- Were any planned initiatives left incomplete?

Repository Assessment

Assess:

- documentation discoverability,
- governance workflow,
- compatibility planning,
- generated documentation strategy,
- split-owner maintenance,
- backend contracts,
- ruleset contracts,
- version/provenance contracts,
- remaining architectural debt.

Required Deliverable

Produce:

AR-BW_campaign5_coordination_cost_reduction_closeout.md

Include:

1. Executive Summary
 2. Campaign Objectives Review
- Evaluate each original Campaign 5 objective.

Classify:

- Fully Achieved
 - Mostly Achieved
 - Partially Achieved
 - Deferred
3. Package Review

Summarise every implementation package.

Document:

- objective,
 - repository impact,
 - architectural value,
 - remaining limitations.
4. Coordination Cost Assessment

Evaluate whether:

- duplicated maintenance decreased,
 - rediscovery effort decreased,
 - ownership ambiguity decreased,
 - architectural navigation improved.
5. Remaining Architectural Debt

Identify unresolved work.

Separate into:

- intentional future work,
 - optional enhancements,
 - genuine gaps.
6. Metrics

Assess:

- Documentation Discoverability
- Governance Clarity
- Compatibility Management
- Extension Readiness
- Architectural Consistency
- Coordination Cost Reduction

Provide qualitative ratings and supporting repository evidence.

7. Risks

Identify any remaining long-term risks.

8. Campaign Verdict

Answer:

Should Campaign 5 be considered complete?

Support the conclusion with repository evidence.

9. Recommendation

Recommend exactly one:

- Close Campaign 5 and proceed to Campaign 6.
- Perform a small targeted Campaign 5 follow-up.
- Reopen Campaign 5 due to unmet objectives.

10. Files Required for External Review

Separate into:

Required

Optional

Success Criteria

Produce a justified assessment of whether Campaign 5 fulfilled its stated objectives.

Do not implement additional infrastructure.

Do not expand repository scope.

Determine whether the campaign is complete.

Stop Condition

Stop immediately after producing the campaign closeout.

Do not begin Campaign 6.

AR-BX — Chassis Strategy Discovery and Evidence Inventory

Campaign

Campaign 6 — Chassis Strategy

Objective

Establish the repository evidence needed to define Ashen Thrones' long-term architectural chassis. This cycle must determine which structural decisions are sufficiently mature to become stable architectural doctrine, which systems are intentional extension mechanisms, which responsibilities should remain internal implementation details, and what elements of future architectural evolution may still be appropriate. Do not redesign the architecture. Treat the current repository as an intentional, production-ready foundation recovered, reconciled, and operationalized through Campaigns 1-5. Assume existing structural decisions are valid unless repository evidence demonstrates a specific long-term limitation, unresolved contradiction, or recurring coordination cost. The purpose of this cycle is discovery and classification, not implementation.

Required Mindset

Do not search for a theoretically clearer architecture merely because alternatives exist. Evaluate whether the current chassis:

- preserves clear responsibility ownership,
- supports feature growth through intuitive extension mechanisms,
- keeps product concerns separate from runtime mechanics,
- preserves replay, evidence, and provenance guarantees,
- avoids unnecessary cross-package coordination,
- distinguishes public contracts from internal implementation,
- and allows future work to proceed primarily as implementation rather than repeated architectural reconsideration.

Produce persistence where existing structure is coherent and adequate. Recommend evolution only when supported by concrete repository evidence.

Primary Questions

- Investigate and answer:
- Which architectural boundaries already function as stable doctrine?
 - Which systems are explicitly or implicitly designed as extension points?
 - Which modules, helpers, registers, adapters, and orchestration paths are internal implementation details?
 - Where should future gameplay systems attach?
 - Where should new AI backends or providers attach?
 - Where should new rulesets, rule variants, or game-mode policies attach?
 - How do replay, provenance, diagnostics, evidence, validation, repair, fallback, and realization participate in the long term chassis?
 - Which repository structures are architectural contracts, and which are organizational conveniences?
 - Which extensible contracts are authoritative?
 - Which documentation artifacts govern architecture, and how are they kept consistent with executable behavior?
 - Which current systems are expected to remain stable even as their implementations evolve?
 - Which architectural areas are intentionally expected to evolve?
 - What foreseeable feature work can be completed without changing foundational architecture?
 - Where could future feature attachment accidentally increase coordination costs?
 - Does the current package and repository organization adequately express the recovered architecture?
 - Is another major architectural campaign likely to be necessary before controlled feature expansion?

Required Investigation

1. Recover Prior Campaign Conclusions

Locate and review the evidence and synthesis artifacts from Campaigns 1-5, especially documents covering:

- recovered architectural doctrine,
- final-vision compatibility,
- concept reconciliation,
- boundary reconciliation,
- coordination-cost reduction,
- ownership,
- public contracts,
- internal implementation boundaries,
- backend extension,
- ruleset extension,
- version and provenance governance,
- executable versus documentation authority, and
- repository-level architectural consistency.

At minimum, locate the most authoritative campaign closures and any active doctrine, ownership ledger, concept map, boundary inventory, contract registry, or conformance assessment created or updated during those campaigns. Do not assume filenames beyond those present in the repository.

2. Map the Current Repository Shape

Inventory the current package and repository organization. Identify:

- top-level packages and directories,
- product-facing entry points,
- runtime entry points,
- gameplay or domain packages,
- orchestration layers,
- backend/provider packages,
- ruleset packages,
- replay and provenance infrastructure,
- validation and repair systems,
- diagnostics and evidence systems,
- public API or contract modules,
- registries,
- adapters,
- test-support packages,
- documentation directories,
- generated artifacts,
- configuration surfaces,
- and legacy or compatibility areas.

For each major area, briefly state its apparent architectural responsibility. Do not produce a raw file dump. Group files by architectural role.

3. Identify Stable Boundary Shapes

Find boundaries that are repeatedly reinforced by:

- imports,
- dependency direction,
- public APIs,
- protocol or interface definitions,
- registries,
- factory functions,
- adapters,
- orchestration ownership,
- tests,
- documentation,
- or campaign conclusions.

Candidate categories may include:

- product/runtime separation,
- ruleset/runtime separation,
- backend/provider abstraction,
- replay projection,
- realization authority,
- provenance ownership,
- validation boundaries,
- repair boundaries,
- fallback ownerships,
- diagnostics and evidence ownership,
- API emission,
- state ownership,
- configuration ownership,
- and serialization or persistence boundaries.

For each candidate stable boundary, record:

- the boundary,
- the owning module or package,
- the evidence supporting it,
- whether it is executable, documented, or both,
- known exceptions,
- and the apparent consequence of violating it.

4. Identify Extension Mechanisms

Locate mechanisms through which new capabilities are intended to enter the system. Investigate:

- backend/provider registration,
- backend capability declarations,
- ruleset registration,
- ruleset contracts,
- version negotiation,
- provenance contracts,
- public protocols or interfaces,
- factories,
- adapters,
- dependency-injection points,
- configuration-driven selection,
- event or hook systems,
- plugin-like mechanisms,
- gameplay-system attachment points, and
- content loading boundaries.

For each mechanism, determine:

- what may be extended,
- who owns the extension contract,
- what an extension is allowed to provide,
- what it must not own,
- how it is selected,
- how its behavior is validated,
- and whether replay remains valid,
- and whether the extension requires exit to central orchestration code.

Distinguish true extension mechanisms from ordinary internal implementation.

5. Identify Internal Implementation Details

Locate systems that should not become general-purpose public extension surfaces. Candidates may include:

- sanitizers,
- repair helpers,
- fallback internals,
- normalization utilities,
- orchestration helpers,
- compatibility shims,
- defensive parsing,
- transport-specific logic,
- temporary adapters,
- projection helpers,
- evidence assembly,
- internal registries,
- or test-only abstractions.

For each candidate, determine:

- why it exists,
- who owns it,
- whether external code currently depends on it,
- whether documentation incorrectly presents it as public,

AR-BY — Chassis Doctrine and Stability Decision Matrix

Campaign

Campaign 6 — Chassis Strategy

Objective

Using the evidence gathered during AR-BX, formally establish the long-term architectural doctrine of Ashen Thrones. This cycle reverses repository observations into explicit architectural policy. Do not perform another repository inventory. Do not redesign the architecture. Instead, determine which architectural decisions are now considered permanent doctrine, which extension mechanisms define the intended future growth path, which implementation details must remain internal, and which architectural areas intentionally remain open to future evolution. The resulting doctrine should become the reference point for future feature development so contributors no longer need to repeatedly reconsider foundational architectural questions.

Required Mindset

Campaigns 1-5 recovered and stabilized the architecture. AR-BX demonstrated that the repository now consistently reflects that architecture. This cycle therefore assumes the current architecture is correct unless repository evidence from AR-BX demonstrates otherwise. The burden of proof now shifts: Architectural change should require justification. Architectural stability should be the default. Avoid creating new abstractions, generalized plugin systems, or hypothetical future frameworks. The goal is architectural confidence—not architectural novelty.

Inputs

- Use:
- AR-BX_chassis_strategy_discovery_and_evidence_inventory.md
 - Campaign 1-5 closure documents
 - Architecture Ownership Ledger
 - Feature Lane Verification guide
 - Backend Contract Registry
 - Ruleset Contract Registry
 - Version & Provenance Contract Registry
 - Version & Refresh Contract Registry
 - Compatibility Residue Register
- Do not repeat repository discovery unless AR-BX explicitly marked evidence as unresolved.

Primary Questions

- Investigate and answer:
- Which architectural boundaries are now permanent doctrine?
 - Which boundaries define the long-term chassis?
 - Can future AI capabilities attach without changing the chassis?
 - Can alternate rulesets attach without redesign?
 - Can future tooling attach without creating new ownership ambiguity?
 - Can replay, provenance, diagnostics, and governance continue expanding independently?
 - Does repository organization still communicate architectural intent?
 - Would another architectural campaign meaningfully reduce future coordination cost?
 - What architectural work were frozen today, what implementation work could proceed safely?
 - Which future changes would legitimately require architectural review?
 - Is Campaign 6 ready to close?

Required Analysis

1. Confirm Stable Doctrine

Review every Stable Doctrine candidate identified in AR-BX. For each:

- confirm,
- reject,
- or downgrade.

Every decision must cite AR-BX evidence. Once confirmed, treat these as long-term architectural doctrine rather than provisional findings.

2. Finalize Extension Mechanisms

Review every extension mechanism. Classify each as:

- Primary Extension Point
- Secondary Extension Point
- Specialized Extension Point
- Internal Only
- Future Only

Explain:

- why it exists,
- who owns it,
- what may attach,
- what may never attach,
- and whether it should remain stable indefinitely.

3. Finalize Internal Boundaries

Review every internal implementation area. Determine:

- whether it should remain permanently internal,
- whether documentation should explicitly discourage external use,
- whether future contributors could accidentally mistake it for an extension point, and
- whether additional architectural guidance is warranted.

Do not recommend publishing additional APIs unless AR-BX demonstrated a real need.

4. Separate Architecture from Implementation

Using AR-BX findings, classify foreseeable work into:

- Pure implementation
- Local architectural evolution
- Major architectural reconsideration

Future gameplay systems should overwhelmingly fall into the first category. If not, explain why.

5. Define Long-Term Architectural Principles

Produce a concise doctrine describing how future work should interact with the chassis. Include guidance such as:

- preserve ownership,
- preserve dependency direction,
- extend contracts rather than orchestration,
- keep replay observational,
- preserve provenance,
- avoid central coordination,
- protect internal implementation details,
- maintain executable authority,
- and evolve architecture deliberately.

These should become repository-level architectural principles.

6. Identify Remaining Evolution Areas

Review AR-BX's deliberate evolution areas. Determine:

- whether they are product evolution,
- implementation evolution,
- architectural evolution,
- or optional future work.

Clearly distinguish optional evolution from architectural debt.

7. Assess Architectural Confidence

Determine whether future contributors can safely assume:

- stable repository organization,
- stable dependency direction,
- stable ownership,
- stable extension paths,
- stable governance,
- and stable feature attachment strategy.

If so, explain why. If not, identify the remaining uncertainty.

8. Determine Remaining Architectural Horizon

Answer: Is another major architectural campaign likely before large-scale feature development? If yes:

- explain exactly why.

If no:

- explain why the repository is ready to transition toward implementation-focused work.

Required Deliverable

Create: AR-BY_chassis_dctrine_and_stability_decision_matrix.md
The document must contain the following sections:

1. Executive Summary

Summarize the long-term architectural posture.

2. Permanent Architectural Doctrine

Use a table: | Architectural Boundary | Decision | Rationale | Supporting Evidence |
Only include doctrine expected to remain stable for years.

3. Official Extension Strategy

Use a table: | Extension Point | Classification | Intended Usage | Ownership | Expected Stability |

4. Internal Architectural Boundaries

Use a table: | Internal Area | Why It Remains Internal | Risk If Exposed | Recommendation |

5. Architecture vs Implementation Matrix

Use a table: | Future Work Area | Implementation | Local Evolution | Major Architecture | Notes |

6. Long-Term Chassis Principles

Produce approximately 10-20 concise architectural principles. These should read like enduring repository doctrine.

7. Remaining Evolution Areas

Identify only those areas intentionally expected to evolve. Do not include ordinary feature implementation.

8. Chassis Stability Assessment

Evaluate:

- ownership clarity,
- dependency stability,
- extension maturity,
- governance maturity,
- coordination resistance,
- or implementation readiness.

9. Architectural Confidence Statement

Answer: Can future contributors confidently build upon this chassis without repeatedly revisiting foundational architecture? Support the answer using AR-BX findings.

10. Recommended Next Cycle

Recommend the next Campaign 6 cycle.

AR-BZ — Chassis Validation and Future Evolution

Assessment

Campaign

Campaign 6 — Chassis Strategy

Objective

Validate that the architectural doctrine established during AR-BY is sufficient to support long-term development without requiring repeated architectural reconsideration. This cycle is not intended to discover new architecture. It is intended to stress-test the doctrine established by Campaigns 1-6 against realistic future development. Determine whether the current chassis genuinely allows years of implementation work without increasing coordination cost or whether any remaining structural uncertainty still warrants another architectural campaign. This is the final analytical validation before Campaign 6 closure.

Required Mindset

Assume the architectural doctrine established in AR-BY is correct. The purpose of this cycle is not to find improvements. Instead, attempt to falsify the conclusion that the chassis is complete. Only recommend further architectural work when repository evidence demonstrates that future implementation would repeatedly encounter unresolved structural decisions. Do not invent hypothetical future requirements. Use only realistic future work supported by the repository and the architectural vision established during Campaigns 1-6.

Inputs

- Use:
- AR-BX_chassis_strategy_discovery_and_evidence_inventory.md
 - AR-BY_chassis_dctrine_and_stability_decision_matrix.md
 - Campaign 1-5 closure documents
 - Architecture Ownership Ledger
 - Feature Lane Verification
 - Backend Contract Registry
 - Ruleset Contract Registry
 - Version & Provenance Contract Registry
 - Governance Refresh Workflow
 - Compatibility Residue Register
- No repository discovery should be performed unless necessary to validate a specific conclusion.

Primary Questions

- Determine:
- Can ordinary gameplay systems now be implemented without architectural reconsideration?
 - Can future AI capabilities attach without changing the chassis?
 - Can alternate rulesets attach without redesign?
 - Can future tooling attach without creating new ownership ambiguity?
 - Can replay, provenance, diagnostics, and governance continue expanding independently?
 - Does repository organization still communicate architectural intent?
 - Would another architectural campaign meaningfully reduce future coordination cost?
 - Is architectural work were frozen today, what implementation work could proceed safely?
 - Which future changes would legitimately require architectural review?
 - Is Campaign 6 ready to close?

Required Analysis

Evaluate representative future work including:

- new gameplay mechanics
- new world systems
- additional AI providers
- alternate rulesets
- richer UI
- debugging tools
- replay tools
- performance improvements
- diagnostics
- campaign content

- For each:
- expected attachment path,
 - architectural boundaries crossed,
 - contracts used,
 - ownership changes required,
 - coordination cost,
 - whether architecture changes.

2. Search for Remaining Architectural Friction

Attempt to identify:

- recurring ownership ambiguity,
- unclear extension mechanisms,
- unstable doctrine,
- dependency uncertainty,
- duplicate authority,
- coordination hotspots,
- unclear package responsibilities,
- or doctrine contradictions.

Only report issues supported by repository evidence.

3. Validate Campaign Goals

Evaluate whether Campaign 6 has achieved:

- stable doctrine,
- stable extension mechanisms,
- stable implementation boundaries,
- implementation-first future work,
- low coordination cost,
- long-term chassis readiness.

4. Validate Future Campaign Strategy

Determine whether future campaigns should primarily consist of:

- implementation,
- feature development,
- gameplay iteration,
- AI capability expansion,
- unclear package responsibilities,
- balancing,
- content creation,
- or contract evolution,

rather than architectural recovery.

5. Determine Remaining Architectural Horizon

If another architectural campaign is recommended:

- identify:
- objective,
- justification,
- expected benefit,
- and why implementation cannot safely contain without it.

Otherwise explain why Campaign 6 should conclude.

Required Deliverable

Create: AR-BZ_chassis_validation_and_future_evolution_assessment.md
The document must contain:

1. Executive Summary

2. Chassis Stress-Test Results

Use a table: | Future Capabilities | Attachment Path | Architecture Change Required | Coordination Cost | Assessment |

3. Remaining Architectural Friction

Only evidence-backed findings.

4. Campaign Objective Validation

Explain whether Campaign 6 successfully achieved its intended goals.

5. Future Development Readiness

Assess:

- gameplay
- AI
- tooling
- content
- governance
- replay
- or product evolution

6. Remaining Architectural Horizon

State whether another architectural campaign is justified.

7. Campaign 6 Closure Readiness

Explicitly answer:

- Ready for closure?
- One additional doctrine cycle recommended?
- Significant architectural work still required?

Support the answer with repository evidence.

8. Recommended Next Cycle

Recommend either:

- Campaign 6 Closure
- or one narrowly scoped final doctrine cycle.

Constraints

Do not:

- redesign architecture,
- introduce new abstractions,
- recommend speculative frameworks,
- reopen Campaigns 1-5,
- duplicate AR-BX,
- or weaken doctrine established by AR-BY without evidence.

Completion Standard

This cycle is complete when:

- the chassis has been stress-tested,
- future implementation has been validated,
- remaining architectural uncertainty has been identified,
- implementation readiness has been assessed,
- and Campaign 6 closure readiness has been objectively determined.

Final Response

After creating the document, respond with:

- the generated filename,
- whether the chassis passed validation,
- any remaining architectural concerns,
- whether another architectural campaign is recommended,
- and whether Campaign 6 should proceed to closure.

Do not generate the closureout during this cycle.

AR-CA — Campaign 6 Chassis Strategy Closureout

Campaign

Campaign 6 — Chassis Strategy

Objective

Formally conclude the Chassis Strategy campaign. Using the findings from AR-BX, AR-BY, and AR-BZ, determine whether the long-term architectural chassis of Ashen Thrones is sufficiently established that future development can proceed without repeatedly reconsidering foundational structural decisions. This document is not intended to introduce new architecture. It is the formal architectural closureout for Campaign 6.

Required Mindset

Assume the recovered architecture is now stable. The purpose of this closureout is to summarize what has become long-term architectural doctrine, identify the officially supported extension mechanisms, distinguish architecture from implementation, and determine whether the Architecture Reconciliation era has reached its intended conclusion. Do not reopen architectural questions unless repository evidence from Campaign 6 demonstrates that they remain unresolved.

Inputs

- Use:
- AR-BX_chassis_strategy_discovery_and_evidence_inventory.md
 - AR-BY_chassis_dctrine_and_stability_decision_matrix.md
 - AR-BZ_chassis_validation_and_future_evolution_assessment.md
- Reference prior campaign closureouts only when necessary for historical context. Do not perform additional repository discovery.

Required Analysis

1. Evaluate Campaign Objectives

Determine whether Campaign 6 successfully established:

- permanent architectural doctrine,
- stable architectural boundaries,
- approved extension mechanisms,
- internal implementation boundaries,
- long-term feature attachment strategy,
- implementation-first development posture,
- long-term coordination resistance.

2. Answer the Campaign Closureout Questions

Provide explicit answers to:

- Which architectural boundaries should now be considered effectively permanent?
- What future AI capabilities are the intended extension points for future capabilities?
- Which architectural responsibilities should remain internal implementation details?
- What future work represents implementation rather than architecture?
- Which architectural areas are intentionally expected to evolve over time?
- Does the current chassis support long-term expansion without increasing coordination cost?
- Should future campaign focus primarily on implementation, or is another major architectural campaign still warranted?

Support every answer using Campaign 6 evidence.

3. Summarize the Final Chassis

Produce a concise description of the architectural chassis that now governs Ashen Thrones. Summarize:

- runtime ownership,
- domain ownership,
- state authority,
- C1K,
- prompt adaptation,
- backend/provider boundaries,
- Final Emission,
- persistence,
- replay,
- provenance,
- diagnostics,
- governance,
- compatibility,
- and extension strategy.

This summary should describe the architecture as it now exists—not as it might someday become.

4. Define the Architectural Doctrine Going Forward

Summarize the enduring architectural principles established during Campaigns 1-6. Focus on:

- ownership,
- dependency direction,
- extension through contracts,
- implementation over redesign,
- executable authority,
- replay independence,
- provenance integrity,
- governance of extensions,
- and deliberate architectural evolution.

5. Define the Future Development Posture

Identify which categories of future work should now dominate repository evolution. Examples include:

- gameplay implementation,
- content,
- AI capability,
- tools,
- diagnostics,
- balancing,
- UI,
- product polish,
- testing,
- performance,
- gameplay iteration,
- AI capability expansion,
- tooling,
- balancing,
- content creation,
- or contract evolution,

Explain why these are implementation activities rather than architectural campaigns.

6. Determine the Remaining Architectural Horizon

Identify only those future changes that legitimately justify architectural review. Do not include ordinary feature work.

Examples may include:

- alternate ruleset architecture,
- additional backend architecture,
- hosted/public API architecture,
- multi-user runtime,
- migration framework,
- or other product-triggered structural evolution.

7. Evaluate the Architecture Reconciliation Era

Assess Campaigns 1-6 collectively. Summarize how the architecture evolved across the campaigns, including:

- recovery,
- vision alignment,
- concept reconciliation,
- boundary reconciliation,
- coordination-cost reduction,
- or gameplay stabilization.

Evaluate whether the original Architecture Reconciliation roadmap achieved its intended goals.

Required Deliverable

Create: AR-CA_campaign6_chassis_strategy_closureout.md
The document must contain:

1. Executive Summary

2. Campaign Objective Assessment

| Objective | Result | Evidence |

3. Campaign Closureout Questions

Answer each explicitly.

4. Final Chassis Summary

A concise description of the permanent architecture.

5. Long-Term Architectural Doctrine

Summarize the enduring doctrine established by Campaigns 1-6.

6. Future Development Posture

Describe the implementation-first direction of the repository.

7. Remaining Architectural Horizon

List only evidence-backed future architectural triggers.

8. Campaign 6 Outcome

State yes or no:

- Continue Campaign 6
- or Campaign 6 Successfully Closed

9. Architecture Reconciliation Era Assessment

Evaluate the success of Campaigns 1-6 as a whole.

10. Recommendations

Recommend the next major campaign. Do not generate its implementation plan.

Constraints

Do not:

- introduce new architecture,
- redesign packages,
- reopen Campaigns 1-5,
- speculate about hypothetical future systems,
- or weaken established doctrine without evidence.

Treat the architecture as production-ready unless Campaign 6 evidence demonstrates otherwise.

Completion Standard

This closureout is complete when:

- Campaign 6 objectives have been evaluated,
- all closureout questions have been answered,
- the permanent chassis has been summarized,
- future implementation posture has been established,
- remaining architectural triggers have been identified,
- the Architecture Reconciliation era has been assessed,
- and a recommendation has been made for the next major campaign.

Final Response

After creating the document, respond with:

- projection helpers,
 - evidence assembly,
 - internal registries,
 - or test-only abstractions.
- For each candidate, determine:
- why it exists,
 - who owns it,
 - whether external code currently depends on it,
 - and what documentation incorrectly presents it as public,
 - and what architectural risk would result from treating it as an extension point.
- 6. Evaluate Feature Attachment Paths**

Use repository evidence to trace how representative future capabilities would attach to the current chassis. Do not implement these capabilities.

- Evaluate at least the following hypothetical examples:
1. A new gameplay subsystem that modifies worldstate behavior.
 2. A new ruleset with materially different domain policies.
 3. A new AI backend or model provider.
 4. A richer player-facing product or UI.
 5. A new persistence, replay inspection, or debugging tool.
 6. A new content package or campaign type.
 7. A new diagnostic or observability capability.

- For each example, identify:
- the expected attachment point,
 - the contracts it would use,
 - the internal systems it should not bypass,
 - the packages likely to change,
 - whether foundational architecture would need to change,
 - and where coordination cost could emerge.

These traces should test the chassis, not propose detailed feature designs.

7. Evaluate Package and Repository Organization

Determine whether the physical repository organization adequately communicates the logical architecture.

- Look for:
- packages containing unrelated responsibilities,
 - public and internal modules mixed together,
 - extension contracts hidden inside implementation packages,
 - circular or counter-directional dependencies,
 - central modules that require edits for every extension,
 - duplicated contract definitions,
 - documentation located far from the executable authority it describes,
 - test organization that obscures responsibility ownership,
 - ambiguous package names,
 - and legacy locations that no longer represent current doctrine.

Classify each issue as one of:

- no issue,
- naming or documentation concern,
- implementation cleanup,
- packaging improvement,
- deliberate future evolution,
- or foundational architectural concern.

Do not recommend repository movement merely for aesthetic consistency.

8. Evaluate Executable and Documentation Governance

Identify the authoritative source for architectural rules.

Determine:

- which contracts are enforced by executable code,
- which are enforced by tests,
- which exist only in documentation,
- which documentation is descriptive,
- which documentation is normative,
- how version and provenance contracts are governed,
- how architectural drift would be detected,
- and whether documentation duplicates executable truth unnecessarily.

Note any cases where architectural doctrine depends on prose that could silently diverge from runtime behavior.

9. Evaluate Long-Term Modularity

Assess whether the current chassis supports long-term growth without turning every feature into a cross-cutting change.

Look for evidence concerning:

- dependency direction,
- ownership concentration,
- split ownership,
- contract stability,
- registry centralization,
- package independence,
- orchestration complexity,
- replay compatibility,
- provenance continuity,
- backend substitution,
- ruleset substitution,
- and product/runtime independence.

Identify where modularity is:

- already structural,
- dependent on conventions,
- dependent on documentation,
- incompletely enforced,
- or intentionally unnecessary.

10. Distinguish Permanence, Extension, Implementation, and Evolution

Classify major architectural areas into four preliminary categories:

Stable Doctrine

Boundaries or responsibilities that should be treated as effectively permanent unless major product requirements change.

Expected Extension Mechanisms

Contracts intentionally designed to accept new backends, rulesets, gameplay capabilities, tooling, content, or product surfaces.

Future Implementation Work

Work that should occur within the existing chassis and does not justify architectural reconsideration.

Deliberate Evolution Areas

Areas that may reasonably evolve over time but should change through explicit architectural decisions rather than incidental feature work.

A fifth category may be used when necessary:

Unresolved

Evidence is insufficient or contradictory.

Do not force certainty where the repository does not support it.

Evidence Standards

Every substantive conclusion must cite concrete repository evidence.

Acceptable evidence includes:

- source files,
- package boundaries,
- imports,
- protocols,
- interfaces,
- registries,
- type definitions,
- factories,
- orchestration paths,
- tests,
- architectural documentation,
- prior campaign artifacts,
- and version-control-visible compatibility mechanisms.

For each important finding, include relevant file paths and symbol names when available.

Clearly distinguish:

- directly observed behavior,
 - documented intent,
 - test-enforced behavior,
 - and interpretation.
- Do not present interpretation as executable fact.

Constraints

Do not:

- implement features,
- refactor production code,
- rename or move packages,
- introduce new frameworks,
- create speculative plugin systems,
- generalize internal helpers into public abstractions,
- reopen decisions resolved by Campaigns 1-5 without contradictory evidence,
- recommend architectural change solely for elegance,
- treat every replaceable function as an extension point,
- or assume that future growth requires greater abstraction.

Small investigative scripts or static-analysis commands may be used, but they must not modify repository behavior.

Required Deliverable

Create:

AR-BX_chassis_strategy_discovery_and_evidence_inventory.md

The document must contain the following sections.

1. Executive Summary

Summarize:

- whether the repository appears ready for long-term chassis definition,
- the strongest candidate stable boundaries,
- the clearest extension mechanisms,
- the most important internal implementation boundaries,
- and any areas requiring deliberate further analysis.

2. Prior Campaign Doctrine Recovered

List the relevant conclusions from Campaigns 1-5 that constrain this campaign.

For each conclusion, include its source document.

3. Repository Architectural Map

Provide a grouped map of the repository by architectural responsibility.

Include major packages, entry points, contracts, orchestration areas, infrastructure, documentation, and tests.

4. Stable Boundary Candidates

Use a table with columns:

| Boundary | Owning Area | Executable Evidence | Documentation Evidence | Exceptions or Ambiguities | Preliminary Classification |

5. Extension Mechanism Inventory

Use a table with columns:

| Extension Mechanism | Extensible Capability | Governing Contract | Selection or Registration Path | Validation / Replay / Provenance Obligations | Central Changes Required? | Maturity |

6. Internal Implementation Boundary Inventory

Use a table with columns:

| Internal Area | Responsibility | Current Consumers | Why It Should Remain Internal | Leakage or Misuse Risk | Evidence |

7. Representative Feature Attachment Traces

For each required hypothetical capability, describe:

- entry or attachment point,
- expected dependency path,
- contracts involved,
- protected internal boundaries,
- likely implementation locations,
- and architectural implications.

8. Package and Repository Organization Assessment

Document any mismatch between physical organization and logical architecture.

- implementation readiness.

9. Architectural Confidence Statement

Answer:

Can future contributors confidently build upon this chassis without repeatedly revisiting foundational architecture?

Support the answer using AR-BX findings.

10. Recommended Next Cycle

Recommend the next Campaign 6 cycle.

It should be significantly narrower than AR-BX.

Constraints

Do not:

- redesign packages,
- invent new abstractions,
- create speculative frameworks,
- recommend generalized plugin systems,
- reopen Campaigns 1-5,
- duplicate AR-BX discovery,
- or recommend change solely because alternatives exist.

Architecture should now become increasingly conservative.

Completion Standard

This cycle is complete when:

- permanent doctrine has been distinguished from observation,
- extension mechanisms have been formally approved,
- implementation has been separated from architecture,
- long-term chassis principles have been documented,
- remaining architectural evolution has been narrowly scoped,
- and future implementation can rely upon the doctrine without repeatedly reopening foundational design.

Final Response

After creating the document, respond with:

1. the generated filename,
2. the permanent doctrine established,
3. the approved extension mechanisms,
4. any remaining architectural uncertainty,
5. and the recommended title and objective of the next cycle.

Do not begin the next cycle.

Final Response

After creating the document, respond with:

1. the generated filename,
2. whether the chassis passed validation,
3. any remaining architectural concerns,
4. whether another architectural campaign is recommended,
5. and whether Campaign 6 should proceed to closure.

Do not generate the closure during this cycle.

- all closure questions have been answered,
- the permanent chassis has been summarized,
- future implementation posture has been established,
- remaining architectural triggers have been identified,
- the Architecture Reconciliation era has been assessed,
- and a recommendation has been made for the next major campaign.

Final Response

After creating the document, respond with:

1. the generated filename,
2. whether Campaign 6 is officially closed,
3. the overall Chassis Readiness assessment,
4. the recommended next major campaign,
5. and a brief explanation of why the Architecture Reconciliation era is complete.

Do not begin the next campaign.

Separate architectural concerns from naming, documentation, and cleanup concerns.

9. Executable and Documentation Governance Assessment

Identify the authoritative source of truth for major architectural contracts and any drift risks.

10. Long-Term Modularity and Coordination Assessment

Evaluate whether future expansion can occur without restoring the split ownership, duplicated contracts, or cross-layer coordination reduced during Campaign 5.

11. Preliminary Chassis Classification

Use a table with columns:

| Architectural Area | Stable Doctrine | Expected Extension Mechanism | Future Implementation Work | Deliberate Evolution Area | Unresolved | Rationale |

Mark the most appropriate category for each area.

12. Risks and Unknowns

List only evidence-backed uncertainties that could materially affect the chassis strategy.

Do not fill this section with generic future risks.

13. Recommended Next Cycle

Recommend the narrowest useful next cycle for Campaign 6.

State:

- the questions it should resolve,
- the evidence it should use,
- whether it is analysis-only or may include limited implementation,
- and why it is necessary before campaign closeout.

Completion Standard

This cycle is complete when:

- the repository's major architectural areas have been mapped,
- candidate permanent boundaries have been identified,
- intended extension mechanisms have been distinguished from internal indirection,
- representative future feature paths have been traced,
- physical repository structure has been compared with logical architecture,
- executable and documentation governance has been assessed,
- future implementation has been distinguished from architectural evolution,
- evidence gaps have been explicitly recorded,
- and the next Campaign 6 cycle can be defined without guessing.

Final Response

After creating the document, respond with:

1. the path of the generated file,
2. a concise summary of the strongest findings,
3. any files or repository areas that could not be inspected,
4. whether additional source files are required for the next cycle,
5. and the recommended title and objective of the next cycle.

Do not begin the next cycle.

From https://github.com/yiqi-g/69a228-e17081d1a45e5f0a37a4777c75d3a7a0f_2632f83a-8105-2558524816

Naming Convention
Use PR-AA, PR-AB, PR-AC... through PR-AZ, then PR-BA, PR-BB, etc.
PR identifies the Product Realization era.
The suffix identifies the individual cycle.

Estimated Length: Variable (continuous)
Starting Cycle: PR-AA

Exercise the recovered architecture by implementing foundational gameplay capabilities through the approved ownership boundaries and feature lanes.

Implementation packages begin expanding product capability without requiring architectural redesign.

Implementation Readiness

Deepen the game's simulation by expanding mechanics, world interaction, NPC behavior, combat, non-combat systems, progression, and player agency.

Gameplay Depth

Improve narrative quality, AI behavior, prompt adaptation, pacing, context utilization, and player experience without violating architectural doctrine.

Player Experience

Expand authored content, campaigns, scenes, encounters, world lore, and content pipelines using the established content architecture.

Content Throughput

Improve usability, UX, diagnostics, performance, tooling, persistence, compatibility cleanup, testing, and developer workflow.

Product Quality

Evaluate genuinely new product requirements that exceed the current chassis.

- Additional AI providers
- Alternate rulesets
- Hosted multiplayer
- Public APIs
- Modding support
- Campaign migration
- Cross-device play

Major product evolution occurs deliberately without compromising the architecture established during the Reconciliation Era.

Evolution Readiness

Campaign 6

Hosted Experiences

That progression feels consistent with the discipline you've applied throughout the project, and it gives Version 1.0 the significance it deserves rather than treating it as just another checkpoint on an endless implementation campaign.

Each campaign should end when its primary product capability has reached a level where additional work produces diminishing strategic returns and the next capability becomes the highest-leverage investment.

Campaign Review Questions

- 1. What product capability was added?
- 2. Did implementation follow architectural doctrine?
- 3. Did any implementation reveal architectural weaknesses?
- 4. What product capability should come next?
- 5. Should the roadmap adapt?

Era Mindset

Assume the architecture is complete.

Future work should maximize player value rather than architectural elegance.

Architecture exists to enable product development, not compete with it.

Prefer implementation over redesign.

Prefer ownership over centralization.

Prefer deliberate evolution over speculative abstraction.

Every completed feature should strengthen confidence that the architecture established during the Architecture Reconciliation era was the correct long-term foundation.